

臺灣自編大學教科書

大模型導論

Introduction to Large Language Models

第 3 篇

Transformer 從零實作

Transformers from Scratch

第 10 章 詞嵌入、語意空間與位置表示

第 11 章 自注意力、因果遮罩與多頭注意力

第 12 章 Transformer Block 與現代 GPT 元件

第 13 章 從零建構 MiniFormosaGPT 與文字生成器

李世淵（機器人叫獸） 編著

助教 李天宇、李宇晴

2026 年 8 月 第一版

序言 把黑盒子拆開，然後自己裝回去

前兩篇你做了很多事：定義需求、盤點臺灣的處境、補齊數學、建好環境、清理語料、訓練 tokenizer、確認授權。但你還沒有寫過任何一行「模型」的程式碼。

這一冊就是要把它補上。四章之後，你會有一個從第一行開始都是自己寫的語言模型——沒有 `from transformers import`，沒有任何一塊是別人的黑盒子。

為什麼要自己寫

這是一個合理的問題。現成的函式庫又快又穩，多數實務工作也不需要你重寫注意力。那為什麼要花四章的時間？

第一個理由是診斷能力。當你的訓練 loss 突然變成 NaN、當模型生成的文字不斷重複、當載入別人的權重後輸出變成亂碼——這些情況沒有錯誤訊息可以搜尋，你必須知道裡面在做什麼才能判斷問題出在哪裡。第 13 章整整一節在講八種失敗模式，那些都是真實會遇到的。

第二個理由是判斷力。第六篇之後你會不斷面對選擇：要用 GQA 還是 MHA、詞彙表要多大、要不要綁定權重、位置編碼該選哪一種。這些選擇沒有標準答案，只有取捨——而理解取捨的前提是理解機制。

第三個理由比較難量化，但我認為最重要：親手做過一次，「大模型」這件事就不再神祕。它會從一個令人敬畏的黑盒子，變成一個你知道每一層在做什麼、也知道它為什麼有這些限制的工程系統。這種祛魅對一個工程師是必要的。

四章的結構是一條直線

這是全書唯一一段「每一章的產出都是下一章的輸入」的連續實作，中間跳過任何一章，後面都會卡住。

第 10 章處理輸入端：把詞元變成向量，並注入位置資訊。核心是 RoPE——你會用程式驗證它的一個漂亮性質：注意力分數只依賴相對位置，與絕對位置無關。

第 11 章是全書的心臟。從無參數的簡化注意力開始，逐步加上可訓練投影、縮放、因果遮罩、多頭，共四個版本。這一章你會親眼看到「除以根號 d 」這個看似多餘的動作，如何決定 softmax 會不會飽和、模型能不能訓練起來。

第 12 章把注意力裝進骨架：正規化、殘差連接、前饋網路，以及把它們組裝起來的順序。這一章的主題是「疊四十層而不崩潰」，而那件事比想像的難。

第 13 章把一切接起來訓練。這一章的重點不是新知識，而是整合與除錯——你會遇到的問題八成不是「不懂原理」，而是「某個地方接錯了」。

你會遇到的困難

本篇的難度高於前兩篇，但難的地方可能與你預期的不同。

真正的困難不在數學。第 11 章的注意力只是幾次矩陣乘法加一次 softmax，第 12 章的殘差連接只是一個加號。困難在於張量的形狀——那個 (B, H, T, d_h) 到底哪一維是什麼、transpose 之後為什麼要 contiguous、為什麼 view 會報一個看起來莫名其妙的錯。

所以第 3 章的形狀推理在這一冊會真正派上用場。我在每一段程式碼中都寫了形狀斷言，並在第 11.5.2 節列出多頭注意力的完整八步形狀鏈。強烈建議你把那張表抄在紙上——第 13 章組裝模型時卡住的人，幾乎都是這一步沒做扎實。

第二個困難是「錯了但不會報錯」。本篇有好幾處這樣的陷阱：因果遮罩寫錯會讓 loss 異常漂亮、位移做了兩次會讓模型學得很慢卻仍在下降、兩層共用同一個正規化會讓效果略差卻查不出原因。我為每一個都設計了對應的測試，而那些測試本身就是本篇的重要產出。

沒有 GPU 也能完整走完

這一點值得單獨說明。本篇的所有實作——包括第 13 章的完整訓練——都能在一般筆電上完成。

tiny 版本（約 200 萬參數、8,000 詞彙表）在 CPU 上訓練約需三小時，一個晚上跑得完，最後會生成通順但語意跳躍的繁體中文短句。那不是簡化版或體驗版，而是完整的訓練流程：同樣的程式碼、同樣的資料管線、同樣的除錯經驗，只是模型小一點。

第 13.7.3 節列出了對 tiny 模型該有的合理期待。請先讀過它——把期待設定對，你才不會誤以為自己做錯了。一個 200 萬參數的模型不會回答問題、不會有知識、也不會遵守指令，那些需要更大的規模與後訓練（第六篇）。本篇的成功標準是「文字看起來像中文」。

關於用語

本書一律使用臺灣的專業用語與教育部標準用字。以下是本冊特別容易混淆的幾組：

本書用語	對應英文	說明與區別
推論	inference	指模型執行、產生輸出的階段。

本書用語	對應英文	說明與區別
推理	reasoning	指模型的思考與邏輯推導能力，如「推理模型」。
函式	function (程式)	程式中的可呼叫單元，如 forward 函式。
函數	function (數學)	數學上的映射，如三角函數、激勵函數。
正規化	normalization	涵蓋層正規化與 Unicode 正規化。不使用「歸一化」。
標準化	standardization	統計上的減均值除標準差，見第 11.3.2 節。
最佳化	optimization	不使用「優化」。最佳化器即 optimizer。
激勵函數	activation function	不使用「激活函數」。
遮罩	mask	不使用「掩碼」。因果遮罩即 causal mask。
殘差連接	residual connection	不使用「殘差連結」。
元件	component	模型的組成部分，如 Transformer block 的元件。
實作	implement	寫成程式碼。不使用「實現」指涉寫程式。
詞元 / 分詞器	token / tokenizer	不使用「標記」「令牌」。
嵌入	embedding	詞元嵌入、位置嵌入皆用此詞。
前饋網路	feed-forward network	不使用「前饋神經網絡」。
快取	cache	KV cache 依慣例保留英文。
介面	interface	不使用「接口」。
資料	data	不使用「數據」。資料集即 dataset。

英文專有名詞在首次出現時並列原文，其後視情況直接使用英文（例如 Transformer、RoPE、SwiGLU、KV cache）——這反映臺灣工程現場的真實習慣。

李世淵（機器人叫獸）

2026 年 8 月 於雲林

目 錄

序言 把黑盒子拆開，然後自己裝回去

目錄

全書架構總覽

第 10 章 詞嵌入、語意空間與位置表示

Embeddings, Semantic Spaces, and Positional Representations

學習目標

10.1 從詞元到向量：嵌入層

10.2 語意空間：從 Word2Vec 到情境化表示

10.3 位置編碼：為沒有順序概念的注意力補上順序

10.4 RoPE：現代模型的標準選擇

10.5 輸出投影與權重綁定

10.6 詞彙擴充後的嵌入初始化

10.7 嵌入視覺化：看得見，但別看錯

程式實作

系統案例：臺灣專有名詞語意檢索原型

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 11 章 自注意力、因果遮罩與多頭注意力

Self-Attention, Causal Masking, and Multi-Head Attention

學習目標

11.1 為什麼需要注意力

11.2 版本一：沒有可訓練權重的簡化注意力

11.3 版本二：縮放點積注意力

11.4 版本三：因果遮罩

11.5 版本四：多頭注意力

11.6 注意力變體：MHA、MQA 與 GQA

11.7 複雜度、記憶體牆與 FlashAttention

程式實作

系統案例：可互動的注意力教學元件

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 12 章 Transformer Block 與現代 GPT 元件

Transformer Blocks and Modern GPT Components

學習目標

12.1 正規化：LayerNorm 與 RMSNorm

12.2 Pre-LN 與 Post-LN：一個順序決定成敗

12.3 前饋網路：GELU 與 SwiGLU

12.4 組裝：Transformer Block

12.5 參數量與 FLOPs 的精確計算

12.6 從 GPT-2 到 Llama-style：元件替換的動機

12.7 初始化與正則化的實務取捨

程式實作

系統案例：MiniFormosaGPT 三種尺寸的規劃

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

第 13 章 從零建構 MiniFormosaGPT 與文字生成器

Building MiniFormosaGPT and a Text Generator from Scratch

學習目標

13.1 組裝完整模型

13.2 損失計算：三個容易錯的地方

13.3 資料管線：從語料到批次

13.4 最小可行訓練迴圈

13.5 生成：從模型到文字

13.6 八種常見的訓練失敗

13.7 在筆電上跑完一次完整訓練

程式實作

系統案例：從亂碼到成句的訓練縮時紀錄

章末評量

研究延伸與版本注意事項

常見問題

教師使用建議

本章小結

成果檢核

索引

以下為 Word 自動目錄欄位。在 Word 中開啟後，於下方按右鍵選擇「更新功能變數」即可產生含頁碼的三層目錄；亦可使用左側導覽窗格依書籤瀏覽。

序言 把黑盒子拆開，然後自己裝回去	2
為什麼要自己寫	2
四章的結構是一條直線	2
你會遇到的困難	3
沒有 GPU 也能完整走完	3
關於用語	3
目 錄	5
全書架構總覽	13
本冊四章的實作遞進	13
第 3 篇	15
詞嵌入、語意空間與位置表示	16
學習目標	16
10.1 從詞元到向量：嵌入層	16
10.1.1 one-hot 與查表是同一件事	16
10.1.2 參數量與記憶體	17
10.1.3 為什麼要乘以維度的平方根	18
10.2 語意空間：從 Word2Vec 到情境化表示	18
10.2.1 分布式假說與它的兩種實現	18
10.2.2 餘弦相似度與語意檢索	19
10.2.3 各向異性：一個會誤導你的現象	20
10.2.4 嵌入層在訓練中的特殊性	20
10.3 位置編碼：為沒有順序概念的注意力補上順序	22
10.3.1 為什麼需要位置資訊	22
10.3.2 四種主要做法	22
10.3.3 可學習位置嵌入	22
10.3.4 正弦位置編碼	23
10.4 RoPE：現代模型的標準選擇	24
10.4.1 核心想法：用旋轉表示位置	24
10.4.2 為什麼旋轉有效：相對位置性質	24
10.4.3 實作	24
10.4.4 RoPE 的長度外推	25
10.4.5 ALiBi：另一條路線	26

10.4.6	三種位置編碼的實測對照	27
10.4.7	一個容易被忽略的細節：RoPE 的維度配對.....	27
10.5	輸出投影與權重綁定	28
10.5.1	嵌入層的鏡像	28
10.5.2	綁定的利弊	28
10.5.3	softmax 之前的最後一步	29
10.6	詞彙擴充後的嵌入初始化	30
10.6.1	問題與四種策略.....	30
10.6.2	完整實作.....	30
10.6.3	擴充之後必須做的事.....	31
10.7	嵌入視覺化：看得見，但別看錯	31
10.7.1	三種降維方法.....	31
10.7.2	四個常見的誤讀.....	32
10.7.3	臺灣語料上的視覺化實務	33
	程式實作	35
	程式 10A 嵌入層與三種位置編碼.....	35
	程式 10B 臺灣專有名詞語意地圖	36
	程式 10C 詞彙擴充的初始化實驗.....	37
	系統案例：臺灣專有名詞語意檢索原型	38
	系統規格	38
	評測設計	39
	交付要求	39
	章末評量	41
	一、概念題.....	41
	二、計算題.....	41
	三、除錯題.....	41
	四、系統設計題.....	42
	五、臺灣情境與倫理題.....	42
	評量提示（給自學者）	42
	研究延伸與版本注意事項	42
	常見問題.....	43
	教師使用建議	43
	本章小結	44
	成果檢核.....	45
	自注意力、因果遮罩與多頭注意力.....	46
	學習目標.....	46
11.1	為什麼需要注意力.....	46
11.1.1	資訊瓶頸的三種形式.....	46
11.1.2	注意力的三個角色	47

11.2 版本一：沒有可訓練權重的簡化注意力	47
11.2.1 最樸素的做法	48
11.2.2 它有什麼問題	48
11.3 版本二：縮放點積注意力	49
11.3.1 三個投影	49
11.3.2 為什麼要除以根號 d ：一個實測	49
11.3.3 實作	50
11.4 版本三：因果遮罩	51
11.4.1 為什麼要遮	51
11.4.2 三種實作方式	51
11.4.3 必須通過的驗證	52
11.4.4 dropout 放在哪裡	53
11.5 版本四：多頭注意力	54
11.5.1 為什麼要多頭	54
11.5.2 形狀變換：本章最容易寫錯的地方	55
11.5.3 完整實作	56
11.5.4 用 PyTorch 的融合實作	57
11.6 注意力變體：MHA、MQA 與 GQA	58
11.6.1 推論時的記憶體瓶頸	58
11.6.2 三種配置	58
11.6.3 GQA 的實作	59
11.7 複雜度、記憶體牆與 FlashAttention	59
11.7.1 複雜度分析	59
11.7.2 記憶體才是真正的牆	60
11.7.3 FlashAttention 的核心想法	61
11.7.4 注意力的其他變體：一張地圖	61
程式實作	63
程式 11A 四個版本的注意力	63
程式 11B 繁體中文注意力熱圖	64
程式 11C 複雜度實測	66
系統案例：可互動的注意力教學元件	67
功能規格	67
交付要求	67
章末評量	69
一、概念題	69
二、計算題	69
三、除錯題	69
四、系統設計題	70
五、臺灣情境與倫理題	70

評量提示（給自學者）	70
研究延伸與版本注意事項	70
常見問題	71
教師使用建議	71
本章小結	72
成果檢核	73
Transformer Block 與現代 GPT 元件	74
學習目標	74
12.1 正規化：LayerNorm 與 RMSNorm	74
12.1.1 為什麼需要正規化	74
12.1.2 LayerNorm	75
12.1.3 RMSNorm：拿掉一半的計算	75
12.2 Pre-LN 與 Post-LN：一個順序決定成敗	77
12.2.1 兩種放法	77
12.2.2 為什麼 Pre-LN 更穩定	77
12.2.3 Pre-LN 的代價：殘差流會膨脹	78
12.3 前饋網路：GELU 與 SwiGLU	79
12.3.1 前饋網路在做什麼	79
12.3.2 激勵函數的演進	79
12.3.3 SwiGLU 的參數量對齊	79
12.4 組裝：Transformer Block	81
12.4.1 完整的資料流	81
12.4.2 實作	81
12.4.3 六個常見的組裝錯誤	82
12.5 參數量與 FLOPs 的精確計算	83
12.5.1 逐項拆解	84
12.5.2 三種尺寸的實算	84
12.5.3 FLOPs 計算	85
12.5.4 從參數量到記憶體	86
12.6 從 GPT-2 到 Llama-style：元件替換的動機	86
12.6.1 六項替換	86
12.6.2 什麼時候該用哪一種	87
12.6.3 本書的設定	87
12.7 初始化與正則化的實務取捨	88
12.7.1 初始化清單	89
12.7.2 dropout 與 weight decay	90
12.7.3 把 block 疊起來會遇到的三個問題	90
12.7.4 一份組裝前的檢查清單	91
程式實作	93

程式 12A 可設定的 Transformer Block.....	93
程式 12B 參數量與 FLOPs 計算器	94
程式 12C Pre-LN 與 Post-LN 的消融實驗	95
系統案例：MiniFormosaGPT 三種尺寸的規劃.....	96
三種尺寸的設計	96
必須交付的五項	97
評分重點	97
章末評量	98
一、概念題	98
二、計算題	98
三、除錯題	98
四、系統設計題	99
五、臺灣情境與倫理題	99
評量提示（給自學者）	99
研究延伸與版本注意事項	100
常見問題	100
教師使用建議	100
本章小結	101
成果檢核	102
從零建構 MiniFormosaGPT 與文字生成器	103
學習目標	103
13.1 組裝完整模型	103
13.1.1 六個元件，一條資料流.....	103
13.1.2 完整實作	104
13.1.3 端到端測試	106
13.2 損失計算：三個容易錯的地方.....	107
13.2.1 位移對齊	107
13.2.2 三種錯誤與它們的症狀.....	108
13.2.3 第一道健全性檢查：過擬合單一批次	108
13.3 資料管線：從語料到批次	109
13.3.1 把語料變成一長串詞元.....	109
13.3.2 滑動視窗取樣	110
13.3.3 DataLoader 設定.....	111
13.4 最小可行訓練迴圈.....	111
13.4.1 核心結構.....	111
13.4.2 checkpoint 的內容	114
13.5 生成：從模型到文字	115
13.5.1 自回歸生成的迴圈	115
13.5.2 為什麼 greedy 生成會重複.....	116

13.5.3 訓練里程碑：從亂碼到成句.....	116
13.6 八種常見的訓練失敗.....	117
13.6.1 診斷的正確順序.....	118
13.6.2 一個真實的除錯案例.....	118
13.7 在筆電上跑完一次完整訓練.....	119
13.7.1 三種尺寸的訓練預算.....	119
13.7.2 A 級路線的完整步驟.....	120
13.7.3 該期待什麼樣的結果.....	120
程式實作.....	121
程式 13A MiniFormosaGPT 完整實作與端到端訓練.....	121
程式 13B 生成器 CLI.....	121
程式 13C 訓練健檢腳本.....	122
系統案例：從亂碼到成句的訓練縮時紀錄.....	123
記錄什麼.....	123
固定的探針提示.....	124
交付要求.....	124
評分重點.....	125
章末評量.....	126
一、概念題.....	126
二、計算題.....	126
三、除錯題.....	126
四、系統設計題.....	127
五、臺灣情境與倫理題.....	127
評量提示（給自學者）.....	127
研究延伸與版本注意事項.....	127
常見問題.....	128
教師使用建議.....	128
本章小結.....	129
成果檢核.....	130
索引.....	131
英文詞條.....	131
中文詞條.....	131

全書架構總覽

全書分為十篇四十章，另有十四組附錄。本冊為第 3 篇，是全書的實作核心，也是第一學期後半的主線。

篇次	章次	主題	核心產出
第 1 篇	1–4	大模型視野、數學與工程基礎	需求規格書、技術演進圖、數學速查表、可重現環境
第 2 篇	5–9	語言建模、臺灣語料與 Token 工程	基準評量規格、文本工程規格、自訓 tokenizer、語料清冊
第 3 篇 (本冊)	10–13	Transformer 從零實作	MiniFormosaGPT 與可生成繁體中文的模型
第 4 篇	14–17	預訓練、解碼與實驗科學	穩定訓練迴圈、尺度化推估與算力預算
第 5 篇	18–21	算力、系統與現代架構	效能剖析、分散式決策、臺灣算力盤點
第 6 篇	22–26	本土化後訓練與對齊	繁中適配模型、LoRA、偏好對齊、邊緣小模型
第 7 篇	27–30	推理、知識更新與提示工程	推理實驗、知識更新流程、RAG 原型
第 8 篇	31–34	應用系統：進階 RAG、Agent 與產品開發	知識系統、受限 Agent、可上線的應用
第 9 篇	35–37	部署、評測、安全與治理	量化部署、臺灣題庫、紅隊與影響評估
第 10 篇	38–40	多模態、具身智慧與總整專題	VLM 應用、臺語語音、VLA、總整專題

本冊四章的實作遞進

章	你會寫出什麼	它解決什麼問題	被誰使用
第 10 章	嵌入層、RoPE、輸出投影	把詞元變成向量並注入位置	第 11–13、20、22 章

章	你會寫出什麼	它解決什麼問題	被誰使用
第 11 章	多頭因果注意力	讓位置之間直接交換資訊	第 12-14、18、20 章
第 12 章	RMSNorm、SwiGLU、Block	讓模型能疊四十層而不崩潰	第 13、22、26 章
第 13 章	MiniFormosaGPT 與生成器	把零件接成能訓練的完整模型	第 14 章之後全部

第 3 篇

Transformer 從零實作

Transformers from Scratch

第 10–13 章

本篇承諾：不依賴高階函式庫，親手寫出 embedding、attention、Transformer block 與可生成文字的 MiniFormosaGPT。

第 10 章

詞嵌入、語意空間與位置表示

Embeddings, Semantic Spaces, and Positional Representations

難度：核心 | 建議授課：第 10 週 | 硬體需求：A 級（一般筆電即可完成全部實作，不需 GPU）

叫獸開講

把「臺南」減掉「臺灣」再加上「日本」，向量會落在「大阪」附近。語意空間不是比喻，它是一個你可以拿尺去量的地方。這一章要做的，是把第二篇處理好的那些詞元，變成模型真正看得懂的東西——一串可以做加減乘除的實數。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明 one-hot、查表與嵌入矩陣的關係，並正確估算嵌入層的參數量與記憶體。
2. 解釋分布式假說如何從 Word2Vec 演進到情境化表示，以及大模型的嵌入與前者的差異。
3. 用餘弦相似度建立語意檢索原型，並辨識各向異性對相似度判讀的影響。
4. 比較絕對位置編碼、正弦位置編碼與可學習位置嵌入的性質與限制。
5. 推導 RoPE 的旋轉形式，並用程式驗證它的內積只取決於相對位置。
6. 說明 ALiBi 的線性偏置機制，並比較它與 RoPE 在長度外推上的差異。
7. 判斷是否要綁定輸入嵌入與輸出投影，並說明綁定的利弊。
8. 實作詞彙擴充後的嵌入初始化，並量測不同初始化策略對收斂的影響。
9. 用 PCA、t-SNE 與 UMAP 視覺化嵌入，並辨識三者常見的誤讀。

本章在全書中的位置

第 3 篇的四章是一條直線：第 10 章把詞元變成向量並注入位置，第 11 章讓這些向量互相觀看（注意力），第 12 章把它們組裝成可堆疊的積木，第 13 章疊起來訓練出 MiniFormosaGPT。這是全書唯一一段「每一章的產出都是下一章的輸入」的連續實作，中間跳過任何一章，後面都會卡住。

10.1 從詞元到向量：嵌入層

10.1.1 one-hot 與查表是同一件事

模型的輸入是一串整數索引（第 7 章的 tokenizer 產出）。要把整數變成向量，最直觀的想法是 one-hot：把索引 5 變成一個長度 V 的向量，第 5 個位置是 1，其餘為 0。然後乘上一個矩陣 E 。

$$e = \text{onehot}(i)^T \cdot E \quad \cdot \text{其中 } E \text{ 的形狀為 } (V, d)$$

但這個矩陣乘法有 99.99% 的計算是在乘零。因為 one-hot 只有一個位置非零，這次乘法的結果其實就是「取出 E 的第 i 列」。

$$e = E[i :]$$

所以實作上不會真的做矩陣乘法，而是直接查表。PyTorch 的 `nn.Embedding` 就是一張查找表，它的反向傳播只更新被用到的那幾列。這是一個很好的例子：數學上的定義與工程上的實作可以不同，只要結果等價。

觀點	運算	成本	什麼時候該用這個觀點
one-hot 乘法	$(B, T, V) @ (V, d)$	極高，且浪費	推導公式、理解輸出層
查表	<code>E[ids]</code>	極低，只有記憶體讀取	所有實作

10.1.2 參數量與記憶體

嵌入矩陣的參數量是 V 乘以 d 。這個數字比多數人預期的的大。

設定	$V \times d$	參數量	備註
小型模型	$32,000 \times 512$	約 1,640 萬	若輸出層不共享，需乘二
中型模型	$32,000 \times 768$	約 2,460 萬	同上
大型多語模型	$128,000 \times 4,096$	約 5.24 億	光嵌入層就超過許多小模型的總參數

最後一列值得記住：一個 128K 詞彙表配 4096 維的嵌入層，本身就有 5.24 億個參數。如果輸入與輸出不共享權重，兩層加起來超過 10 億——這在小模型上會占據總參數的極高比例，也是第 7 章要你謹慎選擇詞彙表大小的具體理由。

嵌入層的最小實作與參數統計

```
import torch
import torch.nn as nn

class TokenEmbedding(nn.Module):
    def __init__(self, vocab_size: int, d_model: int, pad_id: int | None = None):
```

```

super().__init__()
self.emb = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
self.d_model = d_model
# GPT 式初始化：標準差 0.02（見第 3.5.7 節）
nn.init.normal_(self.emb.weight, mean=0.0, std=0.02)
if pad_id is not None:
    with torch.no_grad():
        self.emb.weight[pad_id].zero_()

def forward(self, ids: torch.Tensor) -> torch.Tensor:
    # ids: (B, T) -> (B, T, d)
    assert ids.dtype in (torch.long, torch.int), ids.dtype
    return self.emb(ids)

def embedding_cost(V: int, d: int, tied: bool, dtype_bytes: int = 2):
    n = V * d * (1 if tied else 2)
    return {"params": n,
            "weights_MB": round(n * dtype_bytes / 1e6, 1),
            "adamw_states_MB": round(n * 8 / 1e6, 1)} # m + v 各 FP32

```

注意 `padding_idx` 這個參數：它讓 PAD 詞元的嵌入永遠保持為零，且不接收梯度。這與第 5.2 節的 `loss mask` 是配套的——PAD 既不該影響輸出，也不該被學習。

10.1.3 為什麼要乘以維度的平方根

許多 Transformer 實作會在嵌入之後乘上一個係數：

$$x = E[ids] \cdot \sqrt{d}$$

這個做法源自原始論文，理由與初始化尺度有關。嵌入初始化的標準差通常很小（例如 0.02），而位置編碼的數值範圍是正負一之間。如果直接相加，位置訊號會蓋過詞元訊號。乘上 $\sqrt{d}$ 可以把嵌入的量級拉到與位置編碼相當。

但這不是普世做法。現代的 Llama-style 模型多半不做這個縮放，因為它們用 RoPE（位置資訊不是加在嵌入上，而是作用在注意力的 Q 與 K 上），沒有量級競爭的問題。本書第 13 章的 MiniFormosaGPT 採用 RoPE，因此不做縮放——但你在讀別人的程式碼時要能認出這一行，並知道它為什麼在那裡。

10.2 語意空間：從 Word2Vec 到情境化表示

10.2.1 分布式假說與它的兩種實現

第 2.3.2 節已經介紹過分布式假說：一個詞的意義由它周圍常出現的詞決定。這個假說有兩種實現路徑，差別在於「向量是靜態的還是隨語境改變」。

類型	代表	性質	限制
靜態詞向量	Word2Vec、GloVe	一個詞一個固定向量	無法處理一詞多義
情境化表示	ELMo、BERT、GPT	同一個詞在不同句子有不同向量	需要跑完整個模型才能得到
大模型的嵌入層	GPT 系列的 E	仍是靜態的查找表	情境化發生在後續的 Transformer 層

第三列是本節最重要的澄清，也是學生最常誤解的地方。大模型的嵌入層本身是靜態的——「行」這個字進入模型時，拿到的是同一個向量，不管它是「銀行」還是「行人」。情境化是在後續的注意力層中發生的：第 11 章你會看到，注意力讓每個位置從其他位置蒐集資訊，於是同一個詞在不同語境中的表示逐層分化。

這個分工有一個實務意涵：如果你想要好的句子嵌入（例如第 30 章的檢索），不能只用嵌入層的輸出，必須用模型某一層（通常是中後層）的隱藏狀態。第 30 章會回到這個問題。

10.2.2 餘弦相似度與語意檢索

有了向量，「這兩個詞有多像」就變成了一個幾何問題。標準工具是第 3.2.1 節介紹過的餘弦相似度：

$$\text{cos_sim}(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|)$$

它衡量的是方向而非長度。兩個指向完全相同方向的向量，即使長度差十倍，餘弦相似度仍是 1。這通常是我們要的——詞向量的長度往往與詞頻相關，而我們關心的是語意。

餘弦相似度與最近鄰檢索

```
import torch
import torch.nn.functional as F

def cosine_topk(query_vec, matrix, k=10, exclude=None):
    """在 matrix (N, d) 中找出與 query_vec 最相近的 k 個
    先正規化再做矩陣乘法，等價於逐一算餘弦但快得多"""
    q = F.normalize(query_vec, dim=-1)          # (d,)
    M = F.normalize(matrix, dim=-1)             # (N, d)
    sims = M @ q                                # (N,)
    if exclude is not None:
        sims[exclude] = -float("inf")
    vals, idx = sims.topk(k)
    return idx.tolist(), vals.tolist()

def analogy(emb, tok, a: str, b: str, c: str, k=5):
```

```

"""a 之於 b，如同 c 之於 ?    例：臺南 - 臺灣 + 日本"""
ids = [tok.encode(w)[0] for w in (a, b, c)]
va, vb, vc = (emb.weight[i] for i in ids)
target = va - vb + vc
idx, vals = cosine_topk(target, emb.weight, k + 3, exclude=ids)
return [(tok.decode([i]), round(v, 4)) for i, v in zip(idx, vals)][:k]

```

把正規化與矩陣乘法分開寫是有理由的：正規化只需做一次，之後的檢索就是單純的矩陣乘法，可以在 GPU 上高效執行。這正是第 30 章向量資料庫的基本原理——所謂的向量檢索，核心就是這幾行。

10.2.3 各向異性：一個會誤導你的現象

這是一個實務上很常踩、教科書卻很少提的坑。理論上，隨機的高維向量彼此之間應該接近正交，平均餘弦相似度趨近於零。但真實模型的嵌入不是這樣——它們往往集中在一個狹窄的錐狀區域中，導致任意兩個向量的相似度都偏高。

用一個數值實驗說明。取 500 個隨機的 64 維單位向量，兩兩之間的平均餘弦相似度約為 0.000；但如果讓它們都偏向同一個共同方向（模擬各向異性），平均相似度會飆到約 0.996——幾乎所有東西看起來都很像。

情況	平均餘弦相似度	判讀	處置
理想的等向分布	約 0.00	相似度有鑑別力	直接使用
輕度各向異性	0.2 至 0.5	需以相對排序判讀	看排名而非絕對值
嚴重各向異性	大於 0.9	絕對值毫無意義	需做中心化或白化

實務上的三個對策：第一，永遠看相對排序而不是絕對數值——「A 比 B 更接近 Q」是可靠的，「A 與 Q 的相似度是 0.87」不是。第二，做中心化（減去所有向量的平均）通常能顯著改善。第三，若要正式使用嵌入做檢索，應在你自己的資料上量測相似度分布，而不是套用他人的閾值。

一個給臺灣讀者的補充

各向異性在多語模型上有一個特別的表現：不同語言的向量常常會各自聚成一團，導致「任何兩個中文詞的相似度」都高於「一個中文詞與一個英文詞的相似度」，即使後者語意相同。做跨語言檢索時若不處理這件事，結果會完全失真。第 30 章會給出具體的量測方法。

10.2.4 嵌入層在訓練中的特殊性

嵌入層與模型的其他部分有一個關鍵差異：它的梯度是稀疏的。一個批次中只有出現過的詞元會被更

新，其餘幾萬列完全不動。這造成三個實務後果。

現象	成因	後果	處置
低頻詞元學不好	整個訓練只被更新幾次	罕見詞的表示品質差	第 7 章的詞彙表大小取捨
詞頻與向量長度相關	高頻詞被更新多次	長度差異可能干擾內積	用餘弦相似度而非內積
weight decay 的爭議	對未出現的詞元也施加衰減	低頻詞元被持續拉向零	嵌入層常排除 weight decay
最佳化器狀態龐大	$V \times d$ 的 m 與 v	記憶體壓力	第 19 章的分片策略

第三列值得特別說明。標準的 AdamW 會對所有參數施加權重衰減，包括那些在這個批次中根本沒出現的詞元嵌入。長期下來，低頻詞元的向量會被不斷拉向原點，因為它們「被衰減的次數」遠多於「被更新的次數」。

因此許多實作會把嵌入層（以及 LayerNorm 的參數與 bias）排除在 weight decay 之外。這是一個典型的「預設值不一定對」的例子——照抄別人的最佳化器設定而不看參數分組，可能會讓你的罕見詞元表示悄悄退化。

參數分組：哪些該做 weight decay

```
def build_param_groups(model, weight_decay=0.1):
    decay, no_decay = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue
        # 一維參數 (LayerNorm 權重、bias) 與嵌入層不做衰減
        if p.ndim < 2 or "emb" in name or "norm" in name:
            no_decay.append(p)
        else:
            decay.append(p)
    print(f"weight decay: {sum(p.numel() for p in decay):,} 個參數")
    print(f"no decay: {sum(p.numel() for p in no_decay):,} 個參數")
    return [
        {"params": decay, "weight_decay": weight_decay},
        {"params": no_decay, "weight_decay": 0.0},
    ]
```

這個函式會在第 13 與 15 章的訓練腳本中被直接使用。現在先建立它，並記住那條判準：二維以上的權重矩陣做衰減，一維的縮放與偏置不做，嵌入層視情況（本書選擇不做）。

10.3 位置編碼：為沒有順序概念的注意力補上順序

10.3.1 為什麼需要位置資訊

這是第 2.6.1 節提過、但值得在這裡講清楚的一件事：注意力機制對輸入的順序完全無感。

證明很簡單。注意力的計算是「每個位置與每個位置算分數、softmax、加權求和」。如果你把輸入序列的順序打亂，每一對位置之間的分數不變，只是排列順序改變——輸出也會以同樣的方式被排列。這個性質稱為排列等變（permutation equivariant）。

對語言而言這是災難：「狗咬人」與「人咬狗」在純注意力眼中是同一件事。所以必須額外注入位置資訊。

10.3.2 四種主要做法

方法	注入位置的方式	外推能力	代表模型
可學習位置嵌入	為每個位置學一個向量，加在詞嵌入上	差（超過訓練長度即失效）	BERT、GPT-2
正弦位置編碼	用固定的三角函數產生向量，加在詞嵌入上	理論上可外推，實測有限	原始 Transformer
RoPE	對 Q 與 K 做旋轉，作用在注意力內部	較好，可搭配縮放技術延伸	Llama 系列
ALiBi	在注意力分數上加線性距離懲罰	好，設計上就針對外推	部分長上下文模型

第一與第二種是「加法式」的：位置向量與詞嵌入相加，成為輸入的一部分。第三與第四種是「作用式」的：位置資訊不進入嵌入，而是在注意力計算時才發生作用。這個區別很重要，它決定了位置資訊如何隨層數傳遞。

10.3.3 可學習位置嵌入

最簡單的做法

```
class LearnedPositionEmbedding(nn.Module):
    def __init__(self, max_len: int, d_model: int):
        super().__init__()
        self.pos = nn.Embedding(max_len, d_model)
        nn.init.normal_(self.pos.weight, std=0.02)
        self.max_len = max_len

    def forward(self, x):
```

```
# x: (B, T, d)
B, T, d = x.shape
assert T <= self.max_len, \
    f"序列長度 {T} 超過訓練上限 {self.max_len}"
pos = torch.arange(T, device=x.device)
return x + self.pos(pos)          # 廣播：(T,d) 加到 (B,T,d)
```

那行 `assert` 是這個方法的致命傷的具體體現：訓練時看過的最長長度是多少，推論時就不能超過。因為第 1025 個位置的嵌入從未被訓練過（如果 `max_len` 是 1024），它就是一個隨機向量。

這對實務的影響很直接：一個用可學習位置嵌入訓練、上下文長度 2048 的模型，你無法讓它處理 4096 的輸入——不是效果變差，是根本沒有那些位置的參數。這是為什麼近年的模型幾乎都改用 RoPE。

10.3.4 正弦位置編碼

原始 Transformer 用的是固定的三角函數，不需要學習：

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d)}) \quad ; \quad PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d)})$$

設計的巧思在於不同維度使用不同的頻率：低維度變化快（週期短），高維度變化慢（週期長）。這讓模型可以同時感知細微的相鄰關係與粗略的遠距關係，類似時鐘的秒針與時針。

正弦位置編碼的實作與性質驗證

```
import math

def sinusoidal_pe(max_len: int, d_model: int) -> torch.Tensor:
    pe = torch.zeros(max_len, d_model)
    pos = torch.arange(max_len, dtype=torch.float32).unsqueeze(1)  # (T,1)
    div = torch.exp(torch.arange(0, d_model, 2).float()
                    * (-math.log(10000.0) / d_model))                # (d/2,)
    pe[:, 0::2] = torch.sin(pos * div)
    pe[:, 1::2] = torch.cos(pos * div)
    return pe                                                         # (T, d)

# 性質一：每個位置向量的長度相同
pe = sinusoidal_pe(8, 64)
print(pe.norm(dim=1))        # 全部等於 sqrt(d/2) = 5.6569

# 性質二：內積隨距離遞減（但不是單調的）
print([round(float(pe[0] @ pe[k]), 2) for k in range(8)])
# [32.0, 30.92, 28.3, 25.59, 23.93, 23.5, 23.56, 23.26]
```

第二個性質值得注意：內積確實隨距離下降，但下降到某個程度後就趨於平緩，甚至會小幅回升。這意味著正弦編碼提供的「距離感」在遠距離時會變得模糊。這是它外推能力有限的原因之一。

10.4 RoPE：現代模型的標準選擇

10.4.1 核心想法：用旋轉表示位置

RoPE (Rotary Position Embedding，旋轉位置編碼) 的想法很優雅：不把位置加到向量上，而是把向量旋轉一個與位置成正比的角度。

把 d 維向量看成 $d/2$ 個二維平面上的點。對於位置 m ，把第 j 個平面上的點旋轉 $m \cdot \theta_j$ 弧度，其中 θ_j 是一組預先決定的頻率（與正弦編碼類似，低維度轉得快、高維度轉得慢）。

$$\theta_j = \text{base}^{(-2j/d)} \quad \cdot \text{base 通常取 } 10000$$

10.4.2 為什麼旋轉有效：相對位置性質

這是 RoPE 最漂亮的性質，也是它勝過加法式編碼的根本理由。

旋轉有一個基本的幾何事實：兩個向量各自旋轉之後，它們的內積只取決於旋轉角度的差。用在注意力上，Query 在位置 m 、Key 在位置 n ，經過 RoPE 之後的內積：

$$\langle \text{RoPE}(q, m), \text{RoPE}(k, n) \rangle = f(q, k, m - n)$$

也就是說，注意力分數自動只依賴相對位置 $m - n$ ，而不是絕對位置。這正是語言真正需要的——「前一個詞」的關係在句首與句尾應該是一樣的。

這個性質可以直接用程式驗證，而且結果精確到浮點誤差以內：

m	n	m - n	內積	觀察
5	3	2	-10.551758	同樣的相對距離
10	8	2	-10.551758	得到完全相同的內積
100	98	2	-10.551758	即使絕對位置差很多
7	2	5	-9.005375	不同的相對距離
50	45	5	-9.005375	仍然完全一致

（上表由程式 10A 實測得出，使用 64 維隨機向量。）

10.4.3 實作

formosa/model/rope.py

```
import torch
```



```

def build_rope_cache(seq_len: int, head_dim: int, base: float = 10000.0,
                    device=None, dtype=torch.float32):
    """預先算好 cos 與 sin，訓練與推論時重複使用"""
    assert head_dim % 2 == 0, "head_dim 必須是偶數"
    half = head_dim // 2
    inv_freq = base ** (-torch.arange(0, half, device=device).float() / half)
    t = torch.arange(seq_len, device=device).float()
    freqs = torch.outer(t, inv_freq) # (T, half)
    return freqs.cos().to(dtype), freqs.sin().to(dtype)

def apply_rope(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor):
    """x: (B, H, T, d_h) cos/sin: (T, d_h/2)"""
    B, H, T, dh = x.shape
    assert cos.shape[0] >= T, f"RoPE cache 長度 {cos.shape[0]} 小於序列 {T}"
    half = dh // 2
    x1, x2 = x[..., :half], x[..., half:]
    c = cos[:T].view(1, 1, T, half)
    s = sin[:T].view(1, 1, T, half)
    # 二維旋轉: (x1, x2) -> (x1*cos - x2*sin, x1*sin + x2*cos)
    out = torch.cat([x1 * c - x2 * s, x1 * s + x2 * c], dim=-1)
    assert out.shape == x.shape
    return out

def verify_relative(head_dim=64, base=10000.0):
    """驗證內積只依賴相對位置"""
    g = torch.Generator().manual_seed(0)
    q = torch.randn(head_dim, generator=g)
    k = torch.randn(head_dim, generator=g)
    cos, sin = build_rope_cache(256, head_dim, base)

    def rot(v, pos):
        return apply_rope(v.view(1, 1, 1, -1),
                          cos[pos:pos + 1], sin[pos:pos + 1]).flatten()

    for m, n in [(5, 3), (10, 8), (100, 98)]:
        print(m, n, m - n, float(rot(q, m) @ rot(k, n)))

```

注意 `build_rope_cache` 是預先算好並快取的。`cos` 與 `sin` 只跟位置與維度有關，與輸入內容無關，所以整個訓練過程只需要算一次。這在推論時尤其重要——第 14 章的 KV cache 會反覆使用它。

10.4.4 RoPE 的長度外推

RoPE 比可學習位置嵌入好的地方在於：它沒有「未訓練的位置參數」的問題，任何位置都可以算出對應的旋轉角度。但這不代表它可以無限外推。

問題在於：訓練時模型只見過某個範圍內的相對距離。超出那個範圍，高頻的維度已經轉了很多圈，模型從未見過那樣的組合，注意力分布會失控。

方法	做法	效果	代價
直接外推	什麼都不做	通常在 1.2 倍以上就明顯劣化	無
位置內插	把位置索引壓縮到訓練範圍內	簡單有效，需少量微調	短距離的解析度下降
調高 base	把 base 從 10000 加大	降低旋轉頻率，等效延長	需重新訓練或微調
NTK 感知縮放	對不同頻率採用不同縮放	兼顧長短距離	實作較複雜
YaRN	結合縮放與注意力溫度調整	目前常用的組合方案	需要微調

本章只需要你理解「為什麼會有外推問題」與「大方向的解法是什麼」。第 20 章會完整處理長上下文技術，並讓你實測不同方法在繁中長文件上的表現。

10.4.5 ALiBi：另一條路線

ALiBi 的想法更直接：不動 Q 與 K，而是在注意力分數上加一個與距離成正比的負值。

$$score(i, j) = q_i \cdot k_j + m \cdot (j - i) \quad , \text{其中 } m \text{ 為該注意力頭的斜率 (負值)}$$

距離越遠，懲罰越大。不同的注意力頭使用不同的斜率，形成一個幾何級數。以 8 個頭為例，斜率為 0.5、0.25、0.125、0.0625、0.03125、0.015625、0.0078125、0.00390625——最陡的頭專注於鄰近位置，最緩的頭幾乎不受距離影響，可以看得很遠。

面向	RoPE	ALiBi	備註
作用位置	Q 與 K 的旋轉	注意力分數的偏置	—
外推能力	需搭配縮放技術	設計上即支援	ALiBi 在此較單純
短距離表現	較佳	略遜	RoPE 保留完整的方向資訊
實作複雜度	中	低	ALiBi 只是加一個矩陣
生態採用	主流	較少	本書第 13 章採用 RoPE

本書選擇 RoPE 是因為生態成熟——你在第 22 章要做持續預訓練時，絕大多數可用的開放權重底模都使用 RoPE，理解它是實務上的必要條件。但 ALiBi 值得知道，它是一個「用最簡單的手段解決問題」的好例子。

10.4.6 三種位置編碼的實測對照

把本節與前一節的內容整合成一張可以直接查的表。程式 10A 會要求你在自己的環境中重現這些性質。

性質	可學習嵌入	正弦編碼	RoPE	ALiBi
參數量	$\text{max_len} \times d$	0 (固定)	0 (固定)	0 (固定)
超出訓練長度	無參數可用	可算但效果差	可算，需縮放	設計上支援
絕對或相對	絕對	絕對 (隱含相對)	相對	相對
作用位置	輸入端一次	輸入端一次	每層的 Q 與 K	每層的分數
與 KV cache 相容	相容	相容	相容 (需記位置)	相容
實作行數	約 8 行	約 10 行	約 20 行	約 12 行

倒數第二列在第 14 章會變得重要：使用 RoPE 時，KV cache 中存的是「已經旋轉過的」K，所以每個位置的絕對索引必須被正確追蹤。如果你在續接生成時把位置算錯，模型的行為會突然變得很奇怪——這是實作 KV cache 時最常見的 bug。

10.4.7 一個容易被忽略的細節：RoPE 的維度配對

本章的實作把 `head_dim` 切成前半與後半兩塊做旋轉，也就是把第 j 維與第 $j + \text{half}$ 維配成一對。但有些實作是把相鄰的兩維配對（第 0 與第 1、第 2 與第 3）。

兩種做法在數學上等價——它們都是把向量看成若干個二維平面上的點，只是分組方式不同。但它們產生的數值不同，因此權重不可互換。

配對方式	第 j 個平面是	常見於	注意
半分 (half split)	$(x[j], x[j+\text{half}])$	多數現代實作	本書採用
交錯 (interleaved)	$(x[2j], x[2j+1])$	部分早期實作	與上者不相容

如果你要載入別人的模型權重（第 22 章會做），必須確認雙方的配對方式一致，否則模型的行為會完全錯亂——而且它不會報錯，只會生成沒有意義的文字。這是一個「靜默失敗」的典型示例，值得

在載入權重時寫一個斷言檢查。

10.5 輸出投影與權重綁定

10.5.1 嵌入層的鏡像

模型的最後一步是把隱藏狀態映射回詞彙表，產生 logits。這一層的形狀是 (d, V) ——正好是嵌入矩陣 (V, d) 的轉置。

$$\text{logits} = h \cdot W^{\text{out}} \quad \cdot \text{形狀 } (B, T, d) @ (d, V) \rightarrow (B, T, V)$$

既然形狀對稱，一個自然的想法是：讓它們共用同一組參數。這稱為權重綁定 (weight tying)。

$$W^{\text{out}} = E^T$$

10.5.2 綁定的利弊

面向	綁定	不綁定	判斷
參數量	省下 $V \times d$	兩份獨立參數	小模型受益明顯
正則效果	輸入輸出共享語意空間，有正則作用	較自由	資料少時綁定較佳
表達能力	受限（輸入與輸出的最佳表示未必相同）	較強	大模型不綁定較常見
詞彙擴充	只需改一處	兩處都要改	綁定較單純
初始化	共用一組	需分別處理	綁定較單純

一個直觀的理由說明為什麼綁定有道理：如果模型認為「臺北」與「高雄」在輸入端是相似的（都是城市名），那麼在輸出端它們也應該有相似的機率行為。共用參數強制了這種一致性。

但也有反對的理由：輸入嵌入回答的是「這個詞攜帶什麼資訊」，輸出投影回答的是「什麼情況下該產生這個詞」，兩者未必要用同一組向量表示。實務上，小模型與資料量有限時綁定的收益較大，大模型則常常不綁定。

本書第 13 章的 MiniFormosaGPT 預設綁定，理由是模型小、語料有限，而且它讓詞彙擴充（第 22 章）的實作單純很多。

權重綁定的實作與驗證

```

class LMHead(nn.Module):
    def __init__(self, d_model: int, vocab_size: int,
                  tied_embedding: nn.Embedding | None = None):
        super().__init__()
        self.proj = nn.Linear(d_model, vocab_size, bias=False)
        if tied_embedding is not None:
            # 直接共用同一個 Parameter 物件，不是複製
            self.proj.weight = tied_embedding.weight
        else:
            nn.init.normal_(self.proj.weight, std=0.02)

    def forward(self, h):
        return self.proj(h)          # (B,T,d) -> (B,T,V)

def assert_tied(model):
    """驗證真的綁定了：必須是同一個物件，而非數值相同"""
    a = model.tok_emb.emb.weight
    b = model.lm_head.proj.weight
    assert a is b, "權重未綁定（是兩個獨立的 Parameter）"
    n_shared = a.numel()
    print(f"已綁定，省下 {n_shared:,} 個參數")

```

`assert_tied` 中用的是 `'is'` 而不是 `'torch.equal'`。這個區別很重要：如果只是把數值複製過去，兩者一開始相同，但訓練幾步之後就會分道揚鑣——那不是綁定，是巧合。真正的綁定必須是同一個 `Parameter` 物件。這是一個很容易寫錯、卻不會報錯的地方。

10.5.3 softmax 之前的最後一步

有些模型會在輸出投影之前做最後一次正規化（`final norm`），有些會加上 `logit` 縮放。這些細節在第 12 章會完整處理，本章只需要記住整條路徑：

步驟	運算	形狀	本書章節
1	詞元索引	(B, T)	第 7 章
2	嵌入查表	(B, T, d)	本章 10.1
3	N 層 Transformer	(B, T, d)	第 11、12 章
4	最終正規化	(B, T, d)	第 12 章
5	輸出投影	(B, T, V)	本章 10.5
6	交叉熵或取樣	純量或 (B, T)	第 5、14 章

這張表就是 MiniFormosaGPT 的完整資料流。第 13 章你要寫的程式，本質上就是把這六步接起來。

10.6 詞彙擴充後的嵌入初始化

10.6.1 問題與四種策略

第 7.6 節已經介紹過詞彙擴充的動機。這裡處理它的技術核心：新增的詞元需要新的嵌入向量，而初始化的方式會顯著影響收斂速度。

策略	做法	收斂速度	備註
隨機初始化	從常態分布取樣	最慢	新詞元與既有空間毫無關係
平均初始化	取所有既有嵌入的平均	中	至少落在正確的區域
子詞平均	用該詞在舊 tokenizer 下的切法取平均	最快	本書建議的預設
多語對應	用語意對應的既有詞元嵌入	快（若對照表可靠）	需額外的對照資料

子詞平均之所以有效，理由很直觀：新詞元「智慧製造」在舊 tokenizer 下會被切成「智」「慧」「製」「造」四個詞元。這四個詞元的嵌入平均，已經是一個相當合理的起點——它落在正確的語意鄰域中，模型只需要微調而不是從零學起。

10.6.2 完整實作

formosa/model/expand.py

```
import torch

@torch.no_grad()
def expand_embeddings(model, old_tok, new_tokens: list[str],
                     strategy: str = "subword_mean", noise_std: float = 0.0):
    """在既有模型上加入新詞元並初始化嵌入"""
    emb = model.get_input_embeddings()
    old_size, d = emb.weight.shape
    tied = model.lm_head.proj.weight is emb.weight

    model.resize_token_embeddings(old_size + len(new_tokens))
    emb = model.get_input_embeddings()
    base_mean = emb.weight[:old_size].mean(dim=0)

    stats = {"subword": 0, "fallback": 0}
    for i, tok_str in enumerate(new_tokens):
        row = old_size + i
        if strategy == "random":
            emb.weight[row].normal_(0.0, 0.02)
            continue
        sub_ids = [s for s in old_tok.encode(tok_str) if s < old_size]
        if strategy == "subword_mean" and sub_ids:
```

```

        emb.weight[row] = emb.weight[sub_ids].mean(dim=0)
        stats["subword"] += 1
    else:
        emb.weight[row] = base_mean
        stats["fallback"] += 1
    if noise_std > 0:
        # 加一點雜訊避免多個新詞元初始值完全相同
        emb.weight[row] += torch.randn(d) * noise_std

    if not tied:
        head = model.lm_head.proj.weight # 輸出層需同樣處理
        for i, tok_str in enumerate(new_tokens):
            sub_ids = [s for s in old_tok.encode(tok_str) if s < old_size]
            head[old_size + i] = (head[sub_ids].mean(dim=0) if sub_ids
                                else head[:old_size].mean(dim=0))
        print(f"擴充 {len(new_tokens)} 個詞元：")
        print(f"子詞平均 {stats['subword']}，退回全域平均 {stats['fallback']}")
    return model

```

兩個實作細節值得說明。第一，`noise_std` 的存在是為了避免多個新詞元拿到完全相同的初始值——如果兩個詞元的初始嵌入一模一樣，它們的梯度也會一樣，永遠無法分化。第二，如果沒有綁定權重，輸出層必須同樣處理，否則模型可以「理解」新詞元卻永遠不會「產生」它。

10.6.3 擴充之後必須做的事

1. 驗證形狀：嵌入層與輸出層的大小都要正確更新，且 `tokenizer` 的詞彙表大小要一致。
2. 驗證綁定狀態：如果原本綁定，擴充後必須仍然綁定（有些框架的 `resize` 會意外解開）。
3. 檢查既有能力：擴充本身不該改變模型對既有詞元的行為，先跑一次回歸測試。
4. 安排訓練：新詞元的嵌入必須經過訓練才會有用。只擴充不訓練，效果會比不擴充更差。
5. 記錄：把擴充的詞元清單、初始化策略與時間寫進第 4.7.3 節的環境指紋。

第四點是最常被忽略的。第 7.6.1 節已經提過一次，這裡再強調：剛加進去的詞元對模型而言是全新的符號，它不知道那是什麼意思。第 22 章的持續預訓練會處理這件事，屆時你會實測「擴充後訓練多少步才追上」。

10.7 嵌入視覺化：看得見，但別看錯

10.7.1 三種降維方法

嵌入是幾百維的，人看不見。要看見它，必須降到二維或三維，而降維一定會失真。理解每種方法失真的方式，才能避免誤讀。

方法	原理	保留什麼	容易誤讀之處
PCA	找出變異最大的方向做線性投影	全域結構、大尺度距離	局部細節被壓扁
t-SNE	保持鄰近點的機率關係	局部群聚	群與群之間的距離無意義
UMAP	基於流形假設保持鄰域結構	局部與部分全域結構	參數敏感，群大小無意義

10.7.2 四個常見的誤讀

你看到的	你可能以為	實際上	正確做法
兩群靠得很近	這兩類語意相近	t-SNE 的群間距離沒有意義	回到原空間算相似度
某群特別大	這類詞比較多或比較分散	群的面積不代表密度	看實際樣本數
明顯的分群	模型學到了清楚的類別	可能是降維演算法造出來的	換參數或換方法重跑
兩點重疊	這兩個詞幾乎相同	高維的距離在二維無法完整表達	直接算餘弦相似度

本書的原則很簡單：視覺化用來產生假設，不用來下結論。你在圖上看到「臺灣的縣市名聚成一團」，那是一個值得檢驗的假設；要確認它，請回到原始的高維空間算相似度、做最近鄰檢索、或設計一個可量化的測試。

視覺化與量化驗證並行

```

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import torch.nn.functional as F

def visualize_and_verify(emb_weight, tokens, labels, tok,
                        seed=42, perplexity=30):
    """降維作圖的同時，在原空間做量化驗證"""
    ids = [tok.encode(t)[0] for t in tokens]
    X = emb_weight[ids].detach().float()

    # 1. 視覺化（產生假設）
    pca_xy = PCA(n_components=2, random_state=seed).fit_transform(X.numpy())
    tsne_xy = TSNE(n_components=2, random_state=seed,
                   perplexity=min(perplexity, len(ids) - 1),
                   init="pca").fit_transform(X.numpy())

    # 2. 原空間量化（檢驗假設）：同類的相似度是否高於異類？
    Xn = F.normalize(X, dim=-1)
    S = Xn @ Xn.T

```



```

same, diff = [], []
for i in range(len(ids)):
    for j in range(i + 1, len(ids)):
        (same if labels[i] == labels[j] else diff).append(float(S[i, j]))
report = {
    "mean_same": round(sum(same) / max(len(same), 1), 4),
    "mean_diff": round(sum(diff) / max(len(diff), 1), 4),
    "n_same": len(same), "n_diff": len(diff),
}
report["separation"] = round(report["mean_same"] - report["mean_diff"], 4)
return pca_xy, tsne_xy, report

```

separation 這個指標是本節的核心：它把「看起來有分群」變成一個數字。如果同類的平均相似度只比異類高 0.02，那麼你在圖上看到的分群多半是降維演算法的產物，不是模型真的學到了什麼。

10.7.3 臺灣語料上的視覺化實務

做嵌入視覺化時，繁體中文有幾個特有的注意事項，這些在英文教材中不會提到。

問題	現象	成因	處置
多字詞被切碎	「智慧製造」沒有單一嵌入	tokenizer 把它切成多個詞元	取子詞平均，並註明作法
字型缺字	圖上出現方框	matplotlib 預設字型無中文	指定支援繁中的字型
語言分群主導	中文全部聚一團、英文另一團	各向異性的語言版本	先做語言內中心化
高頻詞主導變異	PCA 前兩軸只反映詞頻	詞頻與向量長度相關	先正規化再降維

第一列是繁中特有的困擾。英文的多數常用詞在 tokenizer 中有單一詞元，可以直接取它的嵌入來作圖；繁中的多字詞經常被切成兩三塊，你必須決定怎麼合成一個代表向量。本書建議取子詞的平均，並在報告中明確說明——因為不同的合成方式會得到不同的圖，不說明就無法被檢驗。

第四列的處置有一個簡單的驗證方法：分別用原始向量與正規化後的向量做 PCA，比較前兩個主成分解釋的變異比例。如果原始版本的第一主成分解釋了異常高的比例（例如超過六成），那多半就是詞頻效應在主導，而不是語意結構。

繁中視覺化的前處理

```

import torch.nn.functional as F

def phrase_vector(emb_weight, tok, phrase: str, mode="mean"):
    """多字詞沒有單一詞元時，合成一個代表向量"""
    ids = tok.encode(phrase)
    vecs = emb_weight[ids]  # (n_sub, d)
    if mode == "mean":

```

```
        return vecs.mean(dim=0)
    if mode == "sum":
        return vecs.sum(dim=0)
    if mode == "first":
        return vecs[0]
    raise ValueError(mode)

def prepare_for_viz(X, center=True, normalize=True):
    """降維前的標準前處理，可大幅改善圖的可讀性"""
    if center:
        X = X - X.mean(dim=0, keepdim=True)      # 移除共同方向
    if normalize:
        X = F.normalize(X, dim=-1)                # 移除長度（詞頻）效應
    return X
```

程式實作

本章三個實作都在 A 級硬體上完成。它們的產出會直接進入第 11 至 13 章的模型實作。

程式 10A 嵌入層與三種位置編碼

目標：實作嵌入層、可學習位置嵌入、正弦位置編碼與 RoPE，並用測試驗證各自的關鍵性質。

測試	驗證什麼	通過標準	對應節次
形狀	各層的輸入輸出形狀正確	與斷言一致	第 3.1 節
正弦長度	每個位置向量的長度相同	皆為 $\sqrt{d/2}$	10.3.4
正弦衰減	內積隨距離下降	前段單調遞減	10.3.4
RoPE 相對性	內積只依賴 $m - n$	誤差小於 $1e-5$	10.4.2
RoPE 保長	旋轉不改變向量長度	誤差小於 $1e-5$	10.4.1
PAD 為零	padding 位置的嵌入恆為零	完全相等	10.1.2

tests/test_embedding_pos.py (節錄)

```
import math, torch, pytest
from formosa.model.embedding import TokenEmbedding, LearnedPositionEmbedding
from formosa.model.rope import build_rope_cache, apply_rope
from formosa.model.sinusoidal import sinusoidal_pe

def test_sinusoidal_norm():
    d = 64
    pe = sinusoidal_pe(128, d)
    expected = math.sqrt(d / 2)
    assert torch.allclose(pe.norm(dim=1),
                           torch.full((128,)), expected), atol=1e-4)

def test_sinusoidal_decay():
    pe = sinusoidal_pe(16, 64)
    ip = [float(pe[0] @ pe[k]) for k in range(6)]
    assert all(ip[i] > ip[i + 1] for i in range(5)), ip

@pytest.mark.parametrize("gap", [1, 2, 5, 17])
def test_rope_relative(gap):
    """相同的相對距離必須得到相同的內積"""
    dh = 64
    g = torch.Generator().manual_seed(0)
    q = torch.randn(dh, generator=g)
    k = torch.randn(dh, generator=g)
    cos, sin = build_rope_cache(512, dh)
```

```

def rot(v, pos):
    return apply_rope(v.view(1, 1, 1, -1),
                      cos[pos:pos + 1], sin[pos:pos + 1]).flatten()

ref = float(rot(q, gap) @ rot(k, 0))
for base in [3, 20, 100, 300]:
    got = float(rot(q, base + gap) @ rot(k, base))
    assert abs(got - ref) < 1e-4, (base, gap, got, ref)

def test_rope_preserves_norm():
    x = torch.randn(2, 4, 16, 64)
    cos, sin = build_rope_cache(16, 64)
    y = apply_rope(x, cos, sin)
    assert torch.allclose(x.norm(dim=-1), y.norm(dim=-1), atol=1e-5)

def test_pad_embedding_zero():
    e = TokenEmbedding(100, 32, pad_id=0)
    out = e(torch.tensor([[0, 5, 0]]))
    assert out[0, 0].abs().max() == 0
    assert out[0, 2].abs().max() == 0

```

延伸挑戰：把 RoPE 的 base 從 10000 改成 500000，重新畫出「內積對相對距離」的曲線。你會看到衰減變得更平緩——這就是 10.4.4 節說的「調高 base 等效延長上下文」的幾何意義。

程式 10B 臺灣專有名詞語意地圖

目標：用臺灣的地名、校名、產業術語建立語意地圖，檢查群聚是否符合直覺，並用原空間的量化指標驗證。

ch10/lab10b_semantic_map.py (節錄)

```

WORD_SETS = {
    "縣市": ["臺北", "新北", "桃園", "臺中", "臺南", "高雄",
              "雲林", "嘉義", "彰化", "屏東", "宜蘭", "花蓮"],
    "大學": ["臺大", "清大", "交大", "成大", "中央", "中興",
              "雲科大", "北科大", "臺科大"],
    "半導體": ["晶圓", "光罩", "曝光", "蝕刻", "封裝", "測試",
                "光阻", "薄膜", "黃光"],
    "工具機": ["主軸", "刀具", "夾治具", "進給", "五軸",
                "公差", "熱變形", "伺服"],
    "食物": ["滷肉飯", "珍珠奶茶", "蚵仔煎", "鳳梨酥",
              "牛肉麵", "臭豆腐"],
}

def build_labels(word_sets):
    words, labels = [], []
    for cat, ws in word_sets.items():
        words += ws
        labels += [cat] * len(ws)
    return words, labels

```

```
def analogy_tests(emb, tok):
    """類比測試：檢查向量算術是否合理"""
    cases = [
        ("臺南", "臺灣", "日本"),      # 城市之於國家
        ("晶圓", "半導體", "工具機"),  # 元件之於產業
        ("臺大", "臺北", "臺南"),      # 學校之於所在地
    ]
    for a, b, c in cases:
        print(f"{a} - {b} + {c} ≈ ", analogy(emb, tok, a, b, c, k=5))
```

必須回答的四個問題（寫進報告）：

- 五類詞彙在 t-SNE 圖上是否分群？把 separation 指標算出來，判斷分群是真的還是視覺假象。
- 哪一類的內部相似度最高、哪一類最分散？為什麼？（提示：與該類詞彙在訓練語料中的出現頻率有關）
- 類比測試的結果合理嗎？如果不合理，可能的原因是什麼？（提示：靜態嵌入層 vs 情境化表示，見 10.2.1 節）
- 把同一組詞用兩個不同的模型（例如一個繁中適配的、一個國際通用的）各做一次，比較 separation 的差異。這個差異說明了什麼？

一個預期中的「失敗」

你很可能會發現類比測試的結果不如 Word2Vec 時代的經典示範漂亮。這不是你做錯了——大模型的嵌入層是靜態的、且經過整個模型的聯合訓練，它的幾何結構與專門訓練的詞向量不同。把這個發現誠實寫進報告，並用 10.2.1 節的分工來解釋，比硬湊出漂亮結果有價值得多。

程式 10C 詞彙擴充的初始化實驗

目標：比較隨機初始化與子詞平均初始化對收斂速度的影響。這是本書第一個「同一件事兩種做法，用實驗決定」的完整案例，方法論沿用第 5.5 節。

ch10/lab10c_init_ablation.py (主流程)

```
import torch, json
from pathlib import Path
from formosa.model.expand import expand_embeddings

def run_ablation(base_model_fn, tok, new_tokens, train_loader,
                 valid_loader, seeds=(0, 1, 2), steps=2000):
    """固定所有變因，只改初始化策略（見第 5.5.2 節）"""
    results = []
    for strategy in ["random", "mean", "subword_mean"]:
        for seed in seeds:
            torch.manual_seed(seed)
            model = base_model_fn()
```

```

expand_embeddings(model, tok, new_tokens, strategy=strategy,
                  noise_std=1e-4)
curve = train_and_track(model, train_loader, valid_loader,
                       steps=steps, eval_every=100)
results.append({
    "strategy": strategy, "seed": seed,
    "curve": curve,          # [(step, valid_loss)]
    "final": curve[-1][1],
    # 追上 baseline 需要幾步？
    "steps_to_target": next(
        (s for s, l in curve if l <= TARGET_LOSS), None),
})

# 依策略彙整：中位數與範圍（見第 5.5.2 節的多 seed 要求）
summary = {}
for st in ["random", "mean", "subword_mean"]:
    rows = [r for r in results if r["strategy"] == st]
    finals = sorted(r["final"] for r in rows)
    summary[st] = {
        "median_final": finals[len(finals) // 2],
        "range": [finals[0], finals[-1]],
        "steps_to_target": [r["steps_to_target"] for r in rows],
    }
return results, summary

```

實驗設計的四個要求（這是第 5 章紀律的第一次實戰）：

- 固定變因：同一個底模、同一組新詞元、同一份資料、同樣的訓練步數與學習率。只有初始化策略不同。
- 至少三個 seed，報告中位數與範圍，不報告單次結果。
- 主要指標是「達到目標損失所需的步數」而非「最終損失」——因為給足夠時間三者可能都會收斂，差異在於速度。
- 同時記錄既有詞元的表現有沒有退化（回歸測試），確認擴充沒有傷害原有能力。

系統案例：臺灣專有名詞語意檢索原型

本章的系統案例是建立一個小型的語意檢索原型：輸入一個臺灣的校務、法規或製造術語，回傳語意最相近的候選。這是第 30 章 RAG 系統的最小雛形，也是驗證嵌入品質的實用工具。

系統規格

元件	功能	本章的做法	第 30 章會升級成
向量來源	把文字變成向量	嵌入層的平均池化	專用的句子嵌入模型
索引	存放並快速檢索	NumPy 矩陣 + topk	向量資料庫 + ANN
相似度	衡量距離	餘弦相似度	混合檢索（稀疏 + 密集）

元件	功能	本章的做法	第 30 章會升級成
重排序	精修結果	無	cross-encoder 重排
評測	衡量好壞	recall@k	完整的檢索評測體系

評測設計

沒有評測的檢索系統只是一個會回傳東西的程式。請建立一份至少 50 題的小型評測集，每題包含一個查詢與至少一個正確答案。

題型	範例	測什麼	難度
同義詞	查「教育部」→ 應找到「教育主管機關」	語意等價	中
上下位	查「工具機」→ 應找到「五軸加工機」	層級關係	中
領域關聯	查「光阻」→ 應找到「曝光」「顯影」	共現關係	低
地理關聯	查「雲林」→ 應找到「斗六」「虎尾」	在地知識	高
縮寫還原	查「雲科大」→ 應找到「國立雲林科技大學」	別名對應	高

最後兩列是本案例的重點，也是國際模型最容易失手的地方。臺灣的地名層級與校名縮寫幾乎不會出現在英文為主的訓練語料中，所以這兩類的 recall 通常明顯低於前三類。把這個差距量出來，就是第 22 章要做繁中適配的具體證據。

交付要求

- 可運行的檢索原型，支援至少 5,000 個詞條的索引與查詢。
- 50 題評測集，含五種題型，每題標註正確答案與出處。
- recall@1、recall@5、recall@10 的分題型結果。
- 至少兩個模型的比較（一個繁中適配、一個國際通用），並解釋差異來源。
- 各向異性檢查：報告全體詞條兩兩相似度的平均與分布，判斷相似度數值是否可直接解讀。

- 一頁分析：哪一類查詢表現最差？你認為原因是嵌入不好、tokenizer 不好、還是語料不足？如何驗證你的判斷？

章末評量

一、概念題

1. 說明 one-hot 乘法與查表為什麼等價，以及為什麼實作上一定用後者。
2. 大模型的嵌入層是靜態的還是情境化的？情境化在哪裡發生？這對句子嵌入的取用方式有什麼意涵？
3. 什麼是各向異性？它如何影響餘弦相似度的判讀？請提出三種對策。
4. 為什麼注意力機制對順序無感？請用排列等變的性質說明。
5. 可學習位置嵌入的致命限制是什麼？為什麼 RoPE 沒有這個問題？
6. 解釋 RoPE 的相對位置性質，並說明為什麼這個性質對語言建模是合理的。
7. 權重綁定的利弊各是什麼？為什麼小模型較常綁定？
8. 為什麼詞彙擴充後必須繼續訓練？只擴充不訓練為什麼會比不擴充更差？

二、計算題

1. 一個 $V = 48,000$ 、 $d = 1,024$ 的模型。計算嵌入層的參數量；若輸出層不綁定，兩層合計多少？用 BF16 儲存需要多少 MB？
2. 承上題，若用 AdamW 訓練，僅這兩層的最佳化器狀態 (m 與 v ，FP32) 需要多少 MB？
3. 正弦位置編碼在 $d = 128$ 時，每個位置向量的長度是多少？請寫出推導。
4. RoPE 的 $\text{base} = 10000$ 、 $\text{head_dim} = 64$ 。計算第 0 個與第 31 個頻率分量 θ_j 的值，並說明兩者的週期相差幾個數量級。
5. ALiBi 在 16 個注意力頭時的斜率序列是什麼？最陡與最緩的斜率相差幾倍？
6. 某模型上下文 4,096，你想用位置內插延伸到 16,384。內插後相鄰位置的角度差變成原來的幾分之一？這對短距離解析度有什麼影響？

三、除錯題

1. 同學實作的 RoPE 測試中，相同相對距離的內積不一致。請列出三個最可能的原因與檢查順序。
2. 同學宣稱做了權重綁定，但訓練後發現輸入嵌入與輸出投影的數值不同。請說明原因與正確做法。
3. 同學做了詞彙擴充，模型能理解新詞但從不生成它們。請指出最可能的原因。
4. 同學的語意檢索原型對任何查詢都回傳幾乎相同的一批結果。請列出三個可能原因。
5. 同學的 t-SNE 圖顯示清楚的五個群，但 separation 指標只有 0.01。請解釋這代表什麼，以及該相信哪一個。

四、系統設計題

1. 你要為一個「校務法規檢索」系統選擇位置編碼方案。文件長度從 200 字到 20,000 字不等，且未來可能需要處理更長的彙編本。請在四種方案中選擇並論證，需涵蓋外推能力、實作複雜度、生態相容性與未來擴充四個面向。
2. 你的團隊要在一個 7B 開放權重模型上擴充 8,000 個臺灣專有名詞。請設計完整流程：候選詞如何挑選、初始化用什麼策略、如何驗證擴充沒有破壞既有能力、以及需要多少訓練才算完成。並說明你會用什麼指標判斷「值得擴充」。

五、臺灣情境與倫理題

1. 本章的語意地圖實驗中，臺灣地名與校名縮寫的表現通常較差。請說明這個現象的成因鏈（從語料到 tokenizer 到嵌入），並指出在哪一個環節介入的成本最低。
2. 嵌入空間會反映訓練語料中的關聯，包括不該有的關聯（例如某些職業與性別的連結）。請設計一個量測方法，檢查你的模型在臺灣情境下是否有類似問題，並說明發現問題後你會怎麼做。

評量提示（給自學者）

- 計算題第 1 題： $48,000 \times 1,024 = 4,915.2$ 萬；不綁定則約 9,830.4 萬；BF16 下約 98.3 MB 與 196.6 MB。
- 計算題第 2 題： m 與 v 各為 FP32（4 bytes），故 $9,830.4 \text{ 萬} \times 8 \approx 786.4 \text{ MB}$ 。
- 計算題第 3 題：每個位置有 $d/2 = 64$ 對 $(\sin, \cos)$ ，每對的平方和為 1，故長度為 $\sqrt{64} = 8$ 。
- 計算題第 4 題： $\theta_0 = 10000^0 = 1$ ； $\theta_{31} = 10000^{(-62/64)} \approx 1.33 \times 10^{-4}$ 。週期相差約四個數量級。
- 計算題第 5 題：起始值為 $2^{(-8/16)} \approx 0.7071$ ，之後為其冪次；最陡約 0.7071、最緩約 $0.7071^{16} \approx 0.0039$ ，兩者相差約 181 倍。
- 計算題第 6 題：變成原來的四分之一；短距離的角度差變小，相鄰位置的可區辨性下降，這是位置內插的主要代價。
- 除錯題第 5 題：應相信 separation。t-SNE 會在沒有真實結構的資料上造出視覺分群，這是它已知的行為。

研究延伸與版本注意事項

- RoPE 原始論文（RoFormer）：篇幅適中、推導清楚，是本章 10.4 節的直接來源，建議精讀旋轉矩陣的部分。

- ALiBi 論文：對長度外推問題的分析很有啟發性，即使你不使用 ALiBi 也值得一讀。
- 位置內插與 NTK 縮放的相關研究：第 20 章會完整處理，若你的專題需要長上下文可提早了解。
- 權重綁定的原始提案與後續實證：關於什麼情況下綁定有益，文獻中有不一致的結論，值得自己在任務上量測。
- 嵌入各向異性的研究：這個現象在多篇論文中被量化分析，對第 30 章的檢索工作有直接參考價值。
- 降維視覺化的誤讀：關於 t-SNE 與 UMAP 如何被誤讀，有幾篇很好的互動式說明，建議在做程式 10B 之前先看過。
- 本章的數學不會過時，但 RoPE 的 base 設定、縮放方法與函式庫 API 變動較快。程式碼以課程倉庫為準。

常見問題

問：為什麼不直接用 Word2Vec 的向量當作嵌入層的初始值？答：可以，而且在小資料的情境下偶爾有幫助。但大模型的嵌入是與整個模型聯合訓練的，它需要的表示與 Word2Vec 追求的目標不同。實務上，隨機初始化加上足夠的訓練，效果通常優於用外部詞向量初始化。

問：RoPE 與 ALiBi 到底該選哪個？答：如果你要做的是持續預訓練或微調（第六篇），答案由底模決定——你不能換。如果你從零訓練，本書建議 RoPE，主要理由是生態成熟：工具、加速核心、長上下文技術都圍繞它發展。

問：位置編碼要不要加在每一層？答：加法式的（可學習、正弦）通常只加在輸入端，靠殘差連接把位置資訊帶到深層。RoPE 則是每一層的注意力都要套用，因為它作用在 Q 與 K 上。這是兩種做法的實作差異，第 12 章組裝 block 時會再遇到。

教師使用建議

本章排在第 10 週，是第 3 篇的第一章，建議 2 小時講授加 1 小時實驗。本章的教學重點是「讓抽象的向量變得可觸摸」，大量使用視覺化與現場演算。

時段	內容	互動設計	注意事項
前 20 分鐘	10.1 至 10.2 節，嵌入與語意空間	現場跑一次類比測試，讓學生猜結果	猜錯的例子最有教學價值

時段	內容	互動設計	注意事項
中間 25 分鐘	10.3 節，為什麼需要位置編碼	在白板上示範「狗咬人 / 人咬狗」的注意力分數相同	排列等變要讓學生自己推一次
後 35 分鐘	10.4 節，RoPE	用兩個二維向量在白板上畫旋轉，示範內積不變	這是全章最需要慢講的一段
最後 30 分鐘	10.5 至 10.7 節與作業說明	展示同一組資料的三種降維圖，請學生說出差異	誤讀的示範效果極佳
實驗課 1 小時	程式 10A 與 10B	兩人一組，各自出一組臺灣詞彙互相檢驗	10C 作為課後作業

一個效果極佳的白板示範：在二維平面上畫兩個向量，標出它們的夾角，然後把兩個向量各自旋轉 30 度，讓學生看到夾角不變、內積不變。接著問：如果一個轉 50 度、一個轉 20 度呢？學生會自己發現「只有差值有關」。這比任何公式推導都快。

高中授課的調整：10.1、10.2 完整教（語意地圖對高中生很有吸引力）；10.3 只教「為什麼需要位置」與可學習位置嵌入，正弦編碼的公式可略；10.4 用旋轉的幾何直覺講，不推公式，但務必做那個白板示範；10.5 至 10.7 挑重點，視覺化誤讀的部分值得完整講。程式 10A 只做 RoPE 的驗證測試，10B 完整做。

本章小結

1. 嵌入層本質上是查找表；one-hot 乘法與查表等價，但實作上必須用後者。
2. 嵌入層的參數量是 $V \times d$ ，在大詞彙表或小模型上可能占總參數的極高比例。
3. 大模型的嵌入層是靜態的，情境化發生在後續的注意力層——取用句子嵌入時不能只用嵌入層輸出。
4. 餘弦相似度衡量方向；各向異性會讓所有相似度都偏高，應以相對排序而非絕對值判讀。
5. 注意力對順序無感（排列等變），因此必須額外注入位置資訊。
6. 可學習位置嵌入無法超出訓練長度，因為那些位置的參數從未被訓練過。
7. 正弦位置編碼用不同頻率的三角函數，每個位置向量長度相同，內積隨距離下降但遠距離趨於平緩。
8. RoPE 用旋轉表示位置，其內積只依賴相對位置 $m - n$ ，這正是語言建模需要的性質。
9. RoPE 仍有外推限制，可用位置內插、調高 base、NTK 縮放或 YaRN 處理，第 20 章會展開。

10. ALiBi 在注意力分數上加線性距離懲罰，不同頭使用幾何級數的斜率，設計上支援外推。
11. 權重綁定省下 $V \times d$ 個參數並帶來正則效果，但限制表達能力；必須用 is 驗證是同一個物件。
12. 詞彙擴充應用子詞平均初始化並加微量雜訊；擴充後必須訓練，且輸出層需同步處理。
13. 視覺化用來產生假設，不用來下結論；t-SNE 的群間距離無意義，應回到原空間量化驗證。

成果檢核

完成本章後你必須繳交：

- 嵌入與位置編碼模組（程式 10A）：通過六項測試，特別是 RoPE 的相對位置性質與保長性質。
- 嵌入分析報告（程式 10B）：五類臺灣詞彙的語意地圖、separation 指標、類比測試結果與誠實的失敗分析。
- 位置編碼比較：三種方法在同一組設定下的性質對照，含正弦編碼的內積衰減曲線與 RoPE 不同 base 的比較。
- 初始化實驗報告（程式 10C）：三種策略、三個 seed、達到目標損失所需步數的中位數與範圍。
- 語意檢索原型：可運行的系統、50 題評測集、分題型的 recall 結果、以及各向異性檢查。
- 一頁反思：哪一類臺灣詞彙的表現最差？成因鏈在哪裡？你會在哪個環節介入？

本章的嵌入模組與 RoPE 實作會被第 11 章（注意力）直接 import，第 12 章組裝 block 時會再次使用，第 13 章的 MiniFormosaGPT 則會把它們接成完整的模型。從這一章開始，你寫的每一行程式都會活到第 40 章。

第 11 章

自注意力、因果遮罩與多頭注意力

Self-Attention, Causal Masking, and Multi-Head Attention

難度：核心 | 建議授課：第 11 週 | 硬體需求：A 級可完成全部實作；有 GPU 者可加測長序列的記憶體曲線

叫獸開講

這一章是整本書的心臟。寫完之後，你不會再覺得注意力是黑盒子——你會知道那張 $T \times T$ 的矩陣裡，每一格代表誰在看誰、看得多用力，以及為什麼那個看似多餘的除以根號 d ，其實是讓整個模型能訓練起來的關鍵。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明長序列建模的資訊瓶頸，以及注意力如何用「直接連結」取代「逐步傳遞」。
2. 實作無可訓練權重的簡化自注意力，並解釋它為什麼還不夠。
3. 推導縮放點積注意力，說明 Q 、 K 、 V 三者的分工與除以根號 d 的必要性。
4. 正確實作因果遮罩，並用測試證明模型無法看到未來。
5. 說明 dropout 在注意力中的位置，以及訓練與推論的行為差異。
6. 把單頭擴展為多頭，正確處理形狀變換與權重切分。
7. 比較 MHA、MQA 與 GQA 的推論記憶體差異，並計算 KV cache 的實際大小。
8. 分析注意力的計算與記憶體複雜度，說明 FlashAttention 的核心想法。
9. 用注意力熱圖觀察繁體中文句子中的指代、標點與長距依賴關係。

本章的分量

這是第 3 篇最厚、也最需要動手的一章。建議把時間分配成：讀 11.1 到 11.3 一次、把程式 11A 的四個版本逐一寫出來、再回頭讀 11.4 到 11.6。純讀不寫的話，你會覺得都懂了，然後在第 13 章組裝模型時發現形狀對不起來。這一章的價值有八成在鍵盤上。

11.1 為什麼需要注意力

11.1.1 資訊瓶頸的三種形式

第 2 章從技術史的角度講過這件事，這裡從資訊流的角度再看一次。要處理一個長度 T 的序列，讓每個位置都能取得需要的資訊，有三種基本做法。

做法	機制	路徑長度	平行度	問題
循環	把歷史壓進一個隱藏狀態	$O(T)$	無	梯度衰減、無法平行
卷積	用固定視窗逐層擴大感受野	$O(\log T)$	高	需要疊很多層才看得遠
注意力	每個位置直接連到每個位置	$O(1)$	高	計算與記憶體為 T^2

「路徑長度」這一欄是關鍵。它指的是資訊從位置 i 傳到位置 j 需要經過幾次運算。在 RNN 中，第 1 個字的資訊要傳到第 100 個字，必須經過 99 次狀態更新，每一次都可能損失一些；在注意力中，這是一步到位的。

第 2.4.5 節的那個例子在這裡有了精確的說明：「那位在雲林科技大學教機器人學.....的老師，昨天在課堂上宣布」——「宣布」要取得「老師」的資訊，在 RNN 中路徑長度是二十幾，在注意力中是 1。

11.1.2 注意力的三個角色

在寫任何程式之前，先建立一個直覺。注意力可以想成一個查資料的動作，涉及三個角色。

角色	英文	直覺	在句子中的例子
查詢	Query	我現在需要什麼資訊	「他」需要知道自己指誰
鍵	Key	我是什麼，可以被誰找到	「小華」宣告自己是一個人名
值	Value	如果被找到，我提供什麼內容	「小華」提供關於小華的語意

這個比喻的關鍵在於 Key 與 Value 是分開的。「用什麼被找到」與「被找到後提供什麼」不必是同一件事——這給了模型額外的表達自由。一個位置可以用某個特徵讓別人找到它，卻提供完全不同的內容。

整個注意力的流程只有三步：用 Query 與所有 Key 算相似度、把相似度變成權重、用權重對所有 Value 做加權平均。接下來的三節就是把這三步逐步做嚴謹。

11.2 版本一：沒有可訓練權重的簡化注意力

11.2.1 最樸素的做法

先做一個沒有任何參數的版本。輸入是一串向量，我們直接拿它們互相做內積當作相似度。

$$score_{ij} = x_i \cdot x_j \quad ; \quad \alpha_{ij} = softmax_j(score_i) \quad ; \quad z_i = \sum_j \alpha_{ij} x_j$$

formosa/model/attention_v1.py — 無參數版本

```
import torch
import torch.nn.functional as F

def simple_attention(x: torch.Tensor) -> torch.Tensor:
    """x: (T, d) -> (T, d) 最樸素的自注意力，沒有任何參數"""
    T, d = x.shape
    scores = x @ x.T # (T, T)
    assert scores.shape == (T, T)
    weights = F.softmax(scores, dim=-1) # 每一列總和為 1
    out = weights @ x # (T, d)
    assert out.shape == x.shape
    return out

def simple_attention_loop(x):
    """同樣的計算，但用迴圈寫，方便對照理解"""
    T, d = x.shape
    out = torch.zeros_like(x)
    for i in range(T):
        s = torch.tensor([x[i] @ x[j] for j in range(T)]) # (T,)
        a = F.softmax(s, dim=0)
        out[i] = sum(a[j] * x[j] for j in range(T))
    return out
```

把兩個版本都寫出來並驗證結果相同，是程式 11A 的第一個測試。迴圈版本讓你看懂在做什麼，矩陣版本讓你看懂為什麼 GPU 喜歡它——第 3.2.2 節說過，能寫成矩陣乘法的就不要寫迴圈。

11.2.2 它有什麼問題

這個版本可以跑，也確實做了「每個位置蒐集其他位置的資訊」這件事。但它有三個致命限制。

限制	為什麼	後果	解法
沒有可學習參數	相似度完全由輸入決定	模型無法學習「該注意什麼」	加入 Q、K、V 投影
自己永遠最像自己	$x_i \cdot x_i$ 通常是該列最大值	注意力被自己主導	投影後 Q 與 K 不同
Key 與 Value 綁死	用同一個 x 當作兩者	無法分離「怎麼被找到」與「提供什麼」	分開投影

第二列值得實測一次：跑 `simple_attention` 並印出 `weights` 的對角線，你會發現大部分位置的注意力都集中在自己身上，因為向量與自己的內積就是長度平方，通常是最大的。這意味著這個版本幾乎沒做什麼事——每個位置基本上還是原來的自己。

第一列是最根本的。整個模型的學習能力來自參數，而這個版本一個參數都沒有。它沒辦法學會「動詞應該注意主詞」或「代名詞應該注意先行詞」，因為它沒有任何可以調整的東西。

11.3 版本二：縮放點積注意力

11.3.1 三個投影

解法是引入三個可學習的線性投影，把同一個輸入分別變成 Query、Key 與 Value。

$$Q = X W^q \quad , \quad K = X W^k \quad , \quad V = X W^v$$

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_h}) V$$

這就是 Transformer 的核心公式。它看起來簡單，但每一個部分都有理由，接下來逐一拆解。

元件	形狀	作用	若拿掉會怎樣
W^q	(d, d_h)	決定「我要找什麼」	無法學習查詢意圖
W^k	(d, d_h)	決定「我如何被找到」	無法學習索引特徵
W^v	(d, d_h)	決定「我提供什麼」	無法與 Key 分離
$Q K^T$	(T, T)	所有位置對所有位置的相似度	—
$\div \sqrt{d_h}$	純量	控制數值範圍	softmax 飽和，梯度消失
softmax	(T, T)	正規化成機率分布	權重無法解釋、可能為負
$\times V$	(T, d_h)	加權平均	—

11.3.2 為什麼要除以根號 d：一個實測

第 2.5.4 節提過這個縮放，這裡把理由講到底，因為它是一個很好的「數值行為決定訓練成敗」的案例。

假設 Q 與 K 的每個元素都是獨立的、平均為 0、變異數為 1 的隨機變數。那麼它們的內積是 d_h 個

乘積的和，其變異數約等於 d_h 。

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_h} \text{Var}(q_i k_i) \approx d_h$$

用程式驗證這個推導：

d_h	實測 Var(q·k)	理論值	除以 $\sqrt{d_h}$ 後
16	16.0	16	0.998
64	63.3	64	0.989
256	257.8	256	1.007

變異數大代表數值範圍大。把很大的數餵給 softmax 會發生什麼？以 $d_h = 64$ 、 $T = 8$ 的隨機資料為例：

情況	最大分數絕對值	最大機率	注意力熵
不縮放	20.4	0.9999	0.301
除以 $\sqrt{d_h}$	2.6	0.6119	1.688
完全均勻（參考）	0	0.125	2.079

不縮放時，softmax 的輸出幾乎是 one-hot：最大機率 0.9999，熵只有 0.301（均勻分布是 2.079）。這代表每個位置只看得到一個位置，其他全部被忽略。

更嚴重的是梯度。softmax 在飽和區的梯度接近零——當某個機率是 0.9999 時，它的梯度是 $0.9999 \times 0.0001 \approx 0.0001$ 。訓練初期若落在這個區域，模型幾乎學不動。除以 $\sqrt{d_h}$ 把變異數拉回 1 左右，讓 softmax 保持在有梯度的區域。

一個常被問到的問題

為什麼是除以 $\sqrt{d_h}$ 而不是 d_h ？因為我們要調整的是標準差而不是變異數。變異數是 d_h ，標準差就是 $\sqrt{d_h}$ ；除以標準差之後，變異數變成 1。這是統計標準化的基本操作，只是在這裡被用來保護 softmax。

11.3.3 實作

| formosa/model/attention_v2.py — 單頭縮放點積注意力

```

import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class SingleHeadAttention(nn.Module):
    def __init__(self, d_model: int, d_head: int, bias: bool = False):
        super().__init__()
        self.w_q = nn.Linear(d_model, d_head, bias=bias)
        self.w_k = nn.Linear(d_model, d_head, bias=bias)
        self.w_v = nn.Linear(d_model, d_head, bias=bias)
        self.scale = 1.0 / math.sqrt(d_head)
        self.d_head = d_head

    def forward(self, x, return_weights: bool = False):
        # x: (B, T, d_model)
        B, T, d = x.shape
        q = self.w_q(x) # (B, T, d_h)
        k = self.w_k(x)
        v = self.w_v(x)
        assert q.shape == (B, T, self.d_head), q.shape

        scores = (q @ k.transpose(-2, -1)) * self.scale # (B, T, T)
        assert scores.shape == (B, T, T), scores.shape

        weights = F.softmax(scores, dim=-1)
        out = weights @ v # (B, T, d_h)
        assert out.shape == (B, T, self.d_head)
        return (out, weights) if return_weights else out

```

注意 `bias` 預設為 `False`。多數現代 Transformer 的 Q、K、V 投影不使用 `bias`——因為緊接著就會做 `softmax`（對整列加同一個常數不改變結果），`bias` 的效果有限，拿掉可以省參數也略微加速。這是一個小細節，但你在第 22 章載入別人的權重時必須確認雙方一致。

11.4 版本三：因果遮罩

11.4.1 為什麼要遮

第 5.2 節已經從訓練目標的角度講過因果遮罩，這裡從實作的角度做一次。

自回歸語言模型要預測第 t 個位置的下一個詞元。如果模型在計算第 t 個位置的表示時可以看到第 $t+1$ 個位置，那它就直接看到答案了——訓練損失會迅速降到接近零，但模型什麼都沒學到，生成時完全不能用。

這是本書最強調的一種錯誤：它不會報錯，訓練曲線還特別漂亮。第 4.3.2 節的因果性測試就是為了抓它而設計的。

11.4.2 三種實作方式

方式	做法	優點	注意
加法遮罩	被遮處加上一個極大的負數	最常用，與 softmax 天然配合	不要用 $-\text{inf}$ ，見下文
布林遮罩	用 <code>masked_fill</code> 填入極小值	語意清楚	本質上同上
下三角乘法	權重乘上下三角矩陣後重新正規化	直觀	多一次正規化，較慢

關於「不要用 $-\text{inf}$ 」：如果整列都被遮住（例如某些 padding 情境），softmax 會得到 0/0 而產生 NaN。安全的做法是使用該 dtype 能表示的最小值。

formosa/model/masking.py

```
import torch

def build_causal_mask(T: int, device=None) -> torch.Tensor:
    """回傳 (T, T) 布林遮罩，True 表示「要被遮掉'"""
    return torch.triu(torch.ones(T, T, dtype=torch.bool, device=device),
                      diagonal=1)

def apply_causal_mask(scores: torch.Tensor) -> torch.Tensor:
    """scores: (B, H, Tq, Tk)"""
    B, H, Tq, Tk = scores.shape
    assert Tq == Tk, f"自注意力需 Tq == Tk，得到 {Tq} 與 {Tk}"
    m = build_causal_mask(Tq, scores.device)
    # 用 dtype 的最小值而非 -inf，避免整列被遮時產生 NaN
    neg = torch.finfo(scores.dtype).min
    return scores.masked_fill(m[None, None, :, :], neg)

def apply_padding_mask(scores: torch.Tensor,
                       pad_mask: torch.Tensor) -> torch.Tensor:
    """pad_mask: (B, T)，True 表示該位置是 PAD"""
    B, H, Tq, Tk = scores.shape
    assert pad_mask.shape == (B, Tk), pad_mask.shape
    neg = torch.finfo(scores.dtype).min
    return scores.masked_fill(pad_mask[:, None, None, :], neg)
```

11.4.3 必須通過的驗證

因果遮罩是全書最需要測試的元件之一。以下四個測試在程式 11A 中是強制的。

tests/test_causal.py

```
import torch, pytest
import torch.nn.functional as F
from formosa.model.masking import apply_causal_mask
```

```

def test_upper_triangle_zero():
    """遮罩後 softmax，上三角必須全為 0"""
    scores = torch.randn(2, 4, 8, 8)
    attn = F.softmax(torch.nn.functional.apply_causal_mask(scores), dim=-1)
    upper = torch.triu(torch.ones(8, 8, dtype=torch.bool), diagonal=1)
    assert attn[..., upper].abs().max() < 1e-9

def test_rows_sum_to_one():
    scores = torch.randn(2, 4, 8, 8)
    attn = F.softmax(torch.nn.functional.apply_causal_mask(scores), dim=-1)
    assert torch.allclose(attn.sum(-1), torch.ones(2, 4, 8), atol=1e-6)

def test_first_position_only_self():
    """第 0 個位置只能看到自己，權重必為 1"""
    scores = torch.randn(1, 1, 6, 6)
    attn = F.softmax(torch.nn.functional.apply_causal_mask(scores), dim=-1)
    assert abs(float(attn[0, 0, 0, 0]) - 1.0) < 1e-6

def test_future_does_not_affect_past(attn_module):
    """改變未來的輸入，過去的輸出必須完全不變
    這是最重要的一個測試——它直接驗證因果性"""
    torch.manual_seed(0)
    x = torch.randn(1, 10, 64)
    x2 = x.clone()
    x2[0, -1] = torch.randn(64) # 只改最後一個位置
    with torch.no_grad():
        a = attn_module(x)
        b = attn_module(x2)
    assert torch.allclose(a[0, :-1], b[0, :-1], atol=1e-5), \
        "未來的輸入影響了過去的輸出，因果遮罩失效"

def test_no_nan_with_full_mask():
    """整列被遮時不應產生 NaN（用 dtype 最小值而非 -inf）"""
    scores = torch.randn(1, 1, 4, 4)
    neg = torch.finfo(scores.dtype).min
    scores[0, 0, 0, :] = neg # 整列遮死
    attn = F.softmax(scores, dim=-1)
    assert not torch.isnan(attn).any()

```

第四個測試是最重要的，因為它不依賴實作細節——不管你的遮罩怎麼寫，只要因果性成立，這個測試就會通過。這種「測性質而非測實作」的測試最有價值，因為它在你重構程式碼時仍然有效。

11.4.4 dropout 放在哪裡

Transformer 中有三個常見的 dropout 位置，其中一個在注意力內部。

位置	作用對象	效果	現代模型的做法
注意力 dropout	softmax 之後的權重	隨機忽略某些位置的資訊	大模型常設為 0
殘差 dropout	子層輸出加回殘差之前	正則化整個子層	同上
嵌入 dropout	詞嵌入加位置編碼之後	輸入層的正則化	同上

注意力 dropout 有一個容易誤解的地方：它作用在 softmax 之後的權重上，所以被丟棄之後那一列的總和不再是 1 (PyTorch 的 dropout 會把保留的部分放大 $1/(1-p)$ 以維持期望值，但單次的總和仍會偏離 1)。這是設計上可以接受的——它就是要製造擾動。

為什麼大模型常把 dropout 設為 0？因為它們的資料量夠大，過擬合不是主要問題，而 dropout 會拖慢收斂。本書第 13 章的 MiniFormosaGPT 因為語料有限，預設會開一點 dropout；但你在做第 22 章的持續預訓練時，多數底模是以 dropout = 0 訓練的，要注意保持一致。

加入 dropout 的注意力

```
class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, d_head, attn_dropout=0.0):
        super().__init__()
        self.w_q = nn.Linear(d_model, d_head, bias=False)
        self.w_k = nn.Linear(d_model, d_head, bias=False)
        self.w_v = nn.Linear(d_model, d_head, bias=False)
        self.attn_drop = nn.Dropout(attn_dropout)
        self.scale = 1.0 / math.sqrt(d_head)

    def forward(self, x):
        q, k, v = self.w_q(x), self.w_k(x), self.w_v(x)
        scores = (q @ k.transpose(-2, -1)) * self.scale
        scores = apply_causal_mask(scores.unsqueeze(1)).squeeze(1)
        w = F.softmax(scores, dim=-1)
        w = self.attn_drop(w)          # 注意：在 softmax 之後
        return w @ v
```

11.5 版本四：多頭注意力

11.5.1 為什麼要多頭

單頭注意力有一個結構性的限制：每個位置只能產生一組注意力權重，也就是只能表達一種關係。但語言中的關係是多重的——同一個動詞可能同時需要注意它的主詞、受詞、時間狀語與否定詞。

多頭的想法是：與其用一個高維的注意力，不如把維度切成 H 份，各自獨立算注意力，最後把結果拼回來。每個頭可以專注於不同類型的關係。

$$MultiHead(X) = Concat(head_1, \dots, head_H) W^o \quad , \quad head_h = Attention(X W^q_h, X$$

$$W^k_h, X W^v_h)$$

面向	單頭 $d_h = d$	多頭 $H \times (d/H)$	差異
參數量	$3 \times d \times d$	$3 \times d \times d$ (相同)	總參數不變
計算量	相同量級	相同量級	幾乎不變
注意力矩陣	1 個 $T \times T$	H 個 $T \times T$	記憶體 $\times H$
表達能力	一種關係	H 種關係	多頭勝

注意第一列：多頭並沒有增加參數。它是把同樣的參數重新切分使用——原本一個 (d, d) 的投影，現在看成 H 個 $(d, d/H)$ 的投影並排。這是一個「免費的午餐」：同樣的成本換到更豐富的表達。

但第三列說明代價在哪：注意力矩陣從一個變成 H 個，記憶體占用乘以 H 。這在長序列時會成為主要瓶頸，11.7 節會算給你看。

11.5.2 形狀變換：本章最容易寫錯的地方

多頭的實作困難不在數學，而在張量形狀的來回變換。第 3.1.2 節的那張表在這裡要真正用上了。

步驟	輸入形狀	輸出形狀	操作
1. 投影	(B, T, d)	$(B, T, 3d)$	一次算出 QKV
2. 拆分 QKV	$(B, T, 3d)$	$3 \times (B, T, d)$	chunk 或 split
3. 拆頭	(B, T, d)	(B, T, H, d_h)	view / reshape
4. 轉置	(B, T, H, d_h)	(B, H, T, d_h)	transpose(1, 2)
5. 注意力	(B, H, T, d_h)	(B, H, T, d_h)	每個頭獨立計算
6. 轉置回	(B, H, T, d_h)	(B, T, H, d_h)	transpose(1, 2)
7. 合併頭	(B, T, H, d_h)	(B, T, d)	contiguous + view
8. 輸出投影	(B, T, d)	(B, T, d)	W^o

第七步的 contiguous 是最容易漏掉的一行。transpose 只是改變 stride，不會真的搬動記憶體；而 view 要求記憶體是連續的。漏掉它，PyTorch 會拋出一個看起來莫名其妙的錯誤訊息。這是初學者在

這一章卡最久的地方。

一個可以省下三小時的建議

在第 4 步與第 6 步之間，把每一步的形狀用 `assert` 寫下來。多寫六行，你就不會在第 13 章組裝模型時對著一個 (8, 512, 12, 64) 與 (8, 12, 512, 64) 的形狀不符錯誤發呆——因為錯誤會在發生的那一行就被抓到，而不是傳到三層之外才爆炸。

11.5.3 完整實作

formosa/model/attention.py — 多頭因果注意力 (本書標準版)

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from formosa.model.rope import apply_rope

class MultiHeadCausalAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int,
                 attn_dropout: float = 0.0, resid_dropout: float = 0.0,
                 bias: bool = False):
        super().__init__()
        assert d_model % n_heads == 0, \
            f"d_model={d_model} 無法被 n_heads={n_heads} 整除"
        self.n_heads = n_heads
        self.d_head = d_model // n_heads
        self.scale = 1.0 / math.sqrt(self.d_head)

        # 三個投影合併成一次矩陣乘法，比分開快
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=bias)
        self.proj = nn.Linear(d_model, d_model, bias=bias)
        self.attn_drop = nn.Dropout(attn_dropout)
        self.resid_drop = nn.Dropout(resid_dropout)

    def forward(self, x, cos=None, sin=None, return_weights=False):
        B, T, C = x.shape
        H, dh = self.n_heads, self.d_head

        qkv = self.qkv(x)  # (B,T,3C)
        q, k, v = qkv.chunk(3, dim=-1)  # 3 × (B,T,C)

        # 拆頭並轉置: (B,T,C) -> (B,H,T,dh)
        q = q.view(B, T, H, dh).transpose(1, 2)
        k = k.view(B, T, H, dh).transpose(1, 2)
        v = v.view(B, T, H, dh).transpose(1, 2)
        assert q.shape == (B, H, T, dh), q.shape

        # RoPE 作用在 Q 與 K 上，V 不旋轉 (見第 10.4 節)
        if cos is not None:
            q = apply_rope(q, cos, sin)
            k = apply_rope(k, cos, sin)

        scores = (q @ k.transpose(-2, -1)) * self.scale  # (B,H,T,T)
        mask = torch.triu(torch.ones(T, T, dtype=torch.bool,
                                     device=x.device), diagonal=1)
```



```

scores = scores.masked_fill(mask[None, None],
                             torch.finfo(scores.dtype).min)

w = F.softmax(scores, dim=-1)
w = self.attn_drop(w)
out = w @ v                                     # (B,H,T,dh)

# 合併頭：(B,H,T,dh) -> (B,T,C) contiguous 不可省
out = out.transpose(1, 2).contiguous().view(B, T, C)
out = self.resid_drop(self.proj(out))
assert out.shape == (B, T, C)
return (out, w) if return_weights else out

```

兩個效能相關的設計。第一，三個投影合併成一次 (d, 3d) 的矩陣乘法——GPU 對一次大的矩陣乘法比三次小的更有效率。第二，遮罩在每次 forward 時建立；在第 13 章的正式版本中會改為 register_buffer 預先建好，避免重複配置記憶體。

11.5.4 用 PyTorch 的融合實作

手寫版本讓你理解原理，但正式訓練時應該使用框架提供的融合核心，它會自動選用 FlashAttention 等最佳化實作。

手寫版與融合版的對照與驗證

```

def forward_fused(self, x, cos=None, sin=None):
    B, T, C = x.shape
    H, dh = self.n_heads, self.d_head
    q, k, v = self.qkv(x).chunk(3, dim=-1)
    q = q.view(B, T, H, dh).transpose(1, 2)
    k = k.view(B, T, H, dh).transpose(1, 2)
    v = v.view(B, T, H, dh).transpose(1, 2)
    if cos is not None:
        q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)

    # is_causal=True 會自動套用因果遮罩，且不 materialize T×T 矩陣
    out = F.scaled_dot_product_attention(
        q, k, v, is_causal=True,
        dropout_p=self.attn_drop.p if self.training else 0.0)

    out = out.transpose(1, 2).contiguous().view(B, T, C)
    return self.resid_drop(self.proj(out))

def test_manual_matches_fused():
    """手寫版與融合版必須數值一致（關掉 dropout）"""
    torch.manual_seed(0)
    m = MultiHeadCausalAttention(64, 4, 0.0, 0.0).eval()
    x = torch.randn(2, 16, 64)
    with torch.no_grad():
        a = m(x)
        b = m.forward_fused(x)
    assert torch.allclose(a, b, atol=1e-5), (a - b).abs().max()

```

這個對照測試是程式 11A 的重點之一。它同時驗證了兩件事：你的手寫實作是對的，而且你知道正式

版本在做什麼。第 13 章訓練 MiniFormosaGPT 時會使用融合版本，因為它在長序列上快得多、也省記憶體。

11.6 注意力變體：MHA、MQA 與 GQA

11.6.1 推論時的記憶體瓶頸

訓練時，注意力的瓶頸是 $T \times T$ 的分數矩陣。但推論時（第 14 章會完整處理）情況不同：為了避免重複計算，我們會快取每一層、每一個位置的 K 與 V，這就是 KV cache。

$$KV\ cache\ 大小 = 2 \times B \times T \times L \times H_{kv} \times d_h \times \text{每個數的位元組}$$

這個數字會隨著生成長度線性增長，而且在長對話中可能超過模型權重本身。減少它的方法是讓多個 Query 頭共用同一組 Key 與 Value 頭。

11.6.2 三種配置

配置	Query 頭數	K/V 頭數	KV cache	品質
MHA (標準)	H	H	最大	最佳
GQA (分組)	H	H / g	減為 1/g	接近 MHA
MQA (單一)	H	1	減為 1/H	略有損失

用具體數字感受一下。假設 $B = 1$ 、 $T = 4096$ 、 $L = 32$ 層、 $H = 32$ 頭、 $d_h = 128$ ，用 BF16 儲存：

配置	K/V 頭數	KV cache 大小	相對 MHA
MHA	32	2.147 GB	1
GQA-8	8	0.537 GB	1/4
GQA-4	4	0.268 GB	1/8
MQA	1	0.067 GB	1/32

從 2.15 GB 降到 0.54 GB，這在單卡部署時是「跑得動」與「跑不動」的差別。這也是為什麼現代的開放權重模型幾乎都採用 GQA——它在品質幾乎無損的前提下，把推論的記憶體壓力降低數倍。

11.6.3 GQA 的實作

GQA : Query 分組共用 K/V

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads, bias=False):
        super().__init__()
        assert n_heads % n_kv_heads == 0, \
            f"n_heads={n_heads} 必須是 n_kv_heads={n_kv_heads} 的倍數"
        self.n_heads, self.n_kv = n_heads, n_kv_heads
        self.n_rep = n_heads // n_kv_heads # 每組共用幾個 Q 頭
        self.d_head = d_model // n_heads
        self.scale = self.d_head ** -0.5

        self.w_q = nn.Linear(d_model, n_heads * self.d_head, bias=bias)
        self.w_k = nn.Linear(d_model, n_kv_heads * self.d_head, bias=bias)
        self.w_v = nn.Linear(d_model, n_kv_heads * self.d_head, bias=bias)
        self.proj = nn.Linear(d_model, d_model, bias=bias)

    @staticmethod
    def _repeat_kv(x, n_rep):
        """(B, n_kv, T, dh) -> (B, n_kv*n_rep, T, dh)"""
        if n_rep == 1:
            return x
        B, n_kv, T, dh = x.shape
        return (x[:, :, :, None].expand(B, n_kv, n_rep, T, dh) # (B,n_kv,1,T,dh)
                .reshape(B, n_kv * n_rep, T, dh)) # 不複製記憶體

    def forward(self, x, cos=None, sin=None):
        B, T, C = x.shape
        dh = self.d_head
        q = self.w_q(x).view(B, T, self.n_heads, dh).transpose(1, 2)
        k = self.w_k(x).view(B, T, self.n_kv, dh).transpose(1, 2)
        v = self.w_v(x).view(B, T, self.n_kv, dh).transpose(1, 2)
        if cos is not None:
            q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)

        k = self._repeat_kv(k, self.n_rep) # 擴展到 n_heads
        v = self._repeat_kv(v, self.n_rep)
        assert k.shape == q.shape, (k.shape, q.shape)

        out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.proj(out)
```

`_repeat_kv` 用的是 `expand` 而非 `repeat`，這一點很重要：`expand` 不會實際複製記憶體，只是建立一個共享底層資料的視圖。KV cache 節省的關鍵在於「儲存時只存 `n_kv` 份」，計算時才展開——如果你在快取階段就展開了，就完全沒有省到。

11.7 複雜度、記憶體牆與 FlashAttention

11.7.1 複雜度分析

用第 3.2.5 節的 FLOPs 公式，把一層注意力的成本拆開。設 $B = 1$ 、序列長度 T 、模型維度 d 。

項目	FLOPs	隨 T 的成長	隨 d 的成長
Q/K/V/O 四個投影	$8 T d^2$	線性	平方
$Q K^T$	$2 T^2 d$	平方	線性
權重 $\times V$	$2 T^2 d$	平方	線性
前饋網路 ($d_{ff} = 4d$)	$16 T d^2$	線性	平方

把它們的占比隨 T 變化畫出來 ($d = 4096$) :

序列長度 T	注意力占比	投影占比	前饋占比
512	2.0%	32.7%	65.3%
4,096	14.3%	28.6%	57.1%
32,768	57.1%	14.3%	28.6%

這張表推翻了一個常見的直覺。在一般的序列長度 (512 到 4096) 下，注意力並不是計算的大宗——前饋網路才是。注意力要到 T 與 d 同量級時才成為主導項。

但這只是計算量。記憶體的故事完全不同。

11.7.2 記憶體才是真正的牆

注意力分數矩陣的形狀是 (B, H, T, T)。以 $B = 8$ 、 $H = 32$ 、BF16 計算：

序列長度 T	分數矩陣大小	相對 T=512	單卡能放下嗎
512	0.13 GB	1	可以
1,024	0.54 GB	4	可以
4,096	8.59 GB	64	勉強
16,384	137.44 GB	1,024	不可能

而且這還只是一層、只是前向傳播。反向傳播需要保留這些中間值，多層又要乘上層數。這就是所謂

的記憶體牆——在長序列時，你會先撞上記憶體上限，而不是算力上限。

11.7.3 FlashAttention 的核心想法

FlashAttention 的洞見是：我們其實不需要把完整的 $T \times T$ 矩陣寫進記憶體。

觀察最終的輸出：每一列的結果是「該列的 softmax 權重與 V 的加權和」。如果能一列一列（或一塊一塊）地算，算完就丟掉中間結果，就不必儲存整個矩陣。

困難在於 softmax 需要整列的總和才能正規化。FlashAttention 用的技巧是線上 softmax：一邊掃描一邊維護「目前的最大值」與「目前的總和」，遇到更大的值就把先前的結果重新縮放。這與第 3.3.4 節的 log-sum-exp 是同一個數學基礎。

面向	標準實作	FlashAttention	結果
$T \times T$ 矩陣	寫進高頻寬記憶體	只在晶片內的快取中分塊處理	記憶體從 $O(T^2)$ 降到 $O(T)$
記憶體讀寫	多次來回搬運	一次讀入、多次重用	大幅減少頻寬瓶頸
計算量	基準	略增（重算部分中間值）	用計算換記憶體
數值結果	基準	數學上等價	差異僅在浮點誤差

第三列值得注意：FlashAttention 的 FLOPs 其實略高於標準實作，因為反向傳播時要重新計算部分中間值。但它仍然快得多——因為現代 GPU 的瓶頸是記憶體頻寬而非算力（第 18 章會完整展開）。這是一個「多算一點、少搬一點，反而更快」的經典案例。

你需要自己實作 FlashAttention 嗎

不需要。它的價值在於理解「為什麼記憶體比算力重要」，而不是自己寫核心程式。實務上呼叫 `F.scaled_dot_product_attention` 就會自動使用它。第 18 章會教你用 profiler 驗證它確實被啟用了，以及量測它帶來的實際差異。

11.7.4 注意力的其他變體：一張地圖

除了 GQA 與 FlashAttention，還有一批試圖突破 T^2 限制的方法。你不需要現在就實作它們，但應該知道它們存在、以及各自在解決什麼問題——第 20 章會完整處理。

類別	做法	省下什麼	代價
滑動視窗	每個位置只看鄰近 W 個位置	複雜度降為 $O(T \cdot W)$	無法直接看到遠處
稀疏注意力	依固定或學習的模式只算部分位置	同上	模式設計困難
線性注意力	改變運算順序避開 $T \times T$ 矩陣	複雜度降為 $O(T)$	表達能力較弱
狀態空間模型	用遞迴狀態取代注意力	$O(T)$ 且推論記憶體固定	長距檢索能力較弱
混合架構	部分層用全域注意力、部分用局部	折衷	設計與調校複雜

一個值得注意的模式：這些方法多半是在「省記憶體」與「保留全域檢索能力」之間取捨。全域注意力最貴，但它保證任何兩個位置都能直接連結；而那正是第 11.1.1 節說的 $O(1)$ 路徑長度——大模型能力的根源之一。

所以到目前為止，主流的做法不是取代全域注意力，而是讓它變便宜（FlashAttention 省記憶體、GQA 省快取），或是在部分層混用局部注意力。這是一個「先把現有的做到極致，再考慮換掉」的工程判斷。

程式實作

本章三個實作是全書最核心的程式碼。11A 產出的模組會被第 12、13 章直接 import 並使用到第 40 章。

程式 11A 四個版本的注意力

目標：逐步實作四個版本，每一版都附形狀斷言與單元測試。這是本書「先手寫再用函式庫」原則的第三次實踐（前兩次是第 3 章的 softmax 與第 7 章的 BPE）。

版本	內容	必須通過的測試	節次
v1	無參數簡化注意力	迴圈版與矩陣版結果一致	11.2
v2	縮放點積、單頭	權重每列總和為 1；縮放前後的熵差異	11.3
v3	加上因果遮罩	五個因果測試（含未來不影響過去）	11.4
v4	多頭 + RoPE + 輸出投影	形狀鏈正確；與融合版數值一致	11.5

tests/test_attention.py (節錄)

```
import math, torch, pytest
import torch.nn.functional as F
from formosa.model.attention import (simple_attention,
                                     SingleHeadAttention,
                                     MultiHeadCausalAttention)

def test_v1_loop_matches_matrix():
    x = torch.randn(6, 16)
    assert torch.allclose(simple_attention(x),
                          simple_attention_loop(x), atol=1e-5)

def test_v1_self_dominates():
    """驗證 11.2.2 的說法：無參數版本的注意力被自己主導"""
    x = torch.randn(8, 32)
    w = F.softmax(x @ x.T, dim=-1)
    diag = w.diagonal()
    assert (diag == w.max(dim=-1).values).float().mean() > 0.7

@pytest.mark.parametrize("d_head", [16, 64, 256])
def test_scaling_controls_entropy(d_head):
    """縮放後的注意力熵必須顯著高於未縮放"""
    torch.manual_seed(0)
```

```

q = torch.randn(1, 8, d_head); k = torch.randn(1, 8, d_head)
raw = q @ k.transpose(-2, -1)

def entropy(s):
    p = F.softmax(s, dim=-1)
    return float(-(p * (p + 1e-12)).log()).sum(-1).mean())

assert entropy(raw / math.sqrt(d_head)) > entropy(raw)

def test_v4_shape_chain():
    B, T, C, H = 2, 16, 64, 4
    m = MultiHeadCausalAttention(C, H)
    out, w = m(torch.randn(B, T, C), return_weights=True)
    assert out.shape == (B, T, C)
    assert w.shape == (B, H, T, T)

def test_v4_heads_are_independent():
    """不同頭的注意力分布應該不同（否則多頭沒有意義）"""
    torch.manual_seed(0)
    m = MultiHeadCausalAttention(64, 4).eval()
    with torch.no_grad():
        _, w = m(torch.randn(1, 12, 64), return_weights=True)
    diffs = [(w[0, i] - w[0, j]).abs().mean()
              for i in range(4) for j in range(i + 1, 4)]
    assert min(diffs) > 1e-3, "多個頭的注意力分布幾乎相同"

```

延伸挑戰：把 `d_head` 從 16 掃到 512，畫出「有無縮放」兩條熵曲線。你會看到未縮放版本的熵隨 `d_head` 增大而急速下降——這就是 11.3.2 節那個論證的完整圖像。

程式 11B 繁體中文注意力熱圖

目標：用真實的繁中句子觀察注意力分布，找出指代、標點與長距依賴的模式。這是本章唯一「看得見結果」的實作，也是最有成就感的一個。

ch11/lab11b_attention_map.py (節錄)

```

import torch
import matplotlib.pyplot as plt
from matplotlib import font_manager

# 繁中字型設定（見第 10.7.3 節）
plt.rcParams["font.sans-serif"] = ["Noto Sans CJK TC", "Microsoft JhengHei"]
plt.rcParams["axes.unicode_minus"] = False

PROBE_SENTENCES = [
    # 指代：「他」應該注意「小華」或「小明」
    "小明把課本借給小華，他明天要考試。",
    # 長距依賴：「宣布」應該注意遠處的「老師」
    "那位在雲林科技大學教機器人學的老師，昨天在課堂上宣布了新規定。",

```



```

# 標點：觀察句號與逗號吸引多少注意力
"第一，確認授權；第二，掃描個資；第三，執行去重。",
# 中英混寫：觀察跨語言的注意力
"請執行 SPC 分析並將結果更新到 CIM 系統。",
]

@torch.no_grad()
def attention_map(model, tok, text, layer=-1, head=0):
    ids = tok.encode(text)
    pieces = [tok.decode([i]) for i in ids]
    out = model(torch.tensor([ids]), output_attentions=True)
    w = out.attentions[layer][0, head] # (T, T)
    return pieces, w

def plot_map(pieces, w, title, out_path):
    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(w.numpy(), cmap="Greens", aspect="auto")
    ax.set_xticks(range(len(pieces)))
    ax.set_yticks(range(len(pieces)))
    ax.set_xticklabels(pieces, rotation=90, fontsize=8)
    ax.set_yticklabels(pieces, fontsize=8)
    ax.set_xlabel("被注意的位置 (Key)")
    ax.set_ylabel("發出注意的位置 (Query)")
    ax.set_title(title)
    fig.colorbar(im); fig.tight_layout(); fig.savefig(out_path, dpi=150)

def summarize(pieces, w):
    """量化描述：每個位置最注意誰"""
    rows = []
    for i, p in enumerate(pieces):
        j = int(w[i, :i + 1].argmax()) # 因果：只看自己與過去
        rows.append({"pos": i, "token": p,
                    "attends_to": pieces[j], "weight": round(float(w[i, j]), 3),
                    "is_self": i == j})
    return rows

```

必須回答的五個問題（寫進報告）：

- 「他」這個詞元最注意哪個位置？不同的層與不同的頭有差異嗎？
- 標點符號（逗號、句號）吸引了多少注意力？有沒有某些頭專門注意標點？
- 第 0 個位置（或 BOS）的注意力權重有什麼特別之處？（提示：許多模型會把它當成「無處可去時的預設目標」）
- 中英混寫句子中，中文詞元會注意英文詞元嗎？跨語言的注意力比同語言弱嗎？
- 比較淺層與深層的注意力模式。哪一層比較像語法關係、哪一層比較像語意關係？

一個常見的觀察

你很可能會發現大量的注意力集中在第一個詞元上，而且它看起來毫無意義。這不是 bug——這個現象被稱

為注意力沉沒 (attention sink) , 一般認為是模型在「沒有特別想看的東西」時的預設去處, 因為 softmax 強制每一列的總和必須是 1, 總得放到某個地方。把這個發現寫進報告, 比假裝沒看到誠實得多。

程式 11C 複雜度實測

目標：量測序列長度對時間與記憶體的实际影響，驗證 11.7 節的理論分析，並比較 MHA 與 GQA 的差異。

ch11/lab11c_complexity.py (節錄)

```
import time, json, torch

def bench_attention(module, B, T, C, device, iters=20, warmup=5):
    x = torch.randn(B, T, C, device=device)
    module = module.to(device)

    for _ in range(warmup):
        module(x)
    if device.type == "cuda":
        torch.cuda.synchronize()
        torch.cuda.reset_peak_memory_stats()

    t0 = time.perf_counter()
    for _ in range(iters):
        out = module(x)
    if device.type == "cuda":
        torch.cuda.synchronize()
    dt = (time.perf_counter() - t0) / iters

    peak = (torch.cuda.max_memory_allocated() / 1e9
            if device.type == "cuda" else float("nan"))
    return {"T": T, "seconds": round(dt, 5), "peak_GB": round(peak, 3)}

def kv_cache_size(B, T, L, n_kv_heads, d_head, bytes_per=2):
    """理論值，與 11.6.2 節的表格對照"""
    return 2 * B * T * L * n_kv_heads * d_head * bytes_per
```

必須產出的三張圖與一份分析：

- 時間對 T 的曲線 (對數座標)。斜率應該接近 2 (平方成長)，但在小 T 時會偏離——為什麼？
- 峰值記憶體對 T 的曲線。比較手寫版與 `F.scaled_dot_product_attention` 的差異，這就是 FlashAttention 的实际效果。
- MHA 與 GQA 在不同 `n_kv_heads` 下的 KV cache 理論大小，並與 11.6.2 節的表格核對。
- 一段分析：在你的硬體上，序列長度到多少會先撞上記憶體上限？這個數字對你的專題有什麼意涵？

系統案例：可互動的注意力教學元件

本章的系統案例是製作一個網頁元件，讓使用者輸入一個繁中句子、選擇層與頭，即時看到注意力權重的變化。這個工具可以用在高中的 AI 社團教學，也可以放進你的專題展示。

功能規格

功能	說明	教學價值	難度
句子輸入	使用者輸入繁中句子，即時分詞顯示	讓學生看到 tokenizer 怎麼切	低
熱圖	T×T 的注意力矩陣，可切換層與頭	核心視覺化	中
連線圖	選定一個詞元，畫出它注意的對象	比熱圖更直觀	中
因果遮罩開關	可切換有無遮罩，對照差異	示範「偷看答案」的後果	低
縮放開關	可切換有無除以 $\sqrt{d_h}$	示範 softmax 飽和	低
數值面板	顯示熵、最大權重、注意力沉沒比例	把觀察變成數字	中

第四與第五個開關是這個工具最有教學價值的部分。學生可以親手把因果遮罩關掉，看到注意力矩陣從三角形變成正方形；也可以把縮放拿掉，看到熱圖從柔和的漸層變成幾條銳利的亮線。這兩個動作比任何解釋都有效。

交付要求

- 可運行的元件（網頁、Notebook widget 或桌面程式皆可），支援至少 32 個詞元的句子。
- 至少五個預設範例句，涵蓋指代、長距依賴、標點、中英混寫與條列結構。
- 兩個對照開關（因果遮罩、縮放）確實可切換並即時更新。
- 數值面板，至少顯示注意力熵與第一個詞元的平均權重。
- 一份五分鐘的教學腳本：如何用這個工具向一位沒學過的高中生解釋注意力。
- 在真實課堂或至少三位同學身上試用過，並記錄他們的問題與你的修改。

最後一項是這個案例真正的評分重點。一個做得很炫但沒人看得懂的工具沒有價值；一個樸素但能讓

人恍然大悟的工具才是好工具。請把試用回饋誠實寫進報告，包括那些讓你發現自己解釋不清楚的地方。

章末評量

一、概念題

1. 從路徑長度與平行度兩個角度，說明注意力相對於循環與卷積的優勢與代價。
2. 為什麼 Key 與 Value 要分開？如果用同一組向量會失去什麼表達能力？
3. 無參數的簡化注意力有哪三個限制？各由哪個設計解決？
4. 推導 $\text{Var}(q \cdot k) \approx d_h$ ，並說明為什麼是除以 $\sqrt{d_h}$ 而非 d_h 。
5. 因果遮罩寫錯為什麼特別危險？它會表現出什麼徵兆？用什麼測試可以抓到？
6. 為什麼遮罩要用 dtype 的最小值而不是 $-\text{inf}$ ？什麼情況下會出問題？
7. 多頭注意力為什麼「不增加參數卻增加表達能力」？它的代價是什麼？
8. 說明 MHA、GQA 與 MQA 的差異，以及 GQA 為何成為現代模型的主流選擇。
9. FlashAttention 為什麼「多算一點反而更快」？這反映了現代 GPU 的什麼特性？

二、計算題

1. 一個 $d_{\text{model}} = 1024$ 、 $n_{\text{heads}} = 16$ 的注意力層。計算 d_{head} 、Q/K/V/O 四個投影的參數量合計，以及若使用 GQA-4 時 K 與 V 投影的參數量。
2. $B = 4$ 、 $H = 16$ 、 $T = 2048$ ，用 BF16 儲存注意力分數矩陣需要多少 GB？若 T 加倍呢？
3. $B = 1$ 、 $T = 8192$ 、 $L = 40$ 層、 $H = 40$ 、 $d_{\text{head}} = 128$ ，BF16。計算 MHA 的 KV cache 大小；改用 GQA-8 後是多少？
4. 設 $d = 4096$ 、 $d_{\text{ff}} = 16384$ 。計算 $T = 1024$ 與 $T = 65536$ 時，注意力 (QK^T 與 AV) 占單層總 FLOPs 的百分比。
5. 某模型的注意力熵為 0.5，序列長度為 64。與均勻分布 (熵為 $\ln 64$) 相比，這代表模型平均在幾個位置之間分配注意力？(提示： $\exp(\text{熵})$ 是有效候選數)

三、除錯題

1. 同學的模型訓練損失在 200 步內就降到 0.02，生成的文字卻完全是亂碼。請說出最可能的原因與檢查方式。
2. 同學的多頭注意力在 forward 時拋出「view size is not compatible with input tensor size and stride」的錯誤。請指出缺了哪一行。
3. 同學實作的注意力在批次中有 padding 時，同一句話單獨跑與湊批次跑結果不同。請列出三個可能原因。

4. 同學的注意力熱圖顯示所有頭的分布幾乎一模一樣。請列出三個可能原因與檢查順序。
5. 同學把 dropout 加在 softmax 之前而非之後。請說明這會造成什麼行為差異。
6. 同學的 GQA 實作記憶體用量與 MHA 相同。請指出最可能的原因 (提示：與 `_repeat_kv` 的時機有關)。

四、系統設計題

1. 你要為一個「校務長文件問答」設計注意力配置。文件長度中位數 8,000 詞元，尖峰 32,000 詞元，部署在單張 24 GB 的 GPU 上。請決定 `n_heads`、`n_kv_heads` 與是否使用 FlashAttention，並用計算支撐你的每一個決定。
2. 你的團隊要在邊緣裝置 (8 GB 記憶體) 部署一個對話模型，對話輪次可能達 20 輪。請分析 KV cache 的成長，提出至少三種控制記憶體的策略，並說明各自的品質代價。

五、臺灣情境與倫理題

1. 在程式 11B 中，中英混寫句子的跨語言注意力通常較弱。請說明這個現象可能的成因鏈 (從語料到 tokenizer 到注意力)，並設計一個實驗驗證你的假設。
2. 注意力權重常被當成「模型的解釋」呈現給使用者。請討論這種做法的問題：注意力權重能否等同於「模型依據什麼做決定」？如果不能，在對外說明時應該如何謹慎表述？

評量提示 (給自學者)

- 計算題第 1 題： $d_{\text{head}} = 64$ ；四個投影各為 1024×1024 ，合計 4,194,304 個參數。GQA-4 時 K 與 V 各為 1024×256 ，合計 524,288。
- 計算題第 2 題： $4 \times 16 \times 2048^2 \times 2 \text{ bytes} \approx 0.54 \text{ GB}$ ；T 加倍後約 2.15 GB (四倍)。
- 計算題第 3 題：MHA 為 $2 \times 1 \times 8192 \times 40 \times 40 \times 128 \times 2 \approx 6.71 \text{ GB}$ ；GQA-8 為其 1/5，約 1.34 GB。
- 計算題第 4 題：注意力 $4T^2d$ ，其他 $8Td^2 + 16Td^2 = 24Td^2$ 。T=1024 時約 4.0%；T=65536 時約 72.7%。
- 計算題第 5 題： $\exp(0.5) \approx 1.65$ ，代表注意力幾乎集中在一到兩個位置上，遠低於均勻分布的 64。
- 除錯題第 2 題：transpose 之後、view 之前缺少 `.contiguous()`。
- 除錯題第 6 題：很可能在儲存 KV cache 之前就呼叫了 `_repeat_kv`。展開必須發生在計算時，快取中只能存 `n_kv` 份。

研究延伸與版本注意事項

- Transformer 原始論文 (Vaswani 等人 2017) 的第 3.2 節是本章的直接來源。讀完本章再回去讀，你會發現那幾頁其實非常精簡。
- FlashAttention 系列論文：核心貢獻是把注意力變成 IO 感知的演算法。即使你不打算實作，理解它對第 18 章的效能分析很有幫助。
- MQA 與 GQA 的原始提案：兩篇都不長，論證清楚，是理解「品質與記憶體如何權衡」的好教材。
- 注意力可解釋性的爭論：關於「注意力權重是否等同於解釋」，學界有相當多的討論與反例，本章的倫理題與此直接相關。
- 注意力沉沒 (attention sink) 的相關研究：這個現象與長上下文的串流處理有關，第 20 章會再提到。
- 本章的數學不會過時，但 PyTorch 的 `scaled_dot_product_attention` 後端選擇邏輯與可用的最佳化核心會隨版本改變。程式碼以課程倉庫為準，並在效能量測時記錄版本。

常見問題

問：既然有 `F.scaled_dot_product_attention`，為什麼要手寫四個版本？答：三個理由。第一，你必須能讀懂與修改別人的注意力實作，而那需要知道每一步在做什麼。第二，第 12、13 章你要組裝完整模型，形狀變換的直覺是必需的。第三，當注意力出問題時 (NaN、記憶體爆炸、遮罩失效)，只有理解內部的人能診斷。

問：頭數該設多少？答：常見的做法是讓 `d_head` 落在 64 到 128 之間，再由 `d_model` 反推頭數。例如 `d_model = 768` 配 12 個頭 (`d_head = 64`)，`d_model = 4096` 配 32 個頭 (`d_head = 128`)。太小的 `d_head` 會讓每個頭的表達能力不足，太大則失去多頭的意義。

問：為什麼我的注意力熱圖看起來一片模糊，沒有明顯的模式？答：可能有三個原因。一是模型還沒訓練好 (隨機初始化的模型注意力接近均勻)；二是你看的層不對 (淺層通常較分散)；三是這個頭確實沒有明顯的功能——不是每個頭都有可解釋的角色，這是正常的。

教師使用建議

本章排在第 11 週，是全書最核心的一章，建議 2 小時講授加 1 小時實驗，並強烈建議額外安排一次補強時段。

時段	內容	互動設計	注意事項
前 20 分鐘	11.1 至 11.2 節，動機與無參數版本	現場跑 v1，印出對角線讓學生發現問題	讓學生自己發現限制
中間 35 分鐘	11.3 節，縮放點積	現場示範不縮放時 softmax 變成 one-hot	這是全章最需要慢講的一段
後 30 分鐘	11.4 至 11.5 節，遮罩與多頭	白板畫 4×4 遮罩；形狀鍵全班一起推一次	形狀鍵務必手推
最後 35 分鐘	11.6 至 11.7 節，變體與複雜度	現場算一次 KV cache 大小	GQA 的節省用數字最有說服力
實驗課 1 小時	程式 11A 的 v1 到 v3	兩人一組，互相檢查形狀斷言	v4 與 11B 作為課後作業

一個效果極佳的現場示範：準備兩份程式碼，一份有因果遮罩、一份沒有，各訓練 200 步。有遮罩的損失降到 4.5 左右，沒遮罩的降到 0.02。然後兩者各生成一段文字——沒遮罩的那個會產生完全的亂碼。學生會親眼看到「漂亮的訓練曲線」如何是一個陷阱。這個示範我每年都做，效果從未失手。

另一個建議：形狀鍵（11.5.2 節那張表）務必讓全班一起在白板上推一次，並要求每個學生在自己的筆記上寫下八個形狀。第 13 章組裝模型時，卡住的學生幾乎都是這一步沒做扎實的。

高中授課的調整：11.1、11.2 完整教（無參數版本很容易懂）；11.3 用「除以 $\sqrt{d_h}$ 讓數字不要太大，否則 softmax 會變成只選一個」的直覺說法，跳過變異數推導但務必做那個示範；11.4 用畫格子的方式教遮罩，五個測試只講第四個；11.5 只講「切成好幾份各自算」的概念，形狀變換可略；11.6、11.7 略過。程式部分做 v1 與 v3，用老師提供的 v4 跑 11B 的熱圖——熱圖對高中生的吸引力非常高。

本章小結

1. 注意力用 $O(1)$ 的路徑長度取代循環的 $O(T)$ ，代價是 T^2 的計算與記憶體。
2. Query、Key、Value 三者分工：查詢意圖、被找到的特徵、提供的內容。Key 與 Value 分開帶來額外的表達自由。
3. 無參數的簡化注意力有三個限制：無可學習參數、自己主導、Key 與 Value 綁死，分別由三個投影解決。
4. $\text{Var}(q \cdot k) \approx d_h$ ，所以要除以 $\sqrt{d_h}$ 。不縮放會讓 softmax 飽和（實測熵從 1.688 掉到 0.301），梯度趨近於零。
5. 因果遮罩寫錯會讓損失異常低卻毫無用處；最可靠的測試是「改變未來的輸入，過去的輸出必須

不變」。

6. 遮罩應填入 dtype 的最小值而非 $-\text{inf}$ ，避免整列被遮時產生 NaN。
7. 注意力 dropout 作用在 softmax 之後；大模型因資料充足常設為 0。
8. 多頭不增加參數，只是把同樣的維度切分使用；代價是注意力矩陣的記憶體乘以 H 。
9. 形狀鏈的八個步驟中，transpose 之後、view 之前的 contiguous 是最容易漏掉的一行。
10. GQA 讓多個 Query 頭共用 K/V，KV cache 減為 $1/g$ ；實測 GQA-8 把 2.15 GB 降到 0.54 GB。
11. 在一般序列長度下，前饋網路才是計算大宗；注意力要到 T 與 d 同量級時才主導。
12. 記憶體才是真正的牆： $T = 16384$ 時單層分數矩陣就要 137 GB。FlashAttention 用分塊與線上 softmax 把記憶體從 $O(T^2)$ 降到 $O(T)$ 。
13. 注意力權重是很好的除錯與教學工具，但不等同於模型的解釋，對外呈現時須謹慎表述。

成果檢核

完成本章後你必須繳交：

- 四個版本的注意力實作（程式 11A）：每一版附形狀斷言，並通過對應的測試。
- 因果性測試報告：五個測試全部通過，特別是「未來不影響過去」。
- 手寫版與融合版的數值一致性驗證：關掉 dropout 後誤差小於 $1e-5$ 。
- 注意力熱圖分析（程式 11B）：至少四個句子、五個問題的回答，含注意力沉沒的觀察。
- 複雜度實測報告（程式 11C）：時間與記憶體對 T 的曲線、MHA 與 GQA 的 KV cache 對照、以及你的硬體上的長度上限。
- 互動教學元件與試用回饋紀錄。

本章產出的 MultiHeadCausalAttention 是全書最重要的一個模組。第 12 章會把它與前饋網路、正規化、殘差連接組裝成 Transformer block；第 13 章會疊起來訓練；第 14 章會為它加上 KV cache；第 18 章會量測它的效能；第 20 章會討論它的變體。你在這一章寫的程式，會一路活到第 40 章的總整專題。

第 12 章

Transformer Block 與現代 GPT 元件

Transformer Blocks and Modern GPT Components

難度：核心 | 建議授課：第 12 週 | 硬體需求：A 級可完成全部實作；消融實驗有 GPU 者可放大規模

叫獸開講

GPT 的祕密不在於它有多複雜，而在於它把同一個積木疊了幾十層。看懂一塊積木，你就看懂了整棟樓。這一章要做的，是把第 11 章那顆心臟裝進一個能反覆堆疊、疊四十層也不會崩潰的骨架裡——而「不會崩潰」這件事，比你想像的難。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 說明 LayerNorm 與 RMSNorm 的計算差異，並解釋為什麼後者成為現代主流。
2. 比較 Pre-LN 與 Post-LN 的梯度行為，說明為什麼深層模型幾乎都用 Pre-LN。
3. 解釋殘差連接如何建立梯度捷徑，並量測殘差流的變異數隨層數的成長。
4. 實作前饋網路，比較 GELU 與 SwiGLU，並正確處理參數量對齊。
5. 正確組裝 Transformer block，說明各元件的順序與常見的組裝錯誤。
6. 精確計算模型的參數量與單次前向的 FLOPs，並驗證你的估算。
7. 說明從 GPT-2 到 Llama-style 的元件替換動機，並判斷何時該採用哪一種。
8. 設計可透過設定檔一鍵切換規模與元件的模型架構。
9. 為三級硬體規劃 tiny、small、base 三種尺寸，並附完整的資源估算。

本章與前後章的關係

第 11 章給了你注意力，本章給你其餘的一切：正規化、殘差、前饋網路，以及把它們組裝起來的順序。第 13 章則是把本章的 block 疊 N 次、接上第 10 章的嵌入與第 7 章的 tokenizer，訓練出第一個能生成繁體中文的模型。換句話說，本章結束時，MiniFormosaGPT 的所有零件就都齊了。

12.1 正規化：LayerNorm 與 RMSNorm

12.1.1 為什麼需要正規化

深層網路有一個根本問題：每一層的輸出分布會隨著訓練而漂移。第 L 層學到的東西，是建立在第 L-1 層的輸出分布上的；一旦前面的層更新了，後面的層就得重新適應。層數越深，這個問題越嚴重。

正規化的作用是把每一層的輸入拉回一個穩定的尺度，讓後續的層有一個可預期的輸入分布。它同時也改善了損失曲面的條件數，讓最佳化更容易。

沒有正規化	會發生什麼	徵兆	本書章節
分布漂移	每層輸入的尺度隨訓練改變	訓練不穩、對學習率極度敏感	第 15 章
梯度尺度失控	深層與淺層的梯度差好幾個數量級	部分層學不動、部分層發散	第 3.6.3 節
數值溢位	激勵值持續放大	出現 inf 或 nan	第 3.6.3 節

12.1.2 LayerNorm

LayerNorm 對每一個位置的特徵向量獨立做標準化——注意是「對每個位置的 d 維向量」，不是跨批次或跨位置。

$$LN(x) = \gamma \odot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta \quad \cdot \text{其中 } \mu \text{ 與 } \sigma^2 \text{ 沿最後一維計算}$$

符號	意義	形狀	說明
μ	該位置的特徵平均	純量（每個位置一個）	沿 d 維求平均
σ^2	該位置的特徵變異數	純量	沿 d 維求變異數
γ	可學習的縮放	$(d,)$	讓模型能還原尺度
β	可學習的偏移	$(d,)$	讓模型能還原位置
ϵ	數值穩定常數	純量	通常 $1e-5$ 或 $1e-6$

為什麼是沿最後一維而不是跨批次？因為語言模型的批次中每個樣本長度不同、內容差異大，跨批次統計不穩定；而且推論時批次大小可能是 1，跨批次統計根本無法計算。這與影像模型常用的 BatchNorm 有本質差異——第 3.5.8 節提過的「Transformer 通常不用 BatchNorm」，理由就在這裡。

12.1.3 RMSNorm：拿掉一半的計算

RMSNorm 的想法是：中心化（減去平均）這一步其實可有可無，真正重要的是控制尺度。拿掉它之

後，計算變簡單、速度變快，效果幾乎不變。

$$RMSNorm(x) = \gamma \odot x / \sqrt{(\text{mean}(x^2) + \varepsilon)}$$

面向	LayerNorm	RMSNorm	差異
計算	減平均、除標準差	只除均方根	少一次遍歷
參數	γ 與 β (2d 個)	只有 γ (d 個)	省一半
速度	基準	約快 10 到 30%	在深層模型上累積可觀
效果	基準	幾乎相同	多數實測無顯著差異
採用	GPT-2、BERT	Llama 系列、多數現代模型	本書採用 RMSNorm

formosa/model/norm.py

```
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, d_model: int, eps: float = 1e-6):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(d_model))
        self.eps = eps

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (... , d) 沿最後一維計算
        # 用 float32 計算統計量，避免 BF16 下的精度損失
        dtype = x.dtype
        x32 = x.float()
        rms = torch.rsqrt(x32.pow(2).mean(-1, keepdim=True) + self.eps)
        return (x32 * rms).to(dtype) * self.weight

class LayerNorm(nn.Module):
    """保留一份 LayerNorm 供對照實驗使用（程式 12C）"""
    def __init__(self, d_model: int, eps: float = 1e-5, bias: bool = True):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(d_model))
        self.bias = nn.Parameter(torch.zeros(d_model)) if bias else None
        self.eps = eps

    def forward(self, x):
        dtype = x.dtype
        x32 = x.float()
        mu = x32.mean(-1, keepdim=True)
        var = x32.var(-1, keepdim=True, unbiased=False)
        y = ((x32 - mu) * torch.rsqrt(var + self.eps)).to(dtype)
        y = y * self.weight
        return y + self.bias if self.bias is not None else y
```

注意兩個實作中都把統計量的計算轉成 float32。這是第 3.6.5 節混合精度原則的具體應用：BF16 的尾數只有 7 位，計算平方和時精度不足，可能導致正規化失準。轉成 float32 算完再轉回來，成本很小但能避免一類難以診斷的訓練不穩。

一個容易忽略的細節

RMSNorm 的 weight 初始化為全 1，而不是常態分布。這是因為它的角色是「調整每個維度的縮放」，起點應該是「不做任何調整」。同理，LayerNorm 的 bias 初始化為全 0。這類參數在第 10.2.4 節的參數分組中應排除 weight decay——把它們拉向零會破壞正規化的作用。

12.2 Pre-LN 與 Post-LN：一個順序決定成敗

12.2.1 兩種放法

正規化該放在子層之前還是之後？這個看似瑣碎的選擇，決定了模型能不能疊到幾十層。

配置	運算順序	殘差路徑	代表
Post-LN	$x \rightarrow \text{子層} \rightarrow \text{加殘差} \rightarrow \text{正規化}$	每層都被正規化打斷	原始 Transformer、BERT
Pre-LN	$x \rightarrow \text{正規化} \rightarrow \text{子層} \rightarrow \text{加殘差}$	殘差路徑完全乾淨	GPT-2 之後、Llama 系列

$$\text{Post-LN: } x \leftarrow \text{Norm}(x + \text{Sublayer}(x))$$

$$\text{Pre-LN: } x \leftarrow x + \text{Sublayer}(\text{Norm}(x))$$

12.2.2 為什麼 Pre-LN 更穩定

關鍵在殘差路徑上有沒有東西擋著。

在 Pre-LN 中，從輸入到輸出有一條完全沒有經過任何非線性或正規化的加法路徑。梯度可以沿著它幾乎無損地一路傳到最淺層——這正是第 3.4.2 節說的「在連乘中加入一條乘以 1 的捷徑」，也與第 2.4.4 節 LSTM 的細胞狀態是同一個想法。

在 Post-LN 中，每一層的殘差加總之後都要再過一次正規化。正規化的雅可比矩陣不是單位矩陣，梯度每經過一層就被改變一次尺度。疊個十幾層之後，淺層的梯度就會嚴重失真。

面向	Post-LN	Pre-LN	後果
梯度捷徑	被正規化打斷	完全乾淨	Pre-LN 可疊更深
需要 warmup	幾乎必需	較不敏感	Post-LN 沒 warmup 常直接發散
最終效果	調得好時略佳	略遜但穩定	穩定性通常更重要
殘差流尺度	每層被壓回	隨層數持續成長	Pre-LN 需最終正規化

第三列值得誠實說明：在小模型、調校得當的情況下，Post-LN 的最終效果有時略勝一籌。但它對超參數極度敏感，而且層數一多就容易發散。現代模型選擇 Pre-LN，是拿一點點效果換取「幾乎一定訓練得起來」——在動輒數十萬元一次的訓練成本下，這個交換非常划算。

12.2.3 Pre-LN 的代價：殘差流會膨脹

上表最後一列指出了 Pre-LN 的副作用。因為每一層都往殘差流上加東西，而中間沒有任何壓縮機制，殘差流的變異數會隨層數持續累積。

用一個簡化模擬說明：假設每層往殘差流加上一個變異數固定的分支輸出，那麼殘差流的總變異數大致隨層數線性成長。實測 24 層之後，變異數約成長為初始的 16 倍。

層數	殘差流變異數（模擬）	相對第 0 層	意涵
0	0.0039	1.0	起點
6	0.0190	4.9	穩定成長
12	0.0340	8.7	—
24	0.0639	16.3	需要最終正規化

這個成長帶來兩個實務要求。第一，Pre-LN 架構在最後一層 block 之後必須再做一次正規化（final norm），否則輸出投影會接到一個尺度失控的向量。第二，殘差分支的初始化應該隨層數縮小——這就是第 3.5.7 節提過的 $\sqrt{(2L)}$ 縮放。

殘差分支的輸出投影初始化： $std = 0.02 / \sqrt{(2L)}$

直覺是：層數越多，每一層對殘差流的貢獻就應該越小，這樣總和才不會失控。GPT-2 的論文中就有

這個設計，而它是一個「不做也能跑、但做了明顯更穩」的細節。

12.3 前饋網路：GELU 與 SwiGLU

12.3.1 前饋網路在做什麼

注意力負責「在位置之間搬運資訊」，前饋網路負責「對每個位置各自做非線性變換」。兩者的分工很清楚：一個橫向、一個縱向。

$$FFN(x) = W_2 \cdot \text{act}(W_1 x) \quad , \text{其中 } W_1: (d, d_{ff})、W_2: (d_{ff}, d)$$

注意 FFN 是逐位置獨立套用的——同一組權重作用在序列的每一個位置上，位置之間沒有互動。這也是為什麼它可以寫成一次大的矩陣乘法，效率很高。

d_{ff} 通常取 $4d$ 。這個「先放大再縮小」的結構有時被解釋為一種鍵值記憶： W_1 的每一列像是一個「模式偵測器」， W_2 的每一列像是對應的「回應內容」。第 28 章談模型編輯時，這個觀點會變得重要。

12.3.2 激勵函數的演進

函數	定義	特性	採用
ReLU	$\max(0, x)$	簡單快速，但在 0 處不可微	早期模型
GELU	$x \cdot \Phi(x)$ ， Φ 為標準常態的累積分布	平滑，負值區有小幅通過	GPT-2、BERT
SiLU / Swish	$x \cdot \text{sigmoid}(x)$	與 GELU 相似，計算較簡單	SwiGLU 的基礎
SwiGLU	$(\text{SiLU}(W_1 x) \odot W_3 x) W_2$	門控結構，需三個矩陣	Llama 系列

SwiGLU 的關鍵是門控 (gating)：一路做激勵、另一路當作閘門，兩者逐元素相乘。這讓網路能夠「有條件地」讓資訊通過，表達能力比單純的激勵函數強。

12.3.3 SwiGLU 的參數量對齊

這是實作 SwiGLU 時最容易出錯的地方。標準 FFN 有兩個矩陣，SwiGLU 有三個——如果 d_{ff} 不變，參數量會多出 50%。

為了公平比較，慣例是把 SwiGLU 的中間維度縮小到原本的三分之二：

$$d_ff(\text{SwiGLU}) = (2/3) \times d_ff(\text{標準}) = (8/3) \times d$$

驗算一下 ($d = 768$ 、標準 $d_ff = 3072$) :

配置	d_ff	矩陣數	參數量
GELU 標準	3,072	2	4,718,592
SwiGLU 對齊後	2,048	3	4,718,592

兩者完全相同。這樣比較才有意義——否則你不知道效果的提升來自結構還是來自多出來的參數 (第 5.5.2 節的消融原則) 。

實務上 d_ff 還會被向上取整到某個倍數 (例如 128 或 256 的倍數)，因為對齊的維度在 GPU 上運算效率較高。這是一個「數學上無所謂、硬體上有差」的細節。

formosa/model/ffn.py

```
import torch.nn as nn
import torch.nn.functional as F

class FeedForward(nn.Module):
    """標準前饋網路，可切換 GELU 或 SwiGLU"""
    def __init__(self, d_model: int, d_ff: int | None = None,
                 activation: str = "swiglu", dropout: float = 0.0,
                 bias: bool = False, multiple_of: int = 128):
        super().__init__()
        self.activation = activation

        if d_ff is None:
            if activation == "swiglu":
                #  $8/3 \times d$ ，再向上取整到 multiple_of 的倍數
                d_ff = int(8 * d_model / 3)
                d_ff = multiple_of * ((d_ff + multiple_of - 1) // multiple_of)
            else:
                d_ff = 4 * d_model
            self.d_ff = d_ff

        if activation == "swiglu":
            self.w1 = nn.Linear(d_model, d_ff, bias=bias) # 激勵路徑
            self.w3 = nn.Linear(d_model, d_ff, bias=bias) # 閘門路徑
            self.w2 = nn.Linear(d_ff, d_model, bias=bias) # 輸出
        else:
            self.w1 = nn.Linear(d_model, d_ff, bias=bias)
            self.w2 = nn.Linear(d_ff, d_model, bias=bias)
            self.w3 = None
        self.drop = nn.Dropout(dropout)

    def forward(self, x):
        if self.activation == "swiglu":
            h = F.silu(self.w1(x)) * self.w3(x) # 門控
```



```

    else:
        h = F.gelu(self.w1(x), approximate="tanh")
        return self.drop(self.w2(h))

def n_params(self) -> int:
    n = 3 if self.activation == "swiglu" else 2
    return n * self.w1.in_features * self.d_ff

```

GELU 的 `approximate="tanh"` 是一個歷史遺留：GPT-2 使用的是 `tanh` 近似版本而非精確版本，兩者數值略有差異。如果你要載入 GPT-2 的權重（第 22 章可能會做），必須用同一個版本，否則會有微小但持續的偏差。

12.4 組裝：Transformer Block

12.4.1 完整的資料流

把前三節與第 11 章的注意力組裝起來。一個 Pre-LN 的 Transformer block 只有六行。

$$x \leftarrow x + \text{Attention}(\text{Norm}_1(x))$$

$$x \leftarrow x + \text{FFN}(\text{Norm}_2(x))$$

步驟	運算	形狀	注意
1	保存殘差 <code>residual = x</code>	(B, T, d)	不可省
2	<code>Norm₁(x)</code>	(B, T, d)	形狀不變
3	<code>Attention</code>	(B, T, d)	第 11 章的模組
4	<code>x = residual + 上一步</code>	(B, T, d)	第一個殘差
5	<code>residual = x ; Norm₂(x)</code>	(B, T, d)	重新保存
6	<code>FFN</code>	(B, T, d)	逐位置
7	<code>x = residual + 上一步</code>	(B, T, d)	第二個殘差

注意輸入與輸出的形狀完全相同，都是 (B, T, d)。這個性質是 Transformer 能夠任意堆疊的根本原因——每一塊積木的介面都一樣，所以想疊幾層就疊幾層。

12.4.2 實作

| [formosa/model/block.py](#)

```

import torch.nn as nn
from formosa.model.norm import RMSNorm, LayerNorm
from formosa.model.attention import MultiHeadCausalAttention
from formosa.model.ffn import FeedForward

NORMS = {"rmsnorm": RMSNorm, "layernorm": LayerNorm}

class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        Norm = NORMS[cfg.norm_type]
        self.norm1 = Norm(cfg.d_model)
        self.attn = MultiHeadCausalAttention(
            cfg.d_model, cfg.n_heads,
            attn_dropout=cfg.attn_dropout,
            resid_dropout=cfg.resid_dropout,
            bias=cfg.bias)
        self.norm2 = Norm(cfg.d_model)
        self.ffn = FeedForward(
            cfg.d_model, cfg.d_ff,
            activation=cfg.activation,
            dropout=cfg.resid_dropout,
            bias=cfg.bias)
        self.pre_norm = cfg.pre_norm

    def forward(self, x, cos=None, sin=None):
        if self.pre_norm:
            x = x + self.attn(self.norm1(x), cos, sin)
            x = x + self.ffn(self.norm2(x))
        else:
            # Post-LN, 供對照實驗
            x = self.norm1(x + self.attn(x, cos, sin))
            x = self.norm2(x + self.ffn(x))
        return x

```

12.4.3 六個常見的組裝錯誤

錯誤	症狀	為什麼危險	如何抓到
忘記殘差	深層時 loss 不降	梯度捷徑消失	參數量正確但訓練不動
殘差加錯對象	加的是正規化後的 x	殘差路徑不再乾淨	對照 12.4.1 的資料流
兩層共用同一個 Norm	效果變差但仍能訓練	兩個子層的統計需求不同	檢查 named_parameters 數量
Pre/Post 混用	訓練不穩	梯度行為不一致	單元測試檢查順序
忘記 final norm	輸出尺度失控	Pre-LN 的殘差流會膨脹	印出最後一層輸出的變異數
殘差分支未做 $\sqrt{2L}$ 縮放	深層時初期不穩	殘差流成長過快	比較不同層數的初期 loss

第三列特別隱蔽：如果你寫成 `self.norm = Norm(d)` 然後在兩處都用它，程式會正常運作、參數量看起來也對（只是少了 `d` 個），但兩個子層被迫共用同一組縮放參數。這種錯誤只會讓效果略差，很難被發現——除非你養成檢查參數清單的習慣。

組裝正確性的測試

```
import torch
from formosa.model.block import TransformerBlock

def test_block_shape(cfg):
    b = TransformerBlock(cfg)
    x = torch.randn(2, 16, cfg.d_model)
    assert b(x).shape == x.shape # 進出形狀必須相同

def test_two_separate_norms(cfg):
    """norm1 與 norm2 必須是不同物件"""
    b = TransformerBlock(cfg)
    assert b.norm1 is not b.norm2
    assert b.norm1.weight is not b.norm2.weight

def test_residual_present(cfg):
    """把子層權重歸零後，block 應退化為恆等映射"""
    b = TransformerBlock(cfg).eval()
    with torch.no_grad():
        for p in b.attn.parameters(): p.zero_()
        for p in b.ffn.parameters(): p.zero_()
    x = torch.randn(2, 8, cfg.d_model)
    with torch.no_grad():
        y = b(x)
    assert torch.allclose(y, x, atol=1e-5), "殘差連接缺失或接錯"

def test_causality_preserved(cfg):
    """block 層級也要驗證因果性（延續第 11 章）"""
    b = TransformerBlock(cfg).eval()
    x = torch.randn(1, 12, cfg.d_model)
    x2 = x.clone(); x2[0, -1] = torch.randn(cfg.d_model)
    with torch.no_grad():
        assert torch.allclose(b(x)[0, :-1], b(x2)[0, :-1], atol=1e-5)
```

`test_residual_present` 是一個很聰明的測試：把所有子層的權重歸零，那麼 Attention 與 FFN 的輸出都是零，如果殘差連接正確，block 就應該原封不動地回傳輸入。任何殘差寫錯的情況都會讓這個測試失敗。這種「用退化情況驗證結構」的技巧值得學起來。

12.5 參數量與 FLOPs 的精確計算

12.5.1 逐項拆解

第 3 章給過粗估公式，這裡做精確版本。能算準參數量是一項基本功——它決定你的模型放不放得下、要多少記憶體、訓練要多久。

元件	參數量	d=768 時	備註
嵌入層	$V \times d$	24.58 M	$V = 32,000$
每層注意力 (QKVO)	$4 d^2$	2.36 M	不含 bias
每層 FFN (GELU)	$2 d \times d_{ff}$	4.72 M	$d_{ff} = 4d$
每層 FFN (SwiGLU)	$3 d \times d_{ff}$	4.72 M	$d_{ff} = 8d/3$ ，對齊後相同
每層正規化 (RMS)	$2 d$	0.0015 M	兩個 norm，可忽略
最終正規化	d	0.0008 M	可忽略
輸出投影	0 (綁定) 或 $V \times d$	0 或 24.58 M	第 10.5 節

$$\text{總參數} \approx V \cdot d + L \cdot (4d^2 + 2 \cdot d \cdot d_{ff}) + \text{綁定時 } 0 \text{ 否則 } V \cdot d$$

12.5.2 三種尺寸的實算

尺寸	設定	每層參數	所有層	總計 (含嵌入)
tiny	d=128, L=4, d_ff=512	0.197 M	0.79 M	4.88 M
small	d=512, L=8, d_ff=2048	3.146 M	25.17 M	41.55 M
base	d=768, L=12, d_ff=3072	7.078 M	84.93 M	109.51 M

($V = 32,000$ 、權重綁定、忽略正規化參數。這三種尺寸對應第 4.4.3 節的 A、B、C 三級硬體。)

注意一個現象：tiny 版本的總參數中，嵌入層占了 4.10 M，超過八成。這就是第 7 章反覆強調的——小模型的詞彙表成本極高。如果你要做真正的小模型（第 26 章的邊緣部署），詞彙表大小是第一

個要重新檢視的參數。

12.5.3 FLOPs 計算

用第 3.2.5 節的公式（一次矩陣乘法約 $2mkn$ ）· 單層前向的 FLOPs 為：

項目	FLOPs (每個樣本)	隨 T	隨 d
QKVO 投影	$8 T d^2$	線性	平方
注意力分數 QK^T	$2 T^2 d$	平方	線性
加權求和 AV	$2 T^2 d$	平方	線性
FFN (兩矩陣)	$4 T d d_{ff}$	線性	平方

第 11.7.1 節已經分析過各項占比。這裡補上一個實務工具：把估算與實測對照，驗證你的計算沒錯。

formosa/model/profile.py

```
def count_params(model, trainable_only=False):
    ps = [p for p in model.parameters()
           if p.requires_grad or not trainable_only]
    return sum(p.numel() for p in ps)

def estimate_params(cfg) -> dict:
    """依設定精確估算，不需建立模型"""
    d, L, V = cfg.d_model, cfg.n_layers, cfg.vocab_size
    d_ff = cfg.d_ff
    attn = 4 * d * d
    ffn = (3 if cfg.activation == "swiglu" else 2) * d * d_ff
    norms = 2 * d if cfg.norm_type == "rmsnorm" else 4 * d
    per_layer = attn + ffn + norms
    emb = V * d * (1 if cfg.tie_weights else 2)
    total = emb + L * per_layer + d
    return {"per_layer": per_layer, "all_layers": L * per_layer,
            "embedding": emb, "total": total,
            "emb_ratio": round(emb / total, 3)}

def estimate_flops(cfg, seq_len: int, batch: int = 1) -> dict:
    """單次前向的 FLOPs；訓練約為 3 倍（前向 + 反向）"""
    d, L, T, B = cfg.d_model, cfg.n_layers, seq_len, batch
    d_ff = cfg.d_ff
    proj = 8 * T * d * d
    attn = 4 * T * T * d
    ffn = 2 * (2 if cfg.activation != "swiglu" else 3) * T * d * d_ff
    per_layer = proj + attn + ffn
    head = 2 * T * d * cfg.vocab_size
    fwd = B * (L * per_layer + head)
    return {"forward": fwd, "train_step": 3 * fwd,
            "attn_ratio": round(attn / per_layer, 3),
            "TFLOPs_forward": round(fwd / 1e12, 4)}
```

```
def verify(model, cfg):
    """估算與實測必須一致——不一致代表你的模型結構與設定不符"""
    est = estimate_params(cfg)["total"]
    act = count_params(model)
    diff = abs(est - act) / act
    assert diff < 0.01, f"估算 {est:,} 與實測 {act:,} 差異 {diff:.1%}"
    print(f"參數量驗證通過：{act:,} (估算 {est:,})")
```

verify 這個函式應該在每次建立模型後執行。如果估算與實測不符，代表你的模型結構與設定檔的認知有落差——可能是漏了某個元件、多了某個 bias、或是 d_ff 被自動調整過。這是一個三十秒就能做完的健全性檢查。

12.5.4 從參數量到記憶體

把第 3.5.5 節的記憶體公式套用到本章的三種尺寸 (AdamW、混合精度、含 FP32 主權重)：

尺寸	參數	權重與梯度	最佳化器狀態	合計 (不含 activation)
tiny 4.88 M	4.88 M	約 20 MB	約 39 MB	約 59 MB
small 41.6 M	41.6 M	約 166 MB	約 333 MB	約 499 MB
base 109.5 M	109.5 M	約 438 MB	約 876 MB	約 1.31 GB

(BF16 權重與梯度各 2 bytes、FP32 主權重與 m、v 各 4 bytes，合計每參數 16 bytes。)

base 版本只需要 1.31 GB 就能訓練，遠低於多數 GPU 的容量——但這還沒算 activation。activation 的大小與批次、序列長度、以及是否使用 checkpointing 有關，通常才是真正的瓶頸。第 18 章會教你完整剖析。

12.6 從 GPT-2 到 Llama-style：元件替換的動機

12.6.1 六項替換

2019 年的 GPT-2 與今天的 Llama-style 模型，骨架幾乎一樣，差別在六個元件。理解每一項的動機，你就理解了近年架構演進的主線。

元件	GPT-2	Llama-style	替換動機
正規化	LayerNorm	RMSNorm	省一半參數與計算，效果相當
正規化位置	Pre-LN	Pre-LN	相同（GPT-2 已改用）
位置編碼	可學習嵌入	RoPE	可外推、無額外參數
激勵函數	GELU	SwiGLU	門控結構，表達力較強
注意力	MHA	GQA	推論時 KV cache 大幅縮小
bias	各處都有	幾乎全部移除	省參數，效果無明顯損失

注意這六項的共同性質：它們都不是「更複雜」，而是「更精簡或更實用」。RMSNorm 比 LayerNorm 簡單、去掉 bias 比保留簡單、GQA 比 MHA 省資源。唯一變複雜的是 SwiGLU，而它用參數量對齊換回了公平性。

這反映了一個趨勢：架構層面的創新已經相當飽和，近年的進步主要來自資料、訓練方法與規模（第 2.8.5 節的年表）。本章教的這六項替換，大致就是 2019 到 2026 年間架構層面的全部主要變動。

12.6.2 什麼時候該用哪一種

情境	建議	理由	章節
從零訓練新模型	全部採用 Llama-style	生態成熟、工具完整	第 13 章
載入既有權重	完全比照底模	任何不一致都會導致失敗	第 22 章
邊緣部署小模型	RMSNorm + GQA 必用	參數與記憶體都要省	第 26 章
教學與研究對照	保留可切換的設定	需要做消融實驗	本章程式 12C

第二列是實務上最重要的：如果你要在別人的模型上做持續預訓練或微調（第六篇的主線），你不能自由選擇這些元件——必須完全比照底模。載入權重時若元件不符，模型不會報錯，只會產生沒有意義的輸出。這是第 10.4.7 節提過的「靜默失敗」的另一個實例。

12.6.3 本書的設定

MiniFormosaGPT 採用 Llama-style 的全部六項，但保留切換能力供對照實驗。

formosa/config.py — 模型設定

```
from dataclasses import dataclass, field

@dataclass
class ModelConfig:
    # 規模
    vocab_size: int = 32000
    d_model: int = 512
    n_layers: int = 8
    n_heads: int = 8
    n_kv_heads: int | None = None      # None 表示 MHA
    d_ff: int | None = None           # None 則依 activation 自動決定
    max_seq_len: int = 1024

    # 元件選擇 (預設為 Llama-style)
    norm_type: str = "rmsnorm"        # rmsnorm | layernorm
    pre_norm: bool = True             # True = Pre-LN
    activation: str = "swiglu"        # swiglu | gelu
    pos_encoding: str = "rope"        # rope | learned | sinusoidal
    bias: bool = False
    tie_weights: bool = True

    # 正則化
    attn_dropout: float = 0.0
    resid_dropout: float = 0.0
    emb_dropout: float = 0.0

    # 初始化
    init_std: float = 0.02
    scale_residual_init: bool = True  # 殘差分支除以 sqrt(2L)

    def __post_init__(self):
        assert self.d_model % self.n_heads == 0, \
            f"d_model={self.d_model} 無法被 n_heads={self.n_heads} 整除"
        if self.n_kv_heads is None:
            self.n_kv_heads = self.n_heads
        assert self.n_heads % self.n_kv_heads == 0, \
            "n_heads 必須是 n_kv_heads 的倍數"
        if self.d_ff is None:
            if self.activation == "swiglu":
                raw = int(8 * self.d_model / 3)
                self.d_ff = 128 * ((raw + 127) // 128)
            else:
                self.d_ff = 4 * self.d_model

    @property
    def d_head(self) -> int:
        return self.d_model // self.n_heads
```

這份設定檔延續第 4.1.2 節的原則：所有驗證都放在 `__post_init__`，讓錯誤在建立設定的那一行就爆炸，而不是在訓練十分鐘後才出現一個難以理解的形狀錯誤。

12.7 初始化與正則化的實務取舍

12.7.1 初始化清單

第 3.5.7 節講過原則，這裡給出本書使用的完整清單。

參數	初始化	標準差	理由
嵌入層	常態分布	0.02	GPT 慣例
注意力 QKV 投影	常態分布	0.02	同上
注意力輸出投影	常態分布	$0.02 / \sqrt{2L}$	殘差分支需縮小
FFN 第一層	常態分布	0.02	—
FFN 輸出層	常態分布	$0.02 / \sqrt{2L}$	殘差分支需縮小
正規化的 γ	全 1	—	起點是「不做調整」
所有 bias (若有)	全 0	—	同上

初始化的實作

```
import math
import torch.nn as nn

def init_weights(model, cfg):
    std = cfg.init_std
    for module in model.modules():
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=std)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=std)

    # 殘差分支的輸出投影：除以 sqrt(2L)
    if cfg.scale_residual_init:
        scale = 1.0 / math.sqrt(2 * cfg.n_layers)
        for name, p in model.named_parameters():
            # 每層有兩個殘差分支：attn.proj 與 ffn.w2
            if name.endswith("attn.proj.weight") or \
               name.endswith("ffn.w2.weight"):
                with torch.no_grad():
                    p.mul_(scale)
    return model

@torch.no_grad()
def check_activation_scale(model, cfg, device="cpu"):
    """訓練前的三十秒健檢：逐層輸出的標準差應在同一量級"""
    model.eval()
    x = torch.randint(0, cfg.vocab_size, (2, 64), device=device)
    h = model.tok_emb(x)
```

```
print(f"{'層':>4} {'std':>8}")
print(f"{'emb':>4} {h.std().item():8.4f}")
for i, blk in enumerate(model.blocks):
    h = blk(h)
    print(f"{i:>4} {h.std().item():8.4f}")
```

check_activation_scale 是第 3.5.7 節建議的那個「三十秒健檢」的實作。訓練開始前跑一次，如果各層的標準差逐層放大或縮小數倍，就代表初始化有問題——這比訓練跑了兩小時才發現 loss 不降要便宜太多。

順帶一提，因為 Pre-LN 的殘差流會累積（12.2.3 節），你會看到標準差確實隨層數緩慢上升。那是正常的；不正常的是指數級的成長或萎縮。

12.7.2 dropout 與 weight decay

正則化	小模型小資料	大模型大資料	本書設定
dropout	0.1 左右	通常 0	tiny/small 用 0.1，base 用 0
weight decay	0.1	0.1	一律 0.1
weight decay 排除	正規化、bias、嵌入	同左	見第 10.2.4 節

這張表的邏輯很簡單：正則化是用來對抗過擬合的，而過擬合只在「模型容量大於資料量」時發生。大模型看的資料多到不可能背完，dropout 反而只是拖慢收斂。第 13 章訓練 MiniFormosaGPT 時，你可以親自驗證這一點——用小語料訓練 base 版本，關掉 dropout 會明顯看到訓練與驗證損失分家。

12.7.3 把 block 疊起來會遇到的三個問題

單一 block 測試通過，不代表疊 N 層之後沒問題。以下三個問題只在堆疊之後才會出現，第 13 章會實際遇到。

問題	現象	成因	對策
記憶體隨層數線性成長	層數加倍就 OOM	每層的 activation 都要保留供反向傳播	activation checkpointing (第 18 章)
初期梯度不穩	前幾百步 loss 劇烈震盪	殘差流尚未穩定	warmup 與 $\sqrt{2L}$ 縮放

問題	現象	成因	對策
深層退化	疊到某個深度後效果不再提升	有效深度受限於梯度品質	調整初始化與學習率

第一列是最直接的限制。反向傳播需要前向時的中間值，所以記憶體占用大致正比於「層數 × 批次 × 序列長度 × 維度」。這意味著在固定的 GPU 上，加深模型就必須減小批次或序列長度——這是第 13 章規劃訓練時必須面對的取舍。

activation checkpointing 的想法是：不保留所有中間值，只保留每隔幾層的檢查點，反向傳播時重新計算中間的部分。這是典型的「用計算換記憶體」，通常能省下大量記憶體而只增加約三成的計算時間。第 18 章會完整處理。

12.7.4 一份組裝前的檢查清單

在進入第 13 章之前，用這份清單確認你的零件都齊備且正確。每一項都對應本章或前兩章的一個測試。

項次	檢查什麼	通過標準	章節
1	嵌入層與 RoPE 快取	形狀正確、相對位置性質成立	第 10 章
2	多頭因果注意力	五個因果測試通過、與融合版一致	第 11 章
3	正規化	統計量以 float32 計算、 γ 初始化為 1	12.1
4	前饋網路	SwiGLU 與 GELU 參數量對齊	12.3
5	Block 組裝	歸零後為恆等、兩個獨立 Norm	12.4
6	參數量	估算與實測差異小於 1%	12.5
7	初始化	逐層標準差在同一量級	12.7
8	設定驗證	錯誤設定會在建立時就拋出例外	第 4.1.2 節

這八項全部通過，你就可以進入第 13 章了。任何一項沒過就往下走，你會在訓練跑了幾小時之後回來找同樣的 bug——而那時候的除錯難度會高上一個數量級，因為你無法確定問題出在模型、資料還是訓練設定。

程式實作

本章三個實作完成之後，MiniFormosaGPT 的所有零件就齊了。第 13 章只需要把它們疊起來、接上資料，就能開始訓練。

程式 12A 可設定的 Transformer Block

目標：實作一個能透過設定檔切換 LayerNorm/RMSNorm、Pre-LN/Post-LN、GELU/SwiGLU 的 block，並通過完整測試。

測試	驗證什麼	通過標準	節次
形狀不變	進出形狀相同	完全相等	12.4.1
殘差存在	子層歸零後退化為恆等	誤差小於 $1e-5$	12.4.3
兩個獨立 Norm	norm1 與 norm2 非同一物件	is not 檢查	12.4.3
因果性保留	未來不影響過去	誤差小於 $1e-5$	第 11.4.3 節
SwiGLU 參數對齊	與 GELU 版參數量相同	差異小於 1%	12.3.3
設定切換	八種組合皆可建立並執行	無例外	12.6.3

tests/test_block.py (節錄)

```
import itertools, pytest, torch
from formosa.config import ModelConfig
from formosa.model.block import TransformerBlock

COMBOS = list(itertools.product(
    ["rmsnorm", "layernorm"], [True, False], ["swiglu", "gelu"]))

@pytest.mark.parametrize("norm,pre,act", COMBOS)
def test_all_combinations(norm, pre, act):
    cfg = ModelConfig(d_model=64, n_heads=4, n_layers=2,
                      norm_type=norm, pre_norm=pre, activation=act)
    b = TransformerBlock(cfg)
    x = torch.randn(2, 8, 64)
    assert b(x).shape == x.shape

def test_swiglu_param_parity():
    """對齊後的 SwiGLU 與 GELU 參數量應接近"""
```

```

a = ModelConfig(d_model=768, activation="gelu")
b = ModelConfig(d_model=768, activation="swiglu")
from formosa.model.ffn import FeedForward
na = FeedForward(768, a.d_ff, "gelu").n_params()
nb = FeedForward(768, b.d_ff, "swiglu").n_params()
assert abs(na - nb) / na < 0.05, (na, nb)

```

延伸挑戰：把 Post-LN 版本疊到 24 層，用相同的學習率訓練 500 步，觀察它是否發散；再對 Pre-LN 做同樣的事。這是程式 12C 的預備實驗，也是 12.2.2 節那個論證的實證。

程式 12B 參數量與 FLOPs 計算器

目標：建立一個工具，輸入設定即輸出參數量、記憶體與訓練資源估算，並與實際建立的模型核對。

scripts/model_calculator.py (節錄)

```

import argparse, json
from formosa.config import ModelConfig
from formosa.model.profile import estimate_params, estimate_flops

def full_report(cfg, seq_len=1024, batch=8, mfu=0.35,
                gpu_tflops=100.0):
    p = estimate_params(cfg)
    f = estimate_flops(cfg, seq_len, batch)

    # 記憶體：BF16 權重+梯度 4 bytes，FP32 主權重+m+v 12 bytes
    bytes_per_param = 16
    mem_states = p["total"] * bytes_per_param / 1e9

    # 一步訓練的時間估算（見第 3.2.5、17 章）
    step_seconds = f["train_step"] / (gpu_tflops * 1e12 * mfu)

    return {
        "params_total_M": round(p["total"] / 1e6, 2),
        "params_per_layer_M": round(p["per_layer"] / 1e6, 3),
        "embedding_ratio": p["emb_ratio"],
        "states_memory_GB": round(mem_states, 3),
        "forward_TFLOPs": f["TFLOPs_forward"],
        "attn_flops_ratio": f["attn_ratio"],
        "est_step_seconds": round(step_seconds, 4),
        "tokens_per_step": batch * seq_len,
        "est_tokens_per_sec": round(batch * seq_len / step_seconds),
    }

PRESETS = {
    "tiny": dict(d_model=128, n_layers=4, n_heads=4, max_seq_len=128),
    "small": dict(d_model=512, n_layers=8, n_heads=8, max_seq_len=512),
    "base": dict(d_model=768, n_layers=12, n_heads=12, max_seq_len=1024),
}

```

必須完成的三項驗證：

- 對三種預設尺寸，估算與實際建立模型後的 count_params 差異必須小於 1%。若不符，找出漏

算的元件。

- 把 estimate_flops 的結果與第 4 章程式 4B 量到的實際吞吐量對照，算出你的裝置在這個模型上的 MFU。
- 掃描 d_model 從 128 到 2048，畫出「嵌入層占總參數的比例」曲線。在什麼尺寸以下，詞彙表會成為主要成本？

程式 12C Pre-LN 與 Post-LN 的消融實驗

目標：用實驗驗證 12.2.2 節的論證。這是本書第二個完整的消融實驗（第一個是第 10 章的初始化），方法論沿用第 5.5 節。

變因	設定	為什麼要固定	節次
層數	4、8、16、24 (自變數)	這是要觀察的維度	—
正規化位置	Pre-LN 與 Post-LN (自變數)	這是要比較的對象	12.2
其餘設定	完全相同	否則無法歸因	第 5.5.2 節
學習率	固定，且分別測試有無 warmup	Post-LN 對此極敏感	第 3.5.6 節
隨機種子	至少 3 個	單次結果不可信	第 5.5.2 節

ch12/lab12c_prenorm_ablation.py (主流程)

```
def run_ablation(layers=(4, 8, 16, 24), seeds=(0, 1, 2),
                 warmup_options=(0, 200), steps=1000):
    results = []
    for L in layers:
        for pre in (True, False):
            for warmup in warmup_options:
                for seed in seeds:
                    torch.manual_seed(seed)
                    cfg = ModelConfig(d_model=256, n_layers=L, n_heads=8,
                                      pre_norm=pre, max_seq_len=256)
                    model = build_model(cfg)
                    hist = train(model, steps=steps, warmup=warmup,
                                lr=3e-4, track_grad_norm=True)
                    results.append({
                        "layers": L, "pre_norm": pre, "warmup": warmup,
                        "seed": seed,
                        "diverged": any(math.isnan(l) for l in hist["loss"]),
                        "final_loss": hist["loss"][-1],
                        "max_grad_norm": max(hist["grad_norm"]),
                        # 殘差流的變異數成長（見 12.2.3 節）
                        "residual_var": hist["residual_var_by_layer"],
                    })
    return results
```

你應該觀察到並寫進報告的四件事：

- 在淺層（4 到 8 層）時，兩者差異不大，Post-LN 甚至可能略佳。這符合 12.2.2 節的說法。
- 層數增加後，Post-LN 在沒有 warmup 時的發散比例明顯上升。統計「發散的執行次數 ÷ 總次數」。
- Pre-LN 的殘差流變異數隨層數上升，且大致呈線性。把它畫出來，與 12.2.3 節的模擬對照。
- 梯度範數的最大值：Post-LN 在訓練初期的峰值通常遠高於 Pre-LN。這是它需要 warmup 的直接證據。

一個誠實的提醒

這個實驗在小模型與小語料上，兩者的差異可能沒有教科書描述的那麼戲劇化——尤其如果你只跑 1000 步。這不是你做錯了。請如實報告你觀察到的效果大小，並說明你的實驗規模可能不足以呈現完整差異。「在我的設定下差異不顯著」是一個合法且誠實的結論，遠比誇大結果有價值。

系統案例：MiniFormosaGPT 三種尺寸的規劃

本章的系統案例是為第 13 章的訓練預作規劃：設計 tiny、small、base 三種尺寸，對應第 4.4.3 節的 A、B、C 三級硬體，並附完整的資源估算。

三種尺寸的設計

項目	tiny (A 級)	small (B 級)	base (C 級)	設計理由
d_model	128	512	768	—
n_layers	4	8	12	—
n_heads	4	8	12	d_head 維持 64
n_kv_heads	4	4	4	GQA，省 KV cache
max_seq_len	128	512	1024	受記憶體限制
vocab_size	8,000	32,000	32,000	tiny 用小詞彙表
dropout	0.1	0.1	0.0	依資料量調整

tiny 版本刻意使用 8,000 的小詞彙表，理由在 12.5.2 節已經說明：在極小的模型上，32,000 的詞彙表會占掉八成以上的參數，讓模型幾乎沒有容量學習語言結構。這是第 7 章 tokenizer 設計與本章模

型設計必須一起考慮的例子。

必須交付的五項

1. 三份設定檔 (YAML)，可直接被第 13 章的訓練腳本讀取。
2. 資源估算表：每種尺寸的參數量、記憶體、單步 FLOPs、以及在你的硬體上的預估訓練速度。
3. 驗證紀錄：estimate_params 與實際 count_params 的對照，差異必須小於 1%。
4. 初始化健檢：三種尺寸各跑一次 check_activation_scale，附逐層標準差的輸出。
5. 時間預算：在你的硬體上，訓練 1 億個詞元各需要多久？據此決定第 13 章的訓練規模。

評分重點

- 估算與實測是否一致。不一致而未追查原因者不予計分。
- 設定選擇是否有理由。「照抄別人的」不算理由，必須連結到硬體限制或第 7、10、11 章的分析。
- 時間預算是否務實。若估算顯示需要三週，那就該調整規模——第 13 章的目標是「跑得完」而非「跑得大」。
- 是否考慮了 activation 記憶體。只算權重與最佳化器狀態的估算會嚴重低估需求。

章末評量

一、概念題

1. 為什麼 Transformer 用 LayerNorm 而非 BatchNorm？請從語言模型的資料特性與推論情境兩方面說明。
2. RMSNorm 相對 LayerNorm 省了什麼？為什麼效果幾乎不變？
3. 為什麼正規化的統計量要用 float32 計算？在 BF16 下不做會有什麼風險？
4. 用梯度路徑的角度說明為什麼 Pre-LN 比 Post-LN 更容易訓練深層模型。
5. Pre-LN 的副作用是什麼？需要哪兩個配套措施？
6. 前饋網路與注意力的分工是什麼？為什麼說一個橫向、一個縱向？
7. SwiGLU 為什麼要把 d_{ff} 縮小到 $8d/3$ ？不縮小會有什麼比較上的問題？
8. 「子層權重歸零後 block 退化為恆等映射」這個測試在驗證什麼？為什麼它很有效？
9. 從 GPT-2 到 Llama-style 的六項替換有什麼共同性質？這反映了什麼趨勢？

二、計算題

1. 設 $V = 32,000$ 、 $d = 1,024$ 、 $L = 16$ 、 $d_{ff} = 4,096$ 、權重綁定、使用 RMSNorm 與 GELU。計算每層參數量、所有層合計、嵌入層、以及總參數量。
2. 承上題，若改用 SwiGLU 並依 $8d/3$ 對齊（取整到 128 的倍數）， d_{ff} 是多少？每層參數量變成多少？
3. 承第 1 題，用 AdamW 混合精度訓練，權重、梯度與最佳化器狀態合計需要多少 GB？
4. 設 $T = 2,048$ 、 $B = 4$ 。計算第 1 題模型的單次前向 FLOPs，以及注意力占單層的比例。
5. 一個 $d = 512$ 、 $L = 8$ 的模型，殘差分支的初始化標準差應為多少？（ $init_std = 0.02$ ）
6. tiny 版本（ $d = 128$ 、 $L = 4$ 、 $V = 32,000$ ）的嵌入層占總參數的比例是多少？若把 V 降到 8,000 呢？

三、除錯題

1. 同學的 24 層模型訓練時 loss 在第 30 步就變成 NaN。請列出四個最可能的原因與檢查順序。
2. 同學的 block 通過了形狀測試，但訓練時 loss 幾乎不降。check_activation_scale 顯示各層標準差從 0.02 一路降到 0.0001。請診斷。
3. 同學的 estimate_params 算出 109.5 M，但實際模型是 134.1 M。請列出三個可能漏算的元件。
4. 同學把 norm1 與 norm2 寫成同一個物件。程式正常運作，參數量也只差 512 個。請說明這會造

成什麼影響，以及如何避免這類錯誤。

5. 同學實作的 SwiGLU 比 GELU 版本效果好，但參數量多了 50%。請說明這個比較的問題與正確做法。
6. 同學的 Pre-LN 模型在最後一層之後直接接輸出投影，訓練初期正常但後期 loss 上升。請說明可能的原因。

四、系統設計題

1. 你要為一個部署在 8 GB 記憶體邊緣裝置上的模型設計架構。目標是繁中的產線維修問答，上下文 2,048。請決定所有的架構參數與元件選擇，並用計算支撐每一個決定。特別說明你如何在詞彙表大小與模型容量之間取捨。
2. 你的實驗室要做一個「架構元件的教學對照實驗」，讓學生比較六項 Llama-style 替換各自的貢獻。請設計這個實驗：需要跑幾組、如何固定變因、用什麼指標、以及如何在有限的算力下得到可信的結論。

五、臺灣情境與倫理題

1. 12.5.2 節指出，tiny 模型的參數有八成在嵌入層。請說明這對繁體中文特別不利的原因（連結第 7 章），並提出至少兩種在小模型上緩解這個問題的方法。
2. 架構論文常宣稱某項改動帶來效果提升。請說明在閱讀這類主張時，你會檢查哪五件事才願意相信它（提示：參數量對齊、訓練預算、種子數、評測集、以及是否公開程式碼）。

評量提示（給自學者）

- 計算題第 1 題：每層 $4d^2 + 2 \cdot d \cdot d_{ff} + 2d = 4,194,304 + 8,388,608 + 2,048 = 12,584,960$ ；16 層約 201.36 M；嵌入 32.77 M；總計約 234.13 M。
- 計算題第 2 題： $8 \times 1024 / 3 \approx 2730$ ，取整到 2,816；每層 FFN 為 $3 \times 1024 \times 2816 = 8,650,752$ ，略高於 GELU 版的 8,388,608（取整造成）。
- 計算題第 3 題： $234.13 \text{ M} \times 16 \text{ bytes} \approx 3.75 \text{ GB}$ 。
- 計算題第 5 題： $0.02 / \sqrt{(2 \times 8)} = 0.02 / 4 = 0.005$ 。
- 計算題第 6 題：tiny 的每層參數 $4 \times 128^2 + 2 \times 128 \times 512 = 65,536 + 131,072 = 196,608$ ，四層 786,432；嵌入 $32,000 \times 128 = 4,096,000$ ；占比約 83.9%。V 降到 8,000 時嵌入為 1,024,000，占比降至約 56.6%。
- 除錯題第 2 題：殘差連接缺失或接錯。訊號在沒有捷徑的情況下逐層衰減，這正是 test_residual_present 要抓的錯誤。

- 除錯題第 3 題：可能是未綁定權重（多出 $V \times d$ ）、bias 未關閉、或 d_{ff} 被自動調整成比設定更大的值。

研究延伸與版本注意事項

- Pre-LN 與 Post-LN 的理論分析：有幾篇論文從梯度尺度的角度給出了嚴謹的推導，對本章 12.2 節有直接補充。
- RMSNorm 原始論文：篇幅很短，論證清楚，適合作為第一篇精讀的架構論文。
- GLU 變體的比較研究：系統性地比較了各種門控激勵函數，並強調了參數量對齊的重要性——這正是 12.3.3 節的來源。
- Llama 系列的技術報告：它們清楚列出了架構選擇與理由，是理解現代模型設計的最佳一手資料。
- 深層 Transformer 的初始化研究：關於 $\sqrt{2L}$ 縮放與其他深度相關的初始化技巧。
- 本章的元件在近年相當穩定，但實作細節（例如 GELU 的近似版本、RMSNorm 的 ϵ 位置）在不同框架間仍有差異。載入他人權重時務必逐項核對。

常見問題

問：這六項替換各自帶來多少提升？答：老實說，公開的、控制良好的消融資料並不多，而且效果會隨規模與資料而變。多數替換的個別提升都不大（百分之一到幾的量級），真正的價值在於它們是「免費或省資源」的——RMSNorm 更快、去 bias 省參數、GQA 省記憶體。當一項改動不花錢又不變差時，採用它就是理性的。

問： d_{ff} 一定要是 $4d$ 嗎？答：不一定，這只是一個廣泛使用的慣例。有研究探討過不同比例，但 $4d$ （或 SwiGLU 的 $8d/3$ ）在實務上表現穩健，且工具鏈都以它為預設。除非你有明確理由，否則沒必要偏離。

問：可以在不同層用不同的設定嗎？答：技術上可以，也有模型這樣做（例如部分層用局部注意力、部分用全域）。但這會讓實作、分散式切分與權重載入都變複雜。本書維持所有層相同，這也是絕大多數模型的做法。

問：為什麼現代模型幾乎都不用 bias？答：實測顯示影響很小，而拿掉它可以省參數、簡化實作、也讓某些最佳化更容易。特別是在注意力的 QKV 投影上，因為後面緊接著 softmax，bias 的效果本來就有限（第 11.3.3 節）。

教師使用建議

本章排在第 12 週，建議 2 小時講授加 1 小時實驗。本章的難度低於第 11 章，但細節多，教學重點是「讓學生養成檢查的習慣」。

時段	內容	互動設計	注意事項
前 25 分鐘	12.1 至 12.2 節，正規化與 Pre/Post-LN	白板畫兩種資料流，請學生找出梯度路徑	梯度捷徑要畫出來
中間 25 分鐘	12.3 節，前饋網路與 SwiGLU	現場算一次 8d/3 的參數對齊	參數對齊是重點
後 30 分鐘	12.4 節，組裝與六個常見錯誤	給一份有 bug 的 block 讓學生找	找錯比講對有效
最後 40 分鐘	12.5 至 12.7 節與作業說明	全班一起算一次 base 版的參數量	算完與程式對照
實驗課 1 小時	程式 12A 與 12B	兩人一組互相檢查參數量估算	12C 作為課後作業

一個效果很好的課堂活動：發給每組一份「有 bug 的 Transformer block」，其中植入六個常見錯誤中的兩個（例如殘差加錯對象、兩層共用同一個 Norm）。要求他們在十五分鐘內用測試找出來。學生會發現：形狀測試通過不代表正確，必須有 `test_residual_present` 這類針對結構的測試。

另一個建議：在講 12.5 節時，讓全班一起手算 base 版本的參數量，然後當場執行 `count_params` 對照。第一次通常會差幾百萬——找出差在哪裡的過程，比直接給答案有價值得多。

高中授課的調整：12.1 只講「正規化是把數字拉回合理範圍」的直覺，公式可略；12.2 用「高速公路上不要設收費站」的比喻講 Pre-LN；12.3 講 FFN 的「先放大再縮小」與門控直覺，SwiGLU 的參數對齊可略；12.4 完整教（組裝很直觀，且六個錯誤的示範效果好）；12.5 只做參數量的粗估；12.6、12.7 挑重點。程式做 12A 的簡化版與 12B。

本章小結

1. 正規化把每層輸入拉回穩定尺度；Transformer 用 LayerNorm 而非 BatchNorm，因為語言資料的批次統計不穩且推論時批次可能為 1。
2. RMSNorm 拿掉中心化，省一半參數與部分計算，效果幾乎不變，是現代主流。
3. 正規化的統計量應以 float32 計算，避免 BF16 尾數不足造成的失準。
4. Pre-LN 讓殘差路徑完全乾淨，梯度可無損傳到淺層；Post-LN 每層都被正規化打斷，深層時容易失真。
5. Pre-LN 的代價是殘差流變異數隨層數線性成長（模擬顯示 24 層後約 16 倍），需要最終正規化

與 $\sqrt{(2L)}$ 初始化縮放。

6. 注意力橫向搬運資訊，前饋網路縱向做逐位置變換； d_{ff} 通常取 $4d$ 。
7. SwiGLU 有三個矩陣，須把 d_{ff} 縮到 $8d/3$ 才能與 GELU 版本參數量對齊，否則比較不公平。
8. 一個 Pre-LN block 只有兩行： $x += \text{Attention}(\text{Norm}(x))$ 、 $x += \text{FFN}(\text{Norm}(x))$ ；進出形狀相同是可堆疊的根本原因。
9. 六個常見組裝錯誤中，最隱蔽的是兩層共用同一個 Norm 與殘差加錯對象；用「歸零後應為恆等」的測試可以有效抓到。
10. 參數量約為 $V \cdot d + L \cdot (4d^2 + 2 \cdot d \cdot d_{ff})$ ；小模型的嵌入層可能占八成以上，這使詞彙表大小成為關鍵設計。
11. 估算與實測的參數量必須一致，不一致代表結構與設定認知有落差，應立即追查。
12. GPT-2 到 Llama-style 的六項替換 (RMSNorm、RoPE、SwiGLU、GQA、去 bias、Pre-LN) 共同性質是更精簡或更省資源，而非更複雜。
13. 載入他人權重時必須完全比照底模的元件配置，任何不一致都會造成靜默失敗。
14. 初始化的關鍵是殘差分支除以 $\sqrt{(2L)}$ ；訓練前用 `check_activation_scale` 做三十秒健檢，可省下數小時的無效訓練。

成果檢核

完成本章後你必須繳交：

- TransformerBlock 模組 (程式 12A)：支援八種設定組合，通過六項測試。
- 參數量與 FLOPs 計算器 (程式 12B)：三種尺寸的估算與實測對照，差異小於 1%。
- 嵌入占比曲線： d_{model} 從 128 到 2048 的掃描結果，並指出詞彙表成為主要成本的臨界點。
- Pre-LN 消融報告 (程式 12C)：四種層數、兩種配置、有無 warmup、三個種子的結果，含發散率統計與殘差流變異數曲線。
- 三種尺寸的設定檔與資源估算表，含 activation 記憶體의 考量。
- 初始化健檢輸出：三種尺寸各一份逐層標準差紀錄。

第 3 篇到此已經備齊所有零件：第 10 章的嵌入與 RoPE、第 11 章的多頭因果注意力、本章的正規化、前饋網路與 block。第 13 章只剩一件事——把它們疊起來、接上第 7 章的 tokenizer 與第 9 章的語料，然後按下訓練。那將是本書第一次，你會看到自己寫的每一行程式，生出一句通順的繁體中文。

第 13 章

從零建構 MiniFormosaGPT 與文字生成器

Building MiniFormosaGPT and a Text Generator from Scratch

難度：核心 / 里程碑 | 建議授課：第 13 週 | 硬體需求：A 級可完成 tiny 版完整訓練（約 2–4 小時）

叫獸開講

當你的模型第一次吐出一句通順的繁體中文，那句話可能毫無意義。但那是你自己寫的每一行程式算出來的——從 tokenizer 到嵌入、從注意力到殘差、從損失到梯度，中間沒有任何一塊是黑盒子。那一刻的感覺，沒有任何 API 能給你。

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 把第 10 到 12 章的元件組裝成完整的 GPT 模型，並通過端到端的形狀與因果性測試。
2. 正確實作損失計算，包括位移對齊、loss mask 與詞元平均。
3. 用「過擬合單一批次」作為第一道健全性檢查，並解讀它失敗時的訊息。
4. 串接第 9 章的語料與第 7 章的 tokenizer，建立滑動視窗取樣的資料管線。
5. 寫出可中斷、可續訓、可重現的最小訓練迴圈。
6. 實作 greedy 解碼與逐詞元串流生成，並理解它與訓練時的差異。
7. 正確儲存與載入 checkpoint，含設定、環境指紋與相容性版本。
8. 辨識八種常見的訓練失敗模式，並知道各自的診斷順序。
9. 在 A 級筆電上完成一次完整的 tiny 版訓練，產出可生成繁體中文的模型。

這一章的性質不同

前十二章都在建立零件與觀念，這一章是第一次把它們全部接起來並實際運轉。因此本章的重點不在新知識，而在整合與除錯——你會遇到的問題八成不是「不懂原理」，而是「某個地方接錯了」。第 13.6 節的失敗模式表是本章最實用的部分，建議在開始寫程式前就先讀過一遍。

13.1 組裝完整模型

13.1.1 六個元件，一條資料流

第 10.5.3 節給過這條路徑，現在要把它變成真的程式。

層級	元件	形狀變化	來自
1	詞元嵌入	$(B,T) \rightarrow (B,T,d)$	第 10.1 節
2	嵌入 dropout	不變	第 12.7.2 節
3	$N \times \text{TransformerBlock}$	不變	第 12.4 節
4	最終正規化	不變	第 12.2.3 節
5	輸出投影	$(B,T,d) \rightarrow (B,T,V)$	第 10.5 節
6	損失或取樣	純量或 (B,T)	第 5.1、13.2 節

RoPE 的 $\cos/\sin$ 快取不在這條路徑上——它是在模型初始化時建好、每一層的注意力共用同一份（第 10.4.3 節）。這是一個容易漏掉的細節：如果你在每一層各建一份，記憶體會浪費 L 倍。

13.1.2 完整實作

formosa/model/gpt.py — MiniFormosaGPT

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from formosa.config import ModelConfig
from formosa.model.norm import RMSNorm, LayerNorm
from formosa.model.block import TransformerBlock
from formosa.model.rope import build_rope_cache

NORMS = {"rmsnorm": RMSNorm, "layernorm": LayerNorm}

class MiniFormosaGPT(nn.Module):
    def __init__(self, cfg: ModelConfig):
        super().__init__()
        self.cfg = cfg

        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.d_model)
        self.emb_drop = nn.Dropout(cfg.emb_dropout)
        self.blocks = nn.ModuleList(
            [TransformerBlock(cfg) for _ in range(cfg.n_layers)])
        self.final_norm = NORMS[cfg.norm_type](cfg.d_model)
        self.lm_head = nn.Linear(cfg.d_model, cfg.vocab_size, bias=False)

        if cfg.tie_weights:
            # 見第 10.5 節
            self.lm_head.weight = self.tok_emb.weight

        # RoPE 快取：整個模型共用一份，註冊為 buffer 以隨裝置搬移
        if cfg.pos_encoding == "rope":
            cos, sin = build_rope_cache(cfg.max_seq_len, cfg.d_head)
            self.register_buffer("rope_cos", cos, persistent=False)
            self.register_buffer("rope_sin", sin, persistent=False)
```



```

else:
    self.rope_cos = self.rope_sin = None

self.apply(self._init)
self._scale_residual_init()

def _init(self, m):
    if isinstance(m, nn.Linear):
        nn.init.normal_(m.weight, 0.0, self.cfg.init_std)
        if m.bias is not None:
            nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Embedding):
        nn.init.normal_(m.weight, 0.0, self.cfg.init_std)

@torch.no_grad()
def _scale_residual_init(self):
    """殘差分支除以 sqrt(2L)，見第 12.7.1 節"""
    if not self.cfg.scale_residual_init:
        return
    scale = (2 * self.cfg.n_layers) ** -0.5
    for name, p in self.named_parameters():
        if name.endswith("attn.proj.weight") or name.endswith("ffn.w2.weight"):
            p.mul_(scale)

def forward(self, idx, targets=None):
    B, T = idx.shape
    assert T <= self.cfg.max_seq_len, \
        f"序列長度 {T} 超過上限 {self.cfg.max_seq_len}"

    x = self.emb_drop(self.tok_emb(idx))          # (B,T,d)
    cos = self.rope_cos[:T] if self.rope_cos is not None else None
    sin = self.rope_sin[:T] if self.rope_sin is not None else None
    for blk in self.blocks:
        x = blk(x, cos, sin)
    x = self.final_norm(x)

    if targets is None:
        return self.lm_head(x[:, -1:, :])        # 生成時只算最後一格 # (B,1,V)

    logits = self.lm_head(x)                      # (B,T,V)
    loss = self.compute_loss(logits, targets)
    return logits, loss

```

注意 forward 在 targets 為 None 時只計算最後一個位置的 logits。這是一個重要的效能最佳化：生成時我們只需要下一個詞元的機率分布，沒必要為前面所有位置都算一次 (d, V) 的投影——那是整個模型最貴的一次矩陣乘法之一。

為什麼 rope 快取要用 persistent=False

register_buffer 的預設行為會把 buffer 存進 state_dict。但 RoPE 快取是可以從設定重新算出來的確定性內容，存進去只會讓 checkpoint 檔案變大、且在改變 max_seq_len 時造成載入衝突。設 persistent=False 讓它隨模型搬移裝置，但不進入 state_dict。這是一個小細節，但在第 22 章載入他人權重時會省下麻煩。

13.1.3 端到端測試

tests/test_gpt.py

```
import torch, pytest
from formosa.config import ModelConfig
from formosa.model.gpt import MiniFormosaGPT

@pytest.fixture
def tiny():
    return ModelConfig(vocab_size=100, d_model=32, n_layers=2,
                      n_heads=4, max_seq_len=16)

def test_forward_shape(tiny):
    m = MiniFormosaGPT(tiny)
    idx = torch.randint(0, 100, (2, 16))
    logits, loss = m(idx, targets=idx)
    assert logits.shape == (2, 16, 100)
    assert loss.ndim == 0

def test_generation_mode_shape(tiny):
    """targets=None 時只回傳最後一格"""
    m = MiniFormosaGPT(tiny)
    out = m(torch.randint(0, 100, (2, 16)))
    assert out.shape == (2, 1, 100)

def test_causality_end_to_end(tiny):
    """整個模型層級的因果性—最重要的一個測試"""
    m = MiniFormosaGPT(tiny).eval()
    idx = torch.randint(0, 100, (1, 12))
    idx2 = idx.clone(); idx2[0, -1] = (idx2[0, -1] + 1) % 100
    with torch.no_grad():
        a = m(idx, targets=idx)[0]
        b = m(idx2, targets=idx2)[0]
    assert torch.allclose(a[0, :-1], b[0, :-1], atol=1e-5), \
        "未來的輸入影響了過去的輸出"

def test_initial_loss_near_uniform(tiny):
    """未訓練模型的損失應接近  $\ln(V)$ —這是最好的初始化檢查"""
    import math
    m = MiniFormosaGPT(tiny)
    idx = torch.randint(0, 100, (4, 16))
    _, loss = m(idx, targets=idx)
    expected = math.log(100) # 約 4.605
    assert abs(loss.item() - expected) < 0.5, \
        f"初始 loss {loss.item():.3f} 偏離均勻分布 {expected:.3f}"

def test_weight_tying(tiny):
    m = MiniFormosaGPT(tiny)
    assert m.lm_head.weight is m.tok_emb.weight # 必須是同一物件
```

`test_initial_loss_near_uniform` 是一個極為實用的檢查。一個剛初始化、什麼都沒學的模式，對每個詞元的預測應該接近均勻分布，因此損失應該接近 $\ln(V)$ 。如果初始損失遠高於這個值，代表初始化的尺度有問題；如果遠低於，代表有東西洩漏了答案。三秒鐘的測試，抓得到一整類問題。

13.2 損失計算：三個容易錯的地方

13.2.1 位移對齊

第 5.1.4 節強調過這件事，這裡給出正確的實作。模型的第 t 個位置預測第 $t+1$ 個詞元，所以計算損失時必須把 logits 與 targets 錯開一格。

formosa/model/loss.py

```
import torch
import torch.nn.functional as F

IGNORE_INDEX = -100

def compute_loss(logits, targets, ignore_index=IGNORE_INDEX):
    """logits: (B, T, V) targets: (B, T)
    第 t 個位置的 logits 預測第 t+1 個 target"""
    B, T, V = logits.shape
    assert targets.shape == (B, T), targets.shape

    shift_logits = logits[:, :-1, :].contiguous()    # (B, T-1, V)
    shift_targets = targets[:, 1:].contiguous()        # (B, T-1)

    loss = F.cross_entropy(
        shift_logits.view(-1, V),                    # (B*(T-1), V)
        shift_targets.view(-1),                       # (B*(T-1),)
        ignore_index=ignore_index,
        reduction="mean",                             # 詞元平均
    )
    return loss

def compute_loss_sum(logits, targets, ignore_index=IGNORE_INDEX):
    """回傳總和與有效詞元數，供梯度累積正確平均（第 5.1.5 節）"""
    B, T, V = logits.shape
    sl = logits[:, :-1, :].reshape(-1, V)
    st = targets[:, 1:].reshape(-1)
    total = F.cross_entropy(sl, st, ignore_index=ignore_index,
                           reduction="sum")
    n_valid = (st != ignore_index).sum()
    return total, n_valid
```

第二個函式是為了梯度累積而存在的。第 5.1.5 節說過：如果每個小批次的有效詞元數不同，直接把各批次的平均損失再平均是錯的。正確做法是累加總和與詞元數，最後才相除。這在指令微調（第 23 章）時特別重要，因為每個樣本的計分長度都不同。

13.2.2 三種錯誤與它們的症狀

錯誤	寫法	症狀	如何抓到
忘記位移	logits 直接對 targets	loss 迅速降到接近 0	初始 loss 就明顯低於 $\ln(V)$
位移方向錯	logits[:,1:] 對 targets[:, -1]	loss 完全不降	手動檢查第一個樣本的對應
平均方式錯	各批次平均再平均	loss 略微偏差，收斂點不同	與 sum 版本對照

第一列是全書最常出現的錯誤。忘記位移之後，模型被要求「預測當前這個詞」——而它已經看到當前這個詞了，所以 loss 會迅速掉到接近零。訓練曲線看起來完美，但模型什麼都沒學到。第 11 章的因果遮罩錯誤也會產生類似症狀，兩者要分開診斷。

手動驗證位移是否正確

```
def inspect_shift(tok, ids, n=6):
    """印出前 n 個位置的「輸入 -> 預測目標」對應"""
    print(f"{'pos':>4} {'input':>10} {'predicts':>10}")
    for t in range(min(n, len(ids) - 1)):
        print(f"{'t':>4} {tok.decode([ids[t]]):>10}"
              f" {tok.decode([ids[t+1]]):>10}")

# 預期輸出（以「臺灣科技大學」為例）：
# pos      input      predicts
# 0         臺          灣
# 1         灣          科
# 2         科          技
# 若看到 input 與 predicts 相同，就是忘了位移
```

13.2.3 第一道健全性檢查：過擬合單一批次

第 4.3.2 節提過這個測試，現在要真正用它。它的邏輯很簡單：一個實作正確的模型，一定把幾十個樣本完全背下來。如果做不到，代表有根本性的錯誤。

scripts/overfit_check.py

```
import torch, math
from formosa.config import ModelConfig
from formosa.model.gpt import MiniFormosaGPT

def overfit_one_batch(cfg, steps=300, lr=1e-3, target=0.05, verbose=True):
    """訓練前必做：模型能不能把一個批次背起來"""
    torch.manual_seed(0)
    model = MiniFormosaGPT(cfg)
    opt = torch.optim.AdamW(model.parameters(), lr=lr)

    x = torch.randint(0, cfg.vocab_size, (4, min(64, cfg.max_seq_len)))
    init_loss = None
```

```

for s in range(steps):
    _, loss = model(x, targets=x)
    if s == 0:
        init_loss = loss.item()
    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()
    if verbose and s % 50 == 0:
        print(f"step {s:4d}  loss {loss.item():.4f}")

final = loss.item()
expected_init = math.log(cfg.vocab_size)
print(f"初始 {init_loss:.3f} (理論 {expected_init:.3f}) "
      f" 最終 {final:.4f}")
assert abs(init_loss - expected_init) < 0.6, "初始化尺度異常"
assert final < target, f"無法過擬合單一批次，final={final:.4f}"
return {"init": init_loss, "final": final}

```

這個測試同時檢查了兩件事：初始損失是否接近 $\ln(V)$ （初始化正確），以及模型能否收斂到接近零（梯度流動正確）。它在 tiny 設定下只需要幾十秒，卻能抓出大部分的實作錯誤。

觀察到	代表什麼	該檢查什麼	章節
初始 loss 遠高於 $\ln(V)$	初始化尺度過大	init_std、殘差縮放	第 12.7.1 節
初始 loss 遠低於 $\ln(V)$	答案洩漏	位移對齊、因果遮罩	13.2.1、第 11.4 節
loss 完全不動	梯度沒有流動	參數註冊、optimizer 拿到參數了嗎	第 4.2.3 節
loss 下降但停在 2.0	容量不足或有 bug	殘差連接、模型大小	第 12.4.3 節
loss 變成 NaN	數值爆炸	學習率、梯度裁切	第 3.6.3 節

13.3 資料管線：從語料到批次

13.3.1 把語料變成一長串詞元

第 9 章產出的是清理過的文本，第 7 章的 tokenizer 可以把它變成詞元。訓練前的最後一步是把它們編碼成一個連續的整數陣列並存成二進位檔——這樣訓練時可以用記憶體映射讀取，不必整份載入。

scripts/prepare_data.py

```

import numpy as np
from pathlib import Path
from formosa.tokenizer import load_tokenizer

```

```

def encode_corpus(jsonl_path: Path, tok, out_path: Path,
                  dtype=np.uint16, add_eos=True):
    """把語料編碼成一長串 token id，存成 .bin
    uint16 上限 65535，若詞彙表更大需改用 uint32"""
    assert tok.vocab_size < np.iinfo(dtype).max, \
        f"詞彙表 {tok.vocab_size} 超過 {dtype} 上限"

    buf, total = [], 0
    import json
    with open(jsonl_path, encoding="utf-8") as f:
        for line in f:
            text = json.loads(line)["text"]
            ids = tok.encode(text)
            if add_eos:
                ids.append(tok.eos_id)      # 讓模型學會停止（第 7.4.1 節）
            buf.extend(ids)
            total += len(ids)
            if len(buf) > 10_000_000:      # 分批寫出，控制記憶體
                _append(out_path, np.array(buf, dtype=dtype)); buf = []

    if buf:
        _append(out_path, np.array(buf, dtype=dtype))
    print(f"{jsonl_path.name}: {total:,} 個詞元 -> {out_path}")
    return total

def _append(path: Path, arr: np.ndarray):
    with open(path, "ab") as f:
        arr.tofile(f)

```

加上 EOS 是一個關鍵細節。如果每份文件之間沒有分隔，模型會學到「一份文件結束後直接接下一份的開頭」這種奇怪的模式，而且永遠學不會停止。第 7.4.1 節列過這個錯誤，這裡是它的實際發生點。

13.3.2 滑動視窗取樣

formosa/data.py

```

import numpy as np
import torch
from torch.utils.data import Dataset

class TokenDataset(Dataset):
    """從一長串 token id 中切出固定長度的訓練樣本"""
    def __init__(self, bin_path, block_size: int, dtype=np.uint16):
        # memmap：不把整份載入記憶體
        self.data = np.memmap(bin_path, dtype=dtype, mode="r")
        self.block = block_size
        assert len(self.data) > block_size + 1, "語料太短"

    def __len__(self):
        return len(self.data) - self.block - 1

```

```
def __getitem__(self, i):
    # x 與 y 錯開一格；位移在此處或 loss 處擇一，不可兩處都做
    chunk = self.data[i : i + self.block + 1].astype(np.int64)
    return torch.from_numpy(chunk[:-1]), torch.from_numpy(chunk[1:])
```

一個必須做出的選擇

位移可以在資料端做（如上，回傳 x 與 y 已錯開），也可以在損失端做（13.2.1 節的 `shift`）。兩者擇一，絕不可同時做——那會變成錯開兩格，模型永遠學不好，而且症狀不明顯（`loss` 會下降，只是降得比應該的慢）。本書的慣例是：資料端回傳未錯開的同一段，位移統一在 `compute_loss` 中處理。上面的 `Dataset` 是另一種常見寫法，若採用它就必須改用不做位移的損失函式。請在你的專案中明確選定一種並寫進 `README`。

13.3.3 DataLoader 設定

參數	A 級 (CPU)	B 級 (GPU)	理由
<code>batch_size</code>	16	8	受記憶體限制
<code>num_workers</code>	0	4	Windows 需 <code>__main__</code> 保護
<code>pin_memory</code>	False	True	加速 CPU 到 GPU 搬移
<code>drop_last</code>	True	True	避免最後一批形狀不同
<code>shuffle</code>	True	True	打亂取樣起點

`drop_last` 設為 `True` 有一個超出「形狀一致」的理由：某些編譯最佳化（如 `torch.compile`）會為每種輸入形狀重新編譯一次，一個大小不同的批次會觸發額外的編譯，拖慢訓練。這是第 18 章會展開的議題。

13.4 最小可行訓練迴圈

13.4.1 核心結構

一個能用的訓練迴圈需要八個元素。這裡先做最小版本，第 15 章會加上排程、混合精度、監控與續訓的完整處理。

元素	本章的做法	第 15 章會升級為	為什麼現在夠用
取樣	DataLoader 隨機取樣	加入 packing 與課程安排	小語料無此需求
前向與損失	直接呼叫	混合精度	CPU 上無 BF16 優勢
反向	loss.backward()	梯度累積	tiny 批次夠小
更新	AdamW 固定學習率	warmup + cosine 排程	短訓練影響有限
裁切	固定 1.0	動態監控	固定值已足夠安全
評估	每 N 步跑驗證集	分域評測	第 5 章的規格
存檔	每 N 步存一次	含最佳模型追蹤	—
紀錄	JSONL 指標	完整實驗追蹤	第 4.7 節的工具

scripts/train.py — 最小訓練迴圈

```
import json, math, time
from pathlib import Path
import torch
from torch.utils.data import DataLoader

from formosa.config import ModelConfig, TrainConfig
from formosa.model.gpt import MiniFormosaGPT
from formosa.data import TokenDataset
from formosa.utils import set_seed, get_device, build_param_groups
from formosa.logging_setup import setup_logging, JsonlMetricLogger
from formosa.fingerprint import fingerprint

@torch.no_grad()
def evaluate(model, loader, device, max_batches=50):
    model.eval()
    total, n = 0.0, 0
    for i, (x, y) in enumerate(loader):
        if i >= max_batches:
            break
        x, y = x.to(device), y.to(device)
        _, loss = model(x, targets=y)
        total += loss.item() * y.numel(); n += y.numel()
    model.train()
    return total / max(n, 1)

def train(mcfg, tcfg, train_bin, valid_bin, out_dir: Path, resume=None):
    set_seed(tcfg.seed)
    device = get_device()
    log = setup_logging(out_dir, tcfg.run_name)
    metrics = JsonlMetricLogger(out_dir / "metrics.jsonl")

    model = MiniFormosaGPT(mcfg).to(device)
    log.info(f"參數量 {sum(p.numel() for p in model.parameters()):,}")
```



```

opt = torch.optim.AdamW(
    build_param_groups(model, tcfg.weight_decay), # 第 10.2.4 節
    lr=tcfg.lr, betas=(0.9, 0.95))

start_step = 0
if resume is not None:
    start_step = load_checkpoint(resume, model, opt, device)
    log.info(f"從第 {start_step} 步續訓")

train_ds = TokenDataset(train_bin, mcfg.max_seq_len)
valid_ds = TokenDataset(valid_bin, mcfg.max_seq_len)
train_dl = DataLoader(train_ds, batch_size=tcfg.batch_size,
                      shuffle=True, drop_last=True,
                      num_workers=tcfg.num_workers,
                      pin_memory=(device.type == "cuda"))
valid_dl = DataLoader(valid_ds, batch_size=tcfg.batch_size,
                      shuffle=False, drop_last=True)

model.train()
t0, step = time.perf_counter(), start_step
best = float("inf")

while step < tcfg.max_steps:
    for x, y in train_dl:
        if step >= tcfg.max_steps:
            break
        x, y = x.to(device), y.to(device)
        _, loss = model(x, targets=y)

        opt.zero_grad(set_to_none=True)
        loss.backward()
        gnorm = torch.nn.utils.clip_grad_norm_(
            model.parameters(), tcfg.grad_clip)
        opt.step()
        step += 1

    if step % tcfg.log_every == 0:
        tps = step * tcfg.batch_size * mcfg.max_seq_len / \
            (time.perf_counter() - t0)
        metrics.log(step, loss=loss.item(),
                   grad_norm=float(gnorm),
                   tokens_per_sec=round(tps))
        log.info(f"step {step:6d} loss {loss.item():.4f}"
                f" gnorm {float(gnorm):.2f} {tps:.0f} tok/s")

    if step % tcfg.eval_every == 0:
        vl = evaluate(model, valid_dl, device)
        ppl = math.exp(min(vl, 20))
        metrics.log(step, valid_loss=vl, valid_ppl=ppl)
        log.info(f" [eval] loss {vl:.4f} ppl {ppl:.2f}")
        if vl < best:
            best = vl
            save_checkpoint(out_dir / "best.pt", model, opt,
                           step, mcfg, tcfg)

    if step % tcfg.save_every == 0:
        save_checkpoint(out_dir / f"step{step:06d}.pt",
                       model, opt, step, mcfg, tcfg)

metrics.close()

```

```
return model
```

幾個值得注意的地方。betas 用的是 (0.9, 0.95) 而非 PyTorch 預設的 (0.9, 0.999)——語言模型訓練的慣例是把二階動量的衰減調快一些。grad_norm 被記錄下來，因為它是訓練健康度最靈敏的指標（第 3.6.4 節）。最佳模型與定期存檔分開，因為前者用於評估、後者用於續訓。

13.4.2 checkpoint 的內容

formosa/checkpoint.py

```
import torch
from dataclasses import asdict

FORMAT_VERSION = 1

def save_checkpoint(path, model, optimizer, step, mcfg, tcfg, fp=None):
    """光憑這個檔案就能知道它是怎麼來的（第 4.8.3 節）"""
    torch.save({
        "format_version": FORMAT_VERSION,
        "model_state": model.state_dict(),
        "optim_state": optimizer.state_dict(),
        "step": step,
        "model_config": asdict(mcfg),
        "train_config": asdict(tcfg),
        "fingerprint": fp,
        "torch_version": torch.__version__,
    }, path)

def load_checkpoint(path, model, optimizer=None, device="cpu"):
    ck = torch.load(path, map_location=device, weights_only=False)
    assert ck["format_version"] == FORMAT_VERSION, \
        f"checkpoint 格式版本 {ck['format_version']} 與目前 {FORMAT_VERSION} 不符"

    # 設定一致性檢查：避免把權重載入結構不同的模型
    saved = ck["model_config"]
    for key in ("d_model", "n_layers", "n_heads", "vocab_size",
               "activation", "norm_type", "pos_encoding", "tie_weights"):
        cur = getattr(model.cfg, key)
        if saved.get(key) != cur:
            raise ValueError(
                f"設定不符：{key} checkpoint={saved.get(key)} 目前={cur}")

    model.load_state_dict(ck["model_state"])
    if optimizer is not None and ck.get("optim_state"):
        optimizer.load_state_dict(ck["optim_state"])
    return ck["step"]
```

那段設定一致性檢查是本節最重要的部分。第 12.6.2 節說過，載入權重時若元件配置不符，模型不會報錯，只會產生沒有意義的輸出。把檢查寫進載入函式，就能把「靜默失敗」變成「明確的錯誤訊息」——這是把治理要求變成技術強制的又一個例子。

13.5 生成：從模型到文字

13.5.1 自回歸生成的迴圈

訓練時所有位置平行計算（第 5.1.3 節的 teacher forcing），生成時卻必須一個一個產生——因為第 $t+1$ 個詞元的輸入包含第 t 個詞元，而它要等模型算出來才知道。

步驟	做什麼	形狀	注意
1	把提示編碼成詞元	$(1, T_0)$	需與訓練時同一個 tokenizer
2	前向，取最後一格 logits	$(1, 1, V)$	13.1.2 節的最佳化
3	從機率分布選一個詞元	$(1, 1)$	本章用 greedy，第 14 章講取樣
4	接到序列尾端	$(1, T_0+1)$	—
5	回到第 2 步，直到 EOS 或長度上限	—	沒有 EOS 就永遠停不下來

formosa/generate.py — greedy 解碼

```
import torch

@torch.no_grad()
def generate_greedy(model, idx, max_new_tokens: int,
                    eos_id: int | None = None):
    """idx: (B, T) 提示詞元。本章只做 greedy，取樣見第 14 章"""
    model.eval()
    max_len = model.cfg.max_seq_len

    for _ in range(max_new_tokens):
        # 超過上下文上限時裁掉最前面（第 14 章會用 KV cache 改善）
        idx_cond = idx if idx.size(1) <= max_len else idx[:, -max_len:]
        logits = model(idx_cond)  # (B, 1, V)
        next_id = logits[:, -1, :].argmax(dim=-1, keepdim=True)  # (B, 1)
        idx = torch.cat([idx, next_id], dim=1)
        if eos_id is not None and (next_id == eos_id).all():
            break
    return idx

@torch.no_grad()
def stream_generate(model, tok, prompt: str, max_new_tokens=100,
                    eos_id=None):
    """逐字串流輸出，讓使用者立刻看到進度"""
    model.eval()
    device = next(model.parameters()).device
    idx = torch.tensor([tok.encode(prompt)], device=device)
    printed = len(prompt)
```

```

for _ in range(max_new_tokens):
    cond = idx[:, -model.cfg.max_seq_len:]
    nxt = model(cond)[ :, -1, :].argmax(-1, keepdim=True)
    idx = torch.cat([idx, nxt], dim=1)
    text = tok.decode(idx[0].tolist())
    # 只印出新增的部分，避免重複輸出
    print(text[printed:], end="", flush=True)
    printed = len(text)
    if eos_id is not None and int(nxt) == eos_id:
        break
print()
return tok.decode(idx[0].tolist())

```

`stream_generate` 中的 `printed` 追蹤是一個實務細節：因為 `tokenizer` 的解碼是對整串進行的，一個新詞元可能改變前面字元的呈現（例如多位元組字元的組合）。逐字比對已印出的長度，比直接印出新詞元的解碼結果更安全。

13.5.2 為什麼 greedy 生成會重複

用 `greedy` 解碼跑一段，你幾乎一定會看到模型開始重複同一句話。這不是 `bug`，而是兩個因素的疊加。

因素	機制	症狀	第 14 章的對策
greedy 的確定性	每步都選最高機率，沒有跳脫機會	進入循環就出不來	溫度與 top-p 取樣
曝光偏差	訓練時看真實前文，生成時看自己的輸出	錯誤累積、逐漸偏離	偏好對齊（第 25 章）
自我強化	重複的文字提高了再次重複的機率	越重複越想重複	repetition penalty

這是第 5.1.3 節那個抽象概念的具體體驗。當你在本章第一次看到模型不斷重複「的的的的」或某個短句時，那正是曝光偏差在你眼前發生。第 14 章的解碼策略就是為了處理它而存在的。

13.5.3 訓練里程碑：從亂碼到成句

訓練過程中定期生成一段文字，是最直觀的進度指標。以下是 `tiny` 版本在中文語料上的典型軌跡（實際數字依語料與設定而異）。

階段	約略 loss	生成的樣子	模型學到了什麼
初始	約 9.0	完全隨機的字元	什麼都沒有
早期	6 至 7	出現高頻字與標點	字元頻率分布
中期	4 至 5	出現常見詞與詞組	字與字的搭配
後期	3 至 4	短句通順但語意跳躍	局部語法
收斂	2.5 至 3.5	句子通順、開始有主題連貫	較長的依賴關係

(loss 為自然對數， $V = 8,000$ 時初始理論值為 $\ln 8000 \approx 8.99$ 。)

第一列的「約 9.0」正是 13.1.3 節那個測試的理論值。如果你的初始 loss 不在這附近，先不要開始訓練——去檢查初始化。

對 tiny 模型該有的期待

一個 2 M 參數、在數千萬詞元上訓練的模型，能做到的極限大概是「生成通順但語意不連貫的短句」。它不會回答問題、不會有知識、也不會遵守指令——那些能力需要更大的規模與後訓練（第六篇）。本章的成功標準是「文字看起來像中文」，不是「模型很有用」。把期待設定對，你才不會誤以為自己做錯了。

13.6 八種常見的訓練失敗

這一節是本章最實用的部分。建議在開始訓練前先讀過，遇到問題時再回來查。

症狀	最可能的原因	診斷方式	章節
loss 不動（維持在 $\ln V$ ）	梯度沒有流動	印出 grad_norm；檢查參數註冊	第 4.2.3 節
loss 迅速降到近 0	答案洩漏	檢查位移對齊與因果遮罩	13.2.1、第 11.4 節
loss 變成 NaN	學習率過大或數值溢位	看 NaN 前幾步的 grad_norm	第 3.6.3 節
loss 下降後突然暴增	梯度尖峰	啟用梯度裁切；降學習率	第 3.6.4 節
驗證 loss 低於訓練 loss	dropout 或切分問題	確認 eval() 模式；檢查資料重疊	第 5.4 節

症狀	最可能的原因	診斷方式	章節
訓練 loss 降、 驗證 loss 升	過擬合	增加資料或正則化	第 12.7.2 節
生成永遠不停	沒有 EOS 或未學到	確認語料編碼時加了 EOS	13.3.1、第 7.4.1 節
生成一直重複	greedy 加曝光偏差	正常現象，改用取樣	13.5.2、第 14 章

13.6.1 診斷的正確順序

遇到問題時，按成本由低到高依序檢查，不要一開始就懷疑演算法。

1. 先看 `grad_norm`。它是零嗎？是 NaN 嗎？還是暴增？這一個數字就能區分掉一半的問題。
2. 跑一次 `overfit_one_batch`。如果連一個批次都背不起來，問題在模型實作，不在資料或超參數。
3. 檢查初始 loss 是否接近 $\ln(V)$ 。偏離代表初始化或損失計算有問題。
4. 手動檢查一個樣本的位移對應（13.2.2 節的 `inspect_shift`）。
5. 確認 `model.train()` 與 `model.eval()` 在正確的位置。
6. 把學習率降十倍再跑一次。如果問題消失，那是最佳化的問題而非實作問題。
7. 最後才懷疑資料。印出幾個實際的批次，用 `tokenizer` 解碼回文字看看。

第七步排在最後不是因為資料不重要，而是因為它最耗時。前六步加起來不到十分鐘，而且能排除大部分情況。這個順序本身就是一種工程紀律。

13.6.2 一個真實的除錯案例

以下是一個綜合了多次課堂經驗的典型情境，值得完整走一遍。

步驟	觀察	推論	行動
1	loss 從 9.0 降到 0.8，只花了 300 步	太快了，不合理	懷疑答案洩漏
2	<code>grad_norm</code> 正常，沒有異常	不是梯度問題	繼續查
3	<code>overfit_one_batch</code> 通過	模型結構應該沒問題	轉向資料與損失
4	<code>inspect_shift</code> 顯示 input 與 predicts 相同	找到了：位移做了兩次	檢查 Dataset 與 loss

步驟	觀察	推論	行動
5	Dataset 回傳已錯開的 x/y · loss 又錯開一次	確認	擇一移除

這正是 13.3.2 節那個提醒的實際後果。兩處都做位移，等於錯開兩格——模型被要求用第 t 個詞元預測第 $t+2$ 個。奇怪的是 loss 仍會下降（因為中文有一定的可預測性），只是降得比應該的慢，而且生成品質很差。這種「錯了但還是會動」的 bug 最難察覺，只能靠 `inspect_shift` 這類手動檢查抓到。

13.7 在筆電上跑完一次完整訓練

13.7.1 三種尺寸的訓練預算

把第 12 章的三種尺寸接上訓練時間估算。用第 3.2.5 節的 6ND 公式，並假設訓練詞元數為參數量的 20 倍。

尺寸	參數量	訓練詞元	總 FLOPs	硬體需求
tiny	2.07 M	41 M	0.52 PFLOPs	A 級可行
small	42.08 M	842 M	212.5 PFLOPs	B 級
base	109.5 M	2,191 M	1,440 PFLOPs	C 級

（tiny 使用 $V = 8,000$ 的小詞彙表，見第 12 章系統案例。）換算成時間：

配置	裝置	估計時間	備註
tiny	CPU (約 0.1 TFLOPS)	約 2.8 小時	一個晚上跑得完
tiny	單張消費級 GPU	約 1 分鐘	幾乎瞬間
small	CPU	約 1,181 小時	不可行
small	單張消費級 GPU	約 6.6 小時	一天內可完成
small	資料中心級 GPU	約 0.5 小時	—

（假設 CPU 的 MFU 為 0.5、消費級 GPU 為 30 TFLOPS 且 MFU 0.30。實際數字請用第 4 章程式 4B

量測你自己的裝置。)

這張表傳達的訊息很明確：**tiny** 版本在筆電上是完全可行的，而 **small** 以上需要 GPU。這也是為什麼本書把 **tiny** 設計成 A 級路線的完整目標——沒有 GPU 的學生一樣能走完整條流程，只是模型小一點。

13.7.2 A 級路線的完整步驟

1. 確認第 12 章的八項檢查全部通過。
2. 用第 7 章的 tokenizer (V = 8,000 版本) 編碼第 9 章的語料，產出 train.bin 與 valid.bin。
3. 跑 overfit_one_batch，確認能收斂到 0.05 以下。
4. 用 tiny 設定啟動訓練，max_steps 設為約 20,000 步 (批次 16、序列 128，約 41 M 詞元)。
5. 每 500 步評估一次，每 2,000 步存檔一次，並生成一段文字記錄下來。
6. 訓練結束後，用最佳 checkpoint 生成十段文字，與訓練里程碑表對照。
7. 計算最終的困惑度、BPC 與 tokens_per_char (第 5.3.5 節的三數字慣例)。

第五步的「生成一段文字記錄下來」值得堅持。把不同步數的生成結果排在一起，你會看到模型從亂碼演化成句子的完整過程——那是這門課最有成就感的產出之一，也是很好的展示素材。

13.7.3 該期待什麼樣的結果

指標	tiny 的合理範圍	意義	參照
驗證 loss	3.0 至 4.5	對應困惑度 20 至 90	第 5.3.1 節
BPC	2.0 至 3.5	可跨 tokenizer 比較	第 5.3.5 節
生成品質	短句通順、語意跳躍	符合模型規模	13.5.3
訓練時間	2 至 4 小時 (CPU)	一個晚上	13.7.1

如果你的驗證 loss 降到 1.5 以下，先不要高興——那很可能是資料洩漏或語料重複 (第 9.3 節)。用第 5.6 節的污染偵測與第 16 章的記憶化檢查確認一下。第 5.3.1 節那張表說過：太好的數字應該先懷疑。

程式實作

本章的三個實作是第 3 篇的收束。完成之後，你手上會有一個真正屬於自己的、能生成繁體中文的模型。

程式 13A MiniFormosaGPT 完整實作與端到端訓練

目標：把第 10 到 12 章的元件組裝成完整模型，接上資料，跑完一次訓練。

交付項	內容	驗收標準	節次
模型	MiniFormosaGPT 類別	六項端到端測試通過	13.1.3
損失	compute_loss 與 sum 版本	位移正確、初始 loss $\approx \ln V$	13.2
資料	prepare_data 與 TokenDataset	編碼可逆、含 EOS	13.3
訓練	train.py	可中斷續訓、結果一致	13.4
checkpoint	save/load 含設定檢查	設定不符時明確報錯	13.4.2
完整訓練	tiny 版跑完 20,000 步	有訓練曲線與生成樣本	13.7

五項必須通過的驗收：

- overfit_one_batch 收斂到 0.05 以下，且初始 loss 與 $\ln(V)$ 的差距小於 0.6。
- 端到端因果性測試通過：改變最後一個輸入詞元，前面所有位置的 logits 完全不變。
- 中斷後續訓，loss 曲線接得上，不會跳回起點。
- 故意把 checkpoint 載入一個 d_model 不同的模型，應該拋出明確的錯誤訊息而非靜默失敗。
- 同一個 seed 連跑兩次，前 200 步的 loss 序列在 CPU 上完全相同。

程式 13B 生成器 CLI

目標：做一個可以互動使用的命令列生成工具，支援提示輸入、長度控制與串流輸出。

scripts/generate.py

```
import argparse
from pathlib import Path
import torch
from formosa.config import ModelConfig
from formosa.model.gpt import MiniFormosaGPT
from formosa.generate import stream_generate
from formosa.tokenizer import load_tokenizer
from formosa.checkpoint import load_checkpoint
```

```

def main():
    ap = argparse.ArgumentParser(description="MiniFormosaGPT 文字生成")
    ap.add_argument("--ckpt", type=Path, required=True)
    ap.add_argument("--tokenizer", type=Path, required=True)
    ap.add_argument("--prompt", type=str, default=None)
    ap.add_argument("--max-new-tokens", type=int, default=100)
    ap.add_argument("--interactive", action="store_true")
    args = ap.parse_args()

    tok = load_tokenizer(args.tokenizer)
    ck = torch.load(args.ckpt, map_location="cpu", weights_only=False)
    cfg = ModelConfig(**ck["model_config"])
    model = MiniFormosaGPT(cfg)
    load_checkpoint(args.ckpt, model)
    print(f"已載入 step {ck['step']}，參數量 "
          f"{sum(p.numel() for p in model.parameters()):,}")

    if args.interactive:
        print("輸入提示（空行結束）：")
        while True:
            try:
                prompt = input("\n> ").strip()
            except (EOFError, KeyboardInterrupt):
                break
            if not prompt:
                break
            stream_generate(model, tok, prompt,
                           args.max_new_tokens, tok.eos_id)
    else:
        stream_generate(model, tok, args.prompt or "臺灣",
                       args.max_new_tokens, tok.eos_id)

if __name__ == "__main__":
    main()

```

延伸挑戰：加入一個 `--checkpoint-sweep` 選項，接受一個目錄，對其中所有的 checkpoint 用同一個提示各生成一段，並排輸出。這會產生一份「模型如何逐步學會中文」的完整紀錄，是很好的展示素材，也是第 16 章實驗報告的好題材。

程式 13C 訓練健檢腳本

目標：把 13.6 節的失敗模式表變成一支可執行的診斷工具，在訓練前與訓練中自動偵測問題。

scripts/health_check.py (節錄)

```

import math, torch

class TrainingSentinel:
    """訓練中的即時哨兵，偵測 13.6 節的八種失敗"""

    def __init__(self, vocab_size, window=50, spike_factor=3.0):
        self.expected_init = math.log(vocab_size)

```

```

self.losses, self.gnorms = [], []
self.window, self.spike = window, spike_factor
self.warnings = []

def step(self, step, loss, grad_norm):
    self.losses.append(loss); self.gnorms.append(grad_norm)
    w = []

    if step == 1 and abs(loss - self.expected_init) > 0.6:
        w.append(f"初始 loss {loss:.3f} 偏離 ln(V)="
                f"{self.expected_init:.3f}，檢查初始化與損失計算")

    if math.isnan(loss) or math.isinf(loss):
        w.append("loss 為 NaN/Inf：降學習率、檢查梯度裁切")

    if step > 200 and loss < 0.1:
        w.append("loss 異常低：檢查位移對齊與因果遮罩（答案洩漏）")

    if len(self.losses) > self.window:
        recent = self.losses[-self.window:]
        base = sorted(recent)[len(recent) // 2]
        if loss > base * self.spike:
            w.append(f"loss 尖峰：{loss:.3f} 遠高於近期中位數 {base:.3f}")

    if step > 100 and max(self.losses[-100:]) - \
        min(self.losses[-100:]) < 0.01:
        w.append("loss 停滯：檢查梯度是否流動、學習率是否為 0")

    if grad_norm == 0.0:
        w.append("grad_norm 為 0：參數可能未被 optimizer 追蹤")

    for msg in w:
        self.warnings.append({"step": step, "msg": msg})
        print(f"[哨兵 step {step}] {msg}")

    return w

```

把這個哨兵接進訓練迴圈，它會在問題發生的當下就告訴你，而不是讓你在六小時後看著一條詭異的曲線猜測。第 15 章的程式 15C 會把它擴充成能自動回滾到前一個 checkpoint 的完整版本。

系統案例：從亂碼到成句的訓練縮時紀錄

本章的系統案例是製作一份完整的訓練歷程紀錄：用臺灣的中小學課文與百科語料訓練 tiny 版模型，記錄從亂碼到成句的每一個里程碑，並做成可展示的縮時素材。

記錄什麼

類別	記錄內容	頻率	用途
指標	loss、grad_norm、tokens/sec	每 50 步	訓練曲線
評估	驗證 loss、PPL、BPC	每 500 步	泛化追蹤
生成	同一組提示的生成結果	每 500 步	質性進展
注意力	同一句話的熱圖	每 2,000 步	看模型如何學會關注
嵌入	同一組詞的語意地圖	每 5,000 步	看語意空間如何成形

後兩列把第 10、11 章的視覺化工具接了進來。這是本篇四章的整合展示：你可以看到嵌入空間從隨機分布逐漸聚出臺灣地名的群、注意力從均勻分布逐漸學會關注標點與詞界。這比任何靜態的說明都有說服力。

固定的探針提示

為了讓不同步數的生成可以比較，必須使用固定的提示集。建議至少五個：

提示	測什麼	早期預期	後期預期
「臺灣」	最常見的詞後面接什麼	亂碼	接得出地理或形容
「今天天氣」	常見搭配	亂碼	「很好」「不錯」之類
「第一條 本要點」	公文語體	亂碼	出現法規句式
「小明對小華說：」	對話結構	亂碼	出現引號與對話
「1234567890」	數字處理	亂碼	延續數字或轉為文字

交付要求

- 完整的訓練紀錄：metrics.jsonl、log 檔、以及至少 40 個時間點的生成樣本。
- 訓練曲線圖：loss、驗證 loss、grad_norm 三條，標出關鍵事件（尖峰、評估點）。
- 里程碑對照表：依 13.5.3 節的格式，附上你實際觀察到的 loss 與生成樣本。
- 縮時素材：把不同步數的生成結果做成可播放的序列（GIF、影片或並排圖皆可）。

- 三數字報告：最終的 PPL、BPC 與 tokens_per_char (第 5.3.5 節)。
- 一頁反思：哪一個階段的進展最讓你意外？如果重來一次，你會改變什麼設定？

評分重點

- 訓練是否真的跑完並可重現。助教會用你的 checkpoint 與腳本重新生成，結果須一致。
- 生成樣本是否誠實。不可只挑最好的幾段——必須包含各階段的實際輸出，含糟糕的那些。
- 是否用三數字報告而非只有 PPL。
- 反思的品質。「都很順利」不予計分；請具體說出你踩到的坑與怎麼發現的。

章末評量

一、概念題

1. 為什麼生成時 forward 只需要計算最後一個位置的 logits？這省下了什麼？
2. RoPE 快取為什麼要用 register_buffer 且設 persistent=False？
3. 為什麼未訓練模型的初始 loss 應該接近 $\ln(V)$ ？偏高與偏低各代表什麼問題？
4. 位移可以在資料端或損失端做。為什麼絕不能兩處都做？兩處都做的症狀是什麼？
5. 「過擬合單一批次」為什麼是最有效的第一道檢查？它能排除哪些問題、不能排除哪些？
6. 為什麼語料編碼時要在每份文件後加 EOS？不加會有什麼後果？
7. greedy 生成為什麼會重複？請從三個因素分別說明。
8. checkpoint 中的設定一致性檢查在防止什麼？為什麼這類錯誤特別危險？

二、計算題

1. 一個 $V = 8,000$ 的模型，未訓練時的理論初始 loss 是多少？對應的困惑度是多少？
2. tiny 設定（2.07 M 參數），若訓練詞元數為參數量的 20 倍，總共需要多少詞元？用 6ND 公式計算總 FLOPs。
3. 承上題，在一台實測 0.1 TFLOPS、MFU 0.5 的 CPU 上，估計需要多少小時？
4. 批次 16、序列長度 128，跑 20,000 步。總共處理了多少詞元？若語料只有 10 M 詞元，相當於跑了幾個 epoch？
5. 某次訓練的驗證 loss 為 3.4，語料的 tokens_per_char 為 1.2。計算 PPL 與 BPC。
6. small 版本（42.08 M 參數、842 M 詞元）在 30 TFLOPS、MFU 0.30 的 GPU 上需要多久？若改用 MFU 0.45 呢？

三、除錯題

1. 訓練啟動後 loss 立刻是 NaN，還沒跑到第一次評估。請列出四個檢查項目與順序。
2. 模型訓練得很好（驗證 loss 3.2），但生成的文字每次都完全相同，且只有幾個字就停了。請診斷。
3. 同學的訓練在第 8,000 步 OOM，前面都正常。請列出三個可能原因。
4. 同學把 checkpoint 載入後繼續訓練，loss 從 3.2 跳到 8.9。請列出三個可能原因。
5. 生成的文字中夾雜大量替換字元（方框或問號）。請說明可能的原因（提示：與第 6、7 章有關）。

6. 同學的訓練速度只有預期的十分之一，GPU 使用率顯示 20%。請列出三個可能的瓶頸。

四、系統設計題

1. 你只有一台 8 GB 記憶體、無獨立顯卡的筆電，要在一週內完成一次有意義的訓練並產出可展示的成果。請規劃：模型尺寸、詞彙表、語料規模、訓練步數、時間分配，以及你如何確保「跑得完」。並說明你會如何向沒有 GPU 的同學證明這條路線是完整的。
2. 你要設計一套「訓練健檢流程」，讓學生在按下訓練之前必須通過。請列出至少八項檢查、各自的通過標準、以及每一項若失敗時的診斷指引。這份流程要能在十分鐘內跑完。

五、臺灣情境與倫理題

1. 你的 tiny 模型生成了一段看起來像真實公文的文章，但內容完全是編造的。請討論：在展示這個成果時，你有什麼責任？如果它被截圖流傳，可能造成什麼問題？你會如何標示？
2. 本章的模型在訓練後可能會逐字複述語料中的某些片段。請說明如何檢查這件事（連結第 9.3 節與第 16 章），以及若你的語料含有第 8 章討論的敏感內容，這代表什麼風險。

評量提示（給自學者）

- 計算題第 1 題： $\ln(8000) \approx 8.987$ ；困惑度即 8,000（等於詞彙表大小，代表完全隨機）。
- 計算題第 2 題：41.4 M 詞元；FLOPs = $6 \times 2.07e6 \times 41.4e6 \approx 5.14 \times 10^{14}$ ，約 0.51 PFLOPs。
- 計算題第 3 題： $5.14e14 \div (0.1e12 \times 0.5) \approx 10,280$ 秒 ≈ 2.86 小時。
- 計算題第 4 題： $16 \times 128 \times 20,000 = 40.96$ M 詞元；約 4.1 個 epoch。
- 計算題第 5 題： $PPL = \exp(3.4) \approx 29.96$ ； $BPC = 3.4 \times 1.2 \div \ln 2 \approx 5.89$ 。
- 計算題第 6 題： $6 \times 42.08e6 \times 842e6 \div (30e12 \times 0.30) \div 3600 \approx 6.56$ 小時；MFU 0.45 時約 4.37 小時。
- 除錯題第 2 題：模型可能立刻生成了 EOS。檢查 EOS 在語料中的比例是否過高（例如文件太短），或提示的編碼是否有誤。
- 除錯題第 4 題：optimizer 狀態未載入、學習率排程重置、或載入的是不同設定的 checkpoint（若無一致性檢查則會靜默出錯）。

研究延伸與版本注意事項

- nanoGPT 這類極簡實作是本章的最佳對照：讀完本章後去看它，你會發現自己已經能逐行讀懂，並認出每一個設計決策的理由。
- Raschka 的《Build a Large Language Model (From Scratch)》第 4 到 5 章與本章高度對應，可

作為第二份參考，特別是它對每一步的圖解。

- GPT-2 的原始實作：雖然元件較舊（LayerNorm、可學習位置嵌入），但它是許多後續程式碼的祖先，值得讀過一次。
- 訓練動態的研究：關於 loss 曲線的階段性、突現行為與記憶化的時機，近年有不少實證研究，與第 16 章的實驗科學直接相關。
- 本章的程式碼結構會被第 14、15、22、23 章持續擴充。請把它放進第 4 章建立的套件中，而非留在 ch13/ 目錄。

常見問題

問：我的 tiny 模型生成的東西還是很爛，是不是哪裡做錯了？答：先對照 13.7.3 節的期待表。一個 2 M 參數的模型能做到「短句通順、語意跳躍」就已經達標。如果你的驗證 loss 在 3.0 到 4.5 之間、生成的字是真的中文字而非亂碼，那就是成功的。「有用」是第六篇之後的事。

問：可以用更大的語料讓 tiny 模型變好嗎？答：邊際效益會遞減。第 17 章的尺度定律會給出精確的答案，但直覺是：模型容量有上限，餵再多資料也裝不進去。與其加資料，不如在你的硬體允許範圍內把模型加大一點。

問：訓練跑到一半電腦要關機怎麼辦？答：這正是 checkpoint 存在的理由。確認你的 save_every 設定合理（tiny 版建議 2,000 步），關機前的最後一個 checkpoint 就是你的進度。續訓時用 --resume 指向它即可——而這也是程式 13A 第三項驗收要測的事。

問：為什麼本章不做 KV cache？答：因為它是一個純粹的推論最佳化，與「模型能不能訓練起來」無關。把它留到第 14 章，可以讓本章專注在組裝與除錯上。而且第 14 章加上 KV cache 時，你會有一個已經能運作的 baseline 可以對照——這比一開始就寫複雜版本要好除錯得多。

教師使用建議

本章排在第 13 週，建議 2 小時講授加 1 小時實驗，並強烈建議額外安排一次實作工作坊。本章是全書最容易卡住的一章，但也是成就感最高的一章。

時段	內容	互動設計	注意事項
前 20 分鐘	13.1 節，組裝與端到端測試	現場執行六項測試	初始 loss $\approx \ln V$ 的檢查要示範

時段	內容	互動設計	注意事項
中間 25 分鐘	13.2 節・損失與位移	用 inspect_shift 帶全班看一次對應	位移是最常見的錯誤
後 25 分鐘	13.3 至 13.4 節・資料與訓練迴圈	現場啟動一次 tiny 訓練	讓學生看到 loss 開始下降
最後 40 分鐘	13.5 至 13.7 節・生成與失敗模式	展示不同步數的生成結果	這一段的效果最好
工作坊 2 小時	各組實際跑通訓練	助教巡迴協助除錯	目標是每組都跑起來

一個效果極佳的開場：課前先用 tiny 設定跑好一次訓練，把第 0、500、2000、5000、20000 步的生成結果做成投影片，一頁一段。上課時一頁一頁翻，讓學生看著亂碼逐漸變成句子。多數學生在這一刻會第一次真正相信「這東西是可以自己做出來的」。

關於工作坊：本章的除錯量遠高於前面各章，建議安排額外時段並準備好 13.6 節的失敗模式表作為助教的診斷手冊。經驗上，八成的問題落在三處——位移做了兩次、忘記加 EOS、以及 DataLoader 在 Windows 上的 num_workers 問題（第 4.5.2 節）。事先準備好這三題的解法，可以省下大量時間。

高中授課的調整：13.1 提供半成品讓學生填空（把 forward 的六個步驟留白）；13.2 完整教，位移的概念用 inspect_shift 的輸出來講最清楚；13.3、13.4 提供現成腳本，只要求理解與執行；13.5 完整教（生成是最有趣的部分）；13.6 挑三個最常見的講；13.7 讓學生實際在自己的筆電上跑 tiny 版——這是整門課的高潮，務必保留足夠時間。

本章小結

1. 完整模型由六層構成：嵌入、dropout、N 個 block、最終正規化、輸出投影、損失或取樣。RoPE 快取全模型共用一份。
2. 生成時 forward 只需計算最後一格的 logits，這省下整個模型最貴的矩陣乘法之一。
3. 未訓練模型的初始 loss 應接近 $\ln(V)$ ；偏高代表初始化尺度問題，偏低代表答案洩漏。
4. 位移對齊必須在資料端或損失端擇一，兩處都做會錯開兩格且症狀不明顯。
5. 「過擬合單一批次」是最有效的第一道檢查，它同時驗證初始化與梯度流動。
6. 語料編碼時每份文件後要加 EOS，否則模型永遠學不會停止。
7. 訓練迴圈的八個元素中，grad_norm 是健康度最靈敏的指標，應該從第一天就記錄。

8. checkpoint 必須包含設定，並在載入時做一致性檢查，把靜默失敗變成明確錯誤。
9. greedy 生成必然重複，這是確定性、曝光偏差與自我強化三者疊加的結果，屬於正常現象。
10. 訓練里程碑從亂碼、高頻字、常見詞、短句到連貫，對應 loss 從 9 降到 3 左右。
11. 八種常見失敗各有明確的診斷方式；診斷順序應由成本低到高，grad_norm 先看、資料最後查。
12. tiny 版本在一般筆電上約 2 到 4 小時可訓練完成，是完整的 A 級路線，不是簡化版。
13. 對 tiny 模型的合理期待是「短句通順、語意跳躍」；驗證 loss 低於 1.5 應先懷疑資料洩漏。

成果檢核

完成本章後你必須繳交一個可執行的 MiniFormosaGPT 專案：

- 完整原始碼：模型、損失、資料、訓練、生成、checkpoint，全部放在第 4 章的套件結構中。
- 測試套件：端到端六項測試 + overfit_one_batch，全部通過。
- 訓練權重：至少一個可載入的 checkpoint，含設定與環境指紋。
- 訓練曲線：loss、驗證 loss、grad_norm 三條，以及 metrics.jsonl 原始資料。
- 生成樣本：至少 40 個時間點、五個固定提示的輸出，含各階段的實際結果（不可只挑好的）。
- 三數字報告：最終 PPL、BPC 與 tokens_per_char。
- 一頁式 Model Card 草案：模型規格、訓練資料、已知限制、以及明確的「不建議用途」。
- 除錯紀錄：你實際遇到的問題與如何發現的，至少三則。

第 3 篇到此結束。回頭看，你在第 10 章寫了嵌入與 RoPE、第 11 章寫了注意力、第 12 章寫了 block、本章把它們接起來訓練出一個會生成繁體中文的模型。這四章的程式碼是全書的骨幹——第 14 章會為它加上解碼策略，第 15 章讓它能穩定訓練更久，第 22 章之後你會在別人的模型上做同樣的事，而那時候你會知道每一層裡面到底發生了什麼。

最後一件事：把你第一次看到模型生出通順句子的那個生成樣本存起來。那是一個值得記住的時刻。

索引

依詞條首字排序，數字為出現的章次。英文詞條排在中文詞條之前。索引依本冊內容編製；跨篇的完整索引見全書合訂本。

英文詞條

AdamW	10、12、13	MHA	11、12
ALiBi	10	MiniFormosaGPT	10、11、12、13
base	10、12、13	MQA	11
baseline	13	one-hot	10、11
BERT	10、12	PCA	10
BF16	10、11、12、13	Post-LN	12
BPC	13	Pre-LN	12
BPE	11	PyTorch	10、11、13
checkpoint	12、13	Query	10、11
d_ff	11、12	RAG	10
DataLoader	13	RMSNorm	12
dropout	11、12、13	RNN	11
EOS	13	RoPE	10、11、12、13
FlashAttention	11	SiLU	12
FLOPs	11、12、13	small	12、13
GELU	12	softmax	10、11、12
GPT	10、11、12、13	SwiGLU	12
GQA	11、12	t-SNE	10
grad_norm	13	teacher forcing	13
greedy 解碼	13	tiny	12、13
Key	10、11	tokenizer	10、11、12、13
KV cache	10、11、12、13	Transformer	10、11、12、13
LayerNorm	10、12、13	UMAP	10
Llama-style	10、12	Value	11
loss mask	10、13	warmup	12、13
LSTM	12	Word2Vec	10
MFU	12、13		

中文詞條

上下文長度	10	參數對齊	12
-------	----	------	----

子詞平均	10	張量	11
分布式假說	10	情境化表示	10
反向傳播	10、11、12	排列等變	10
尺度定律	13	梯度	10、11、12、13
可重現	13	梯度捷徑	12
可學習位置嵌入	10、13	梯度累積	13
正弦位置編碼	10	梯度裁切	13
正規化	10、11、12、13	混合精度	12、13
交叉熵	10	設定一致性檢查	13
各向異性	10	設定檔	12
因果遮罩	11、13	嵌入	10、11、12、13
污染	13	殘差流	12
自注意力	11	殘差連接	10、11、12、13
串流生成	13	評測	10、12、13
位移對齊	13	詞元	10、11、12、13
位置內插	10	詞彙擴充	10
位置編碼	10、11、12	開放權重	10、11
困惑度	13	微調	10、12、13
批次大小	12	滑動視窗	11、13
注意力	10、11、12、13	滑動視窗取樣	13
注意力沉沒	11	過擬合單一批次	13
注意力熵	11	算力	11、12
長度外推	10	語意地圖	10、13
門控	12	線上 softmax	11
前饋網路	11、12	餘弦相似度	10
查表	10	學習率	10、12、13
相對位置	10、12	隨機種子	12
消融	12	環境指紋	10、13
訓練里程碑	13	縮放點積	11
訓練預算	12、13	繁體中文	10、11、12、13
記憶體牆	11	曝光偏差	13
參數分組	10、12	續訓	10、13
參數量	10、11、12、13	權重綁定	10、12

大模型導論 Introduction to Large Language Models

第 3 篇 Transformer 從零實作 (第 10–13 章)

編著 李世淵 (機器人叫獸) | 助教 李天宇、李宇晴

課程與勘誤 : @prof (<https://page.line.me/prof>)

臺灣自編大學教科書系列 | 2026 年 8 月 第一版